

Reservoir Computing Diffusion Probabilistic Models for Time Series Forecasting

Jacopo Rauseo

21-327-663

University of St.Gallen

Master of Arts in Quantitative Economics and Finance

Supervisor: Prof. Ph.D. Lyudmila Grigoryeva

Co-Supervisor: Prof. Ph.D. Matthias R. Fengler

Acknowledgments

I would like to thank my supervisor, Prof. Lyudmila Grigoryeva, for all the time she has devoted to me. Her support and guidance made me really enjoy all my months of thesis writing and conception. I would also like to thank Dr Giovanni Ballarin, who first came up with the idea of combining Reservoir Computing with Denoising Diffusion Probabilistic Models. His support during the writing of this thesis have been of great help to me and the result would be much worse without his support. I am also grateful to my co-supervisor, Prof. Matthias R. Fengler.

Finally, I would like to thank my family and my girlfriend for the support they have given me throughout my Master's studies.

CONTENTS

Abstract	1
1 Introduction	2
2 Background: Time Series Distributional Forecasting	5
3 Diffusion Models	7
3.1 Denoising Diffusion Probabilistic Models	7
3.2 Denoising Networks	14
3.3 Score-Based Generative Models	16
3.4 Conditional Denoising Diffusion Probabilistic Models	17
3.5 Diffusion Models in Continuous Time	18
4 Time Series Diffusion	22
4.1 Problem Formulation	22
4.2 Time Series Autoregressive Denoising Diffusion Models (TimeGrad)	22
5 Diffusion Models and Reservoir Computing	27
5.1 Echo State Networks	27
5.2 Time Series Diffusion Model and ESNs (ESNGrad)	28
5.3 Full Reservoir Computing Time Series Diffusion	30
6 Evaluation on Simulated Time Series	35
6.1 A Generative Study of an AR(1) Process	36
6.2 A Generative Study of a GARCH(1,1) Process	40
7 An Application to Economic Data	43
7.1 Data and Setting	43
7.2 Results	45
8 Conclusion	49
References	51

Appendices	i
A Additional Material	i
A.1 Forward and Reverse Processes in DDPM	i
A.2 Decomposition of the VLB	i
A.3 Reverse distribution based on the noise	ii
A.4 Convergence Criteria	iii
A.5 Random Weight Denoising Networks	iv
A.6 Additional Algorithms	v
B Figures and Tables	vii
C Additional Figures	xvii
C.1 Model Architectures	xvii
D Declaration of Auxiliary tools	xix
E Disclosures	xx
F Declaration of Authorship	xxi

Abstract

Autoregressive diffusion models have recently emerged as an interesting framework for probabilistic time series forecasting and generation, offering a flexible and nonparametric approach to learning complex conditional distributions. However, such models rely on recurrent structures trained via backpropagation through time, which are computationally demanding and prone to noisy gradients. Reservoir Computing models offer an efficient training paradigm: they reduce the computational cost by fixing most of the parameters at random and allowing only a small set to be estimated, often in closed form. In this thesis, we propose ESNGrad and RWDN, two novel models based on the autoregressive diffusion framework for time series modeling, in which trainable recurrent components are replaced by Reservoir Computing architectures. ESNGrad substitutes the Recurrent Neural Network with an Echo State Network, while RWDN substitutes the deep neural network component with Random Weight Neural Networks, reducing estimation to a sequence of closed form least-squares problems and eliminating the need for gradient-based optimization. We evaluate the generative performance of our proposed models on simulated data by comparing estimated conditional distributions to the ground truth via the Wasserstein distance, and we assess predictive accuracy on real-world univariate U.S. CPI data. Both models achieve forecasting accuracy and generative performance comparable to TimeGrad, a state-of-the-art autoregressive diffusion model, while requiring only a fraction of the training time. In our experiments, ESNGrad reduces training time by approximately 80% relative to TimeGrad, while RWDN achieves reductions exceeding 95%. These results suggest that combining diffusion-based generative modeling with Reservoir Computing provides an efficient and competitive alternative for probabilistic time series analysis.

Keywords: *Time Series Analysis, Denoising Diffusion Probabilistic Models, Probabilistic Forecasting, Reservoir Computing.*

1 Introduction

Quantifying risk has become a central topic in economics and finance. In an uncertain world, policymakers and economists are increasingly interested in assessing the likelihood of adverse events rather than simply relying on point forecasts. A natural framework for doing so is to model the conditional distribution of a target variable, which provides more information than the conditional mean alone. In particular, it enables the characterization of tail risk, the assessment of extreme outcomes, and the construction of prediction intervals. Two prominent applications of this approach in macroeconomics are Growth-at-Risk (GaR) (Adrian et al. 2019), which studies the lower tail of the conditional distribution of GDP growth to evaluate how financial conditions affect the risk of economic slowdowns, and Inflation-at-Risk (IaR) (López-Salido and Loria 2024), which models the conditional distribution of inflation to assess both downside and upside inflation risks. Beyond risk assessment, knowledge of the full conditional distribution plays a central role in scenario analysis, stress testing, and synthetic data generation, making probabilistic forecasting a valuable tool for both policymakers and researchers.

Traditional econometric approaches to probabilistic forecasting are typically based on parametric assumptions about the data-generating process. Density forecasting has emerged, in particular, in financial risk management, where modeling the full conditional distribution of returns is essential for computing risk measures such as Value-at-Risk and Expected Shortfall (Diebold et al. 1998; Tay and Wallis 2000). Early parametric approaches modeled the conditional distribution using ARCH and GARCH-type models (Engle 1982; Bollerslev 1986), which capture volatility but impose strong distributional assumptions. Bayesian methods offer a natural alternative for density forecasting, as they produce a full posterior predictive distribution. In macroeconomics, Bayesian Vector Autoregressions (BVARs) have become the standard reference for joint density forecasting of multivariate time series (Litterman 1986; Geweke and Amisano 2010). Simulation-based methods such as particle filters (Gordon et al. 1993) and Markov Chain Monte Carlo (Geweke 1989) further extend this framework to nonlinear and non-Gaussian state space models. Bootstrap-based methods provide a nonparametric alternative, constructing prediction intervals without distributional assumptions (Pascual et al. 2006). On the other hand, IaR and GaR generally rely on quantile regression methods, where only particular quantiles of the conditional distribution are modeled rather than the full conditional one (Adrian et al. 2026). While quantile regressions are flexible and distribution-free, they model each quantile independently, which can lead to quantile crossing and does not always deliver a coherent conditional distribution. While these approaches are generally interpretable and computationally efficient, they may struggle to capture complex nonlinear dynamics or to deal with non stationary time series. Recent advances in deep generative modeling have introduced flexible tools for probabilistic forecasting and synthetic data generation. In particular, diffusion models, originally developed for image generation (Ho et al. 2020), have recently emerged as a promising framework for time series analysis.

Diffusion models are inspired by non-equilibrium thermodynamics and define a generative process through the gradual corruption of data with Gaussian noise (Sohl-Dickstein et al. 2015). The reverse denoising process is learned by training a deep neural network to predict the noise added to the original sample. For this reason, the network is commonly referred to as a *denoising network* (Ho et al. 2020). Through such a process, diffusion

models implicitly learn the probability distribution of the data by approximating the reverse variational distribution, allowing them to generate samples that share the same statistical properties as the true data-generating process. The rise of generative AI has spurred a large number of applications for diffusion models and prompted a more in-depth technical study of their properties, including accelerated sampling schemes and improved score-matching objectives (Song et al. 2021a, 2020). Their flexibility and strong empirical performance have led to a growing literature on time series diffusion models, with applications including probabilistic forecasting (Rasul et al. 2021; Yan et al. 2021), financial time series generation (Acciaio et al. 2024), limit order book simulation (Coletta et al. 2023; Nagy et al. 2023), and time series imputation (Lin et al. 2024). Relative to classical forecasting approaches, diffusion models provide a fully nonparametric framework for learning conditional distributions and generating realistic trajectories. Autoregressive time series diffusion models such as TimeGrad (Rasul et al. 2021) and ScoreGrad (Yan et al. 2021) are highly flexible and can be applied to a variety of tasks, including probabilistic forecasting and the generation of synthetic data for scenario analysis.

Despite their strong generative capabilities, many autoregressive time series diffusion models remain computationally demanding and difficult to train. This happens because to model the conditional distribution, TimeGrad employs recurrent neural networks such as LSTMs that are trained by backpropagation through time. This can lead to noisy gradients and poor model performance even if the training loss converged (Orvieto et al. 2023). Moreover, while a large portion of the recent literature has focused on accelerating the sampling stage of diffusion models (Song et al. 2021a; Lu et al. 2022), comparatively less attention has been given to improving training efficiency. This thesis investigates whether Reservoir Computing (RC) can provide an efficient alternative framework for time series diffusion models (Grigoryeva and Ortega 2018; Gonon and Ortega 2020). Reservoir models use random sampling to fix most of the model parameters while only a handful are allowed to be trained. We suggest that replacing the fully trainable components of time series diffusion models, in particular their recurrent architectures, with reservoir-based alternatives can substantially reduce computational cost while preserving high-quality generation and forecasting performance.

We aim to answer two main research questions. First, *can reservoir-based architectures achieve competitive probabilistic forecasting and generative performance compared to fully trainable diffusion models?* Second, *does this performance come at significantly lower computational cost and faster training times?* To this end, we introduce two novel reservoir-based time series diffusion models built upon the TimeGrad framework: *ESNGrad* and *RWDN*. *ESNGrad* replaces the recurrent temporal encoder of TimeGrad with an Echo State Network (ESN), while *RWDN* further substitutes the trainable denoising network with Random Weight Neural Networks (RWNNs). In this setting, model estimation reduces to a sequence of closed-form problems solvable via least squares, eliminating the need for gradient-based optimization entirely. We evaluate the proposed models against a fully trainable TimeGrad baseline to assess their performance in both forecasting and generation. To the best of our knowledge, this is the first work to combine reservoir computing and diffusion models for probabilistic time series forecasting. Related work includes the application of reservoir computing to variational autoencoders (Wang et al. 2023), a class of generative models that is conceptually similar to diffusion models. A second line of work explores the replacement of trainable components in neural architectures, including denoising networks, with random feature-based methods (Saha and Tran 2025). However, the intersection of reservoir computing and diffusion-based generative

modeling remains largely unexplored, with only limited prior work in image generation.

This thesis contributes to the existing literature on distributional forecasting in four ways. First, it introduces a novel integration between reservoir computing and diffusion models for time series forecasting and generation. Second, it focuses on training efficiency, an aspect that has received comparatively limited attention in the diffusion literature relative to sampling acceleration. Third, it explores the application of generative diffusion models to macroeconomic distributional forecasting problems, including Inflation-at-Risk analysis. Fourth, it aims to provide a unified and accessible presentation of diffusion models in a time series setting, clarifying notation and connecting the diffusion literature with more standard econometric and time series formulations.

The remainder of this thesis is structured as follows. Section 2 introduces some concepts related to distributional forecasting and briefly discusses the pitfalls of quantile regressions. Section 3 illustrates the theoretical background on diffusion models for cross-sectional data. Section 4 examines how diffusion models can be applied in the time series setting, covering both forecasting and generation. Section 5 presents the core contribution of this thesis, detailing how reservoir computing and random weights neural networks can be integrated into time series generative models. Section 6 evaluates the proposed approach on simulated data, while Section 7 presents a real-world forecasting exercise that compares our reservoir-based diffusion models to an already tested and fully trained recurrent diffusion model. Finally, Section 8 concludes the thesis and discusses directions for future work, including possible improvements to the proposed models.

Code. The code for the models can found on [GitHub](#)

Notation We start by clarifying the notation of this thesis. We use $\mathbb{N}$ to indicate the set of natural numbers with zero included and $\mathbb{R}$ the set of all real numbers. $\mathbb{Z}$ denotes the set of all integers. We use bold lowercase letters, like $\mathbf{x}$, to indicate column vectors and $\mathbf{x}^\top$ to indicate its transpose. A vector in $\mathbb{R}^N$ has N entries, denoted by $x_i \in \mathbb{R}$ for $i \in \{1, \dots, N\}$. For any vector $\mathbf{x} \in \mathbb{R}^N$, $\|\mathbf{x}\| = \sqrt{x_1^2 + \dots + x_N^2}$ indicates the Euclidean Norm. We denote $\mathbb{M}_{N,M}$ the set of real coefficient matrices with N rows and M columns. When $N = M$, the notation is lightened to $\mathbb{M}_N$. $I_N \in \mathbb{M}_N$ indicates the identity matrix. For a matrix $A \in \mathbb{M}_{N,M}$, $\|A\|_2$ denotes the spectral norm. Moreover, $\|A\|_2 = \sigma_{\max}(A)$, where $\sigma_{\max}(A)$ is the largest singular value of A .

The notation $\mathbf{x}_t \in \mathbb{R}^N$, $\{\mathbf{x}_t\}_{t \in \mathbb{Z}}$, indicates a time series process of dimension N , where $\mathbf{x}_t = [x_{1,t}, \dots, x_{N,t}]^\top$. In this thesis, all time series are discrete-time processes. Discrete diffusion processes are indexed by an exponent $k \in \mathbb{N}$ enclosed by round brackets, like $\mathbf{x}_t^{(k)}$ for a time series process or $\mathbf{x}^{(k)}$ for cross sectional data. We write $\mathbf{x}_t^{(0)} := \mathbf{x}_t$ interchangeably to indicate the original noiseless process. To shorten the notation, we write $\mathbf{x}_t^{(k_1:k_2)} := \{\mathbf{x}_t^{(k)}\}_{k=k_1}^{k_2}$, for $k_1, k_2 \in \mathbb{N}$ and $k_1 < k_2$, to represent a finite and ordered sequence of the diffusion process. $\mathbf{x}(\kappa)$ denotes the continuous time version of $\mathbf{x}^{(k)}$, with $\kappa \in [0, 1]$. Samples generated from a diffusion process have a hat on top, for example $\hat{\mathbf{x}}$ or $\hat{\mathbf{x}}_t$, to distinguish them from the latent generative process.

2 Background: Time Series Distributional Forecasting

Consider a time series of dimension N , $\{\mathbf{x}_t \in \mathbb{R}^N\}_{t=1}^T$. Econometricians are generally interested in forecasting the outcomes of this time series at some point in the future, for example, at time $T + h$. In particular, predicting future outcomes of $\mathbf{x}_{T+h}$ given its past realizations up to time T requires modeling the conditional distribution $q(\mathbf{x}_{T+h} \mid \mathbf{x}_T, \dots, \mathbf{x}_1)$.

To illustrate this, suppose that our time series follows a VAR(p) *data-generating process* (DGP) (Hamilton 1994):

$$\mathbf{x}_t = \mathbf{c} + A_1 \mathbf{x}_{t-1} + A_2 \mathbf{x}_{t-2} + \dots + A_p \mathbf{x}_{t-p} + \boldsymbol{\varepsilon}_t, \quad (2.1)$$

where $\boldsymbol{\varepsilon}_t \stackrel{\text{iid}}{\sim} \mathcal{N}(\mathbf{0}, \Omega)$ and $\Omega \in \mathbb{M}_N$ is diagonal. Suppose that the coefficients $\{A_j\}_{j=1}^p$ are known, so that no estimation is required. Then, the *conditional distribution* of the next-period realization $\mathbf{x}_{T+1}$, given the observed sample, is (Hamilton 1994):

$$\begin{aligned} \boldsymbol{\mu}_{T+1} &= \mathbf{c} + A_1 \mathbf{x}_T + A_2 \mathbf{x}_{T-1} + \dots + A_p \mathbf{x}_{T-p+1}, \\ \mathbf{x}_{T+1} \mid \mathbf{x}_T, \dots, \mathbf{x}_{T-p+1} &\sim \mathcal{N}(\boldsymbol{\mu}_{T+1}, \Omega). \end{aligned}$$

Note that, since we are considering a VAR(p) process, the conditional distribution $q(\mathbf{x}_{T+1} \mid \mathbf{x}_T, \dots, \mathbf{x}_{T-p+1})$ is equivalent to $q(\mathbf{x}_{T+1} \mid \mathbf{x}_T, \dots, \mathbf{x}_1)$. Since the innovations $\boldsymbol{\varepsilon}_t$ are Gaussian, the one-step-ahead conditional distribution is also Gaussian. Here we show the case $h = 1$, but the same reasoning applies for a general forecast horizon.

Econometricians often use the mean of the conditional distribution, i.e. the *point forecast*, as a prediction of the future realization of the process. However, the point forecast does not convey information about the dispersion of the distribution, nor about higher-order features such as skewness, which may be crucial for understanding the direction and risks of future outcomes. For instance, one may be interested in predicting a specific quantile, such as the 0.9-quantile of the conditional distribution $q(\mathbf{x}_{T+1} \mid \mathbf{x}_T, \dots, \mathbf{x}_1)$. If we assume that the data-generating process follows the VAR(p) model in (2.1), then for the i -th component of $\mathbf{x}_{T+1}$, the conditional τ -quantile is given by (Hyndman and Athanasopoulos 2021):

$$Q_{x_{i,T+1}}(\tau \mid \mathbf{x}_T, \dots, \mathbf{x}_1) = \mu_{i,T+1} + \Phi^{-1}(\tau) \sqrt{\Omega_{ii}}, \quad (2.2)$$

where $Q_{x_{i,T+1}}(\tau \mid \cdot)$ denotes the conditional τ -quantile, $\Phi^{-1}(\cdot)$ is the inverse CDF of the standard Gaussian distribution, Ω_{ii} is the i -th diagonal element of the forecast covariance matrix Ω , and $\mu_{i,T+1}$ is the i -th element of the conditional mean vector $\boldsymbol{\mu}_{T+1}$.

In general, we do not know the true DGP of our data, and assuming that the conditional distribution is symmetric and Gaussian, as many econometric models do, can be too restrictive (Hyndman and Athanasopoulos 2021). For this reason, econometricians often estimate Growth-at-Risk or Inflation-at-Risk using *quantile regression* (Adrian et al. 2019; López-Salido and Loria 2024). A quantile regression models the τ -quantile of the conditional distribution $q(\mathbf{x}_{T+h} \mid \mathbf{x}_T, \dots, \mathbf{x}_1)$ using the linear form:

$$Q_{x_{i,T+h}}(\tau \mid \mathbf{x}_T, \dots, \mathbf{x}_{p+1}) = \alpha_{i,h}(\tau) + \sum_{j=0}^{p-1} \beta_{i,j,h}(\tau)' \mathbf{x}_{T-j}.$$

However, quantile regression has several important limitations, as it may produce quantile crossing or inconsistent estimates if not properly specified, and may yield statistically insignificant coefficients when p is large. For this reason, many studies that deploy quantile regression rely on a small set of state variables rather than explicitly modeling long lag structures (Adrian et al. 2019). Another limitation of quantile regression is that it does not provide a full estimate of the conditional density.

This work aims to address the main drawbacks of the existing literature in distributional forecasting by adopting a flexible, nonparametric approach based on time series diffusion models. In contrast to quantile regressions, diffusion models are able to approximate the full conditional likelihood $q(\mathbf{x}_{T+h} \mid \mathbf{x}_T, \dots, \mathbf{x}_{p+1})$ and capture the entire predictive distribution rather than individual quantiles. Moreover, diffusion models naturally incorporate complex nonlinear and autoregressive dependencies in the data-generating process (Yan et al. 2021). Importantly, diffusion models do not require specifying a particular parametric form for the DGP, such as Gaussian innovation or linear dynamics as in a VAR(p) model, and can therefore flexibly adapt to features such as asymmetry in the conditional distribution.

3 Diffusion Models

This Section covers the theoretical background behind diffusion models, a family of generative models that exploit a forward noising process and a learned reverse denoising process to approximate the distribution of the data. We first focus on cross-sectional data to explain how diffusion models approximate likelihoods. This provides the foundation for studying the approximation of conditional densities of the form $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \dots, \mathbf{x}_1)$, which is discussed in more detail in Section 4. We begin by presenting the Denoising Diffusion Probabilistic Models (DDPMs) of Ho et al. (2020), a discrete-time formulation that serves as the foundation for many diffusion-based approaches, including time series models such as TimeGrad (see Section 4.2). We then examine the connection between DDPMs and score-based generative models. Drawing a relationship between the two is particularly useful to understand how diffusion models can learn a conditional distribution. Finally, following Song et al. (2021b) and Kingma et al. (2021), we show that DDPMs and score-based models have a unified continuous-time representation and discuss how different numerical solvers correspond to alternative sampling strategies.

3.1 Denoising Diffusion Probabilistic Models

Denoising Diffusion Probabilistic Models (DDPM) provide a flexible framework for generating high quality samples from complex data distributions. Given a dataset $\{\mathbf{x}_i \in \mathbb{R}^N\}_{i=1}^n$ of observations drawn from an unknown distribution $q(\mathbf{x})$, DDPM defines a fixed forward Markov chain that progressively corrupts the data by injecting Gaussian noise over K steps, transforming $\mathbf{x}^{(0)} \sim q(\mathbf{x})$ into approximate standard Gaussian noise $\mathbf{x}^{(K)} \approx \mathcal{N}(\mathbf{0}, I_N)$. A denoising network is then trained to learn the reverse Markov chain, recovering the data distribution by iteratively removing noise. By approximating the reverse transition kernels at each step, the model implicitly learns a sequence of variational distributions whose composition approximates $q(\mathbf{x})$, enabling the generation of new samples $\{\hat{\mathbf{x}}_j\}_{j=1}^m$ that share the statistical properties of the true data-generating process.

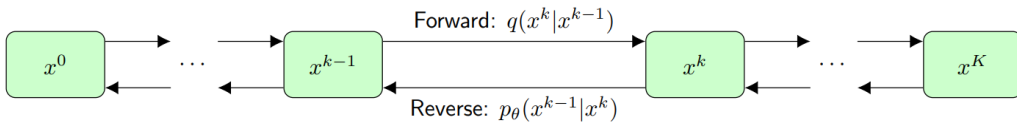


Figure 1: The forward and reverse processes in a diffusion process. Illustration from Meijer and Chen (2023) Figure 3, Page 5

Forward Process. We define a sequence of latent random variables $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}$ that perturb the original data $\mathbf{x} \sim q(\mathbf{x})$ such that:

$$\mathbf{x}^{(k)} = \sqrt{1 - \beta_k} \mathbf{x}^{(k-1)} + \sqrt{\beta_k} \mathbf{z}^{(k)}, \quad (3.1)$$

where $\{\beta_k \in [0, 1]\}_{k=1}^K$ is the *variance schedule*, $\mathbf{z}^{(k)} \sim \mathcal{N}(\mathbf{0}, I_N)$ and by definition $\mathbf{x} := \mathbf{x}^{(0)}$. Note that this process is constructed in such a way that for $K \rightarrow \infty$ the distribution $q(\mathbf{x}^{(K)})$ converges to isotropic Gaussian noise $\mathcal{N}(\mathbf{0}, I_N)$. We call the posterior $q(\mathbf{x}^{(1:K)} | \mathbf{x}^0)$ the *forward process* (or diffusion process). Since the process (3.1) is Markovian, we have

that

$$q(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)}) = \prod_{k=1}^K q(\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)}). \quad (3.2)$$

The distribution $q(\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)})$ is called the *forward transition distribution*, and it is a Gaussian distribution. From (3.1) it is straightforward that

$$\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)} \sim \mathcal{N}(\sqrt{1-\beta_k}\mathbf{x}^{(k-1)}, \beta_k I_N). \quad (3.3)$$

Using a reparameterization trick, we can directly sample an arbitrary $\mathbf{x}^{(k)}$. Let $\alpha_k = 1-\beta_k$ and $\bar{\alpha}_k = \prod_{i=1}^k \alpha_i$, then (Ho et al. 2020):

$$\begin{aligned} \mathbf{x}^{(k)} &= \sqrt{\alpha_k}\mathbf{x}^{(k-1)} + \sqrt{1-\alpha_k}\mathbf{z}^{(k)} \\ &= \sqrt{\alpha_k\alpha_{k-1}}\mathbf{x}^{(k-2)} + \sqrt{1-\alpha_{k-1}}\sqrt{\alpha_k}\mathbf{z}^{(k-1)} + \sqrt{1-\alpha_k}\mathbf{z}^{(k)} \\ &= \sqrt{\alpha_k\alpha_{k-1}}\mathbf{x}^{(k-2)} + \sqrt{1-\alpha_{k-1}\alpha_k}\bar{\mathbf{z}}^{(k-2)} \\ &= \dots \\ &= \sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)} + \sqrt{1-\bar{\alpha}_k}\bar{\mathbf{z}}^{(0)} \end{aligned} \quad (3.4)$$

where $\bar{\mathbf{z}}^{(k-2)} \sim \mathcal{N}(0, I_N)$ is obtained by the linear combination of the Gaussian noise $\mathbf{z}^{(k)}$ and $\mathbf{z}^{(k-1)}$. Indeed, in (3.4) we use the fact that the sum of two independent Gaussian random variables $\mathcal{N}(\mathbf{0}, \sigma_1^2 I_N)$ and $\mathcal{N}(\mathbf{0}, \sigma_2^2 I_N)$ is another Gaussian distribution $\mathcal{N}(\mathbf{0}, (\sigma_1^2 + \sigma_2^2)I_N)$. We call this process the *reparametrized forward process*:

$$\mathbf{x}^{(k)} = \sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)} + \sqrt{1-\bar{\alpha}_k}\boldsymbol{\epsilon}. \quad (3.5)$$

The reparameterization holds only if the variance schedule is strictly increasing, i.e. $\beta_1 < \beta_2 < \dots < \beta_K$. From (3.5) we see that we can directly sample $\mathbf{x}^{(k)}$ directly from $q(\mathbf{x}^{(k)}|\mathbf{x}^{(0)})$ without the need to run the entire forward process:

$$\mathbf{x}^{(k)}|\mathbf{x}^{(0)} \sim \mathcal{N}(\sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)}, (1-\bar{\alpha}_k)I_N). \quad (3.6)$$

The difference between $\mathbf{z}^{(k)}$ and $\boldsymbol{\epsilon}$ is that the first noise is associated with the step-wise forward process (3.1) while the second is associated with the reparametrized forward process (3.6). Since it is easier to just sample $\mathbf{x}^{(k)}$ from $\mathbf{x}^{(0)}$ rather than run the entire Markov chain, we will generally use the reparametrized forward.

Choice of β_k . The forward process (3.2) can be learned by reparameterization or kept constant as hyperparameters. One can define many fixed variance schedules $\{\beta_k\}_{k=1}^K$ for the Markovian forward process (3.2). Ho et al. (2020) use a linear schedule between $\beta_1 = 10^{-4}$ and $\beta_K = 0.02$, while Nichol and Dhariwal (2021) define a cosine schedule in terms of $\bar{\alpha}_k$:

$$\bar{\alpha}_k = \frac{f(k)}{f(0)}, \quad f(k) = \cos\left(\frac{k/K + s}{1+s} \frac{\pi}{2}\right)^2,$$

where s is a small value and $\beta_k = 1 - \frac{\bar{\alpha}_k}{\bar{\alpha}_{k-1}}$. The authors constrain β_k to be in the interval $[0, 1)$ by limiting larger values to 0.999.

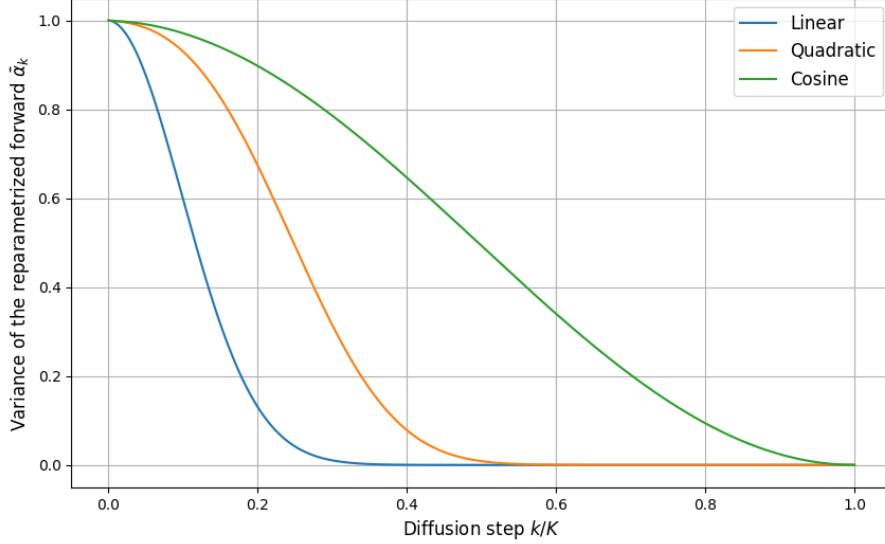


Figure 2: Different variance schedules $\bar{\alpha}_t$ as in Nichol and Dhariwal (2021) when $K = 1000$. Own illustration.

Reverse Process. If one could know the exact *reverse process* $p(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$ it would be possible to sample from $q(\mathbf{x})$ starting from the pure Gaussian noise $\mathcal{N}(\mathbf{0}, I_N)$.¹ However, the posterior $p(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$ is not directly tractable (Weng 2021). Nevertheless, we can learn it using a transition kernel $p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$ that starts at $p(\mathbf{x}^{(K)})$:

$$p_\theta(\mathbf{x}^{(0:K)}) = p(\mathbf{x}^{(K)}) \prod_{k=1}^K p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}). \quad (3.7)$$

The key assumption in DDPM is that the reverse transition kernel is approximately Gaussian when the variance schedule $\{\beta_k\}_{k=1}^K$ is sufficiently small (Nakkiran et al. 2024). For this reason, DDPM models typically require a large number of diffusion steps to ensure that, first, at the end of the forward process $\mathbf{x}^{(K)}$ is approximately isotropic Gaussian noise $\mathcal{N}(\mathbf{0}, I_N)$, and, second, the reverse process can be approximated by Gaussian kernels. If both assumptions hold, we can define the parameterized reverse transition kernel as:

$$p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}) \sim \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{x}^{(k)}, k), \boldsymbol{\Sigma}_\theta(\mathbf{x}^{(k)}, k)), \quad (3.8)$$

where the mean and covariance are parameterized functions of the noisy input $\mathbf{x}^{(k)}$ and the diffusion step k . In practice, the diffusion models literature typically parameterizes these functions using deep neural networks, i.e. $\boldsymbol{\mu}_\theta : \mathbb{R}^N \times \mathbb{R} \rightarrow \mathbb{R}^N$ and $\boldsymbol{\Sigma}_\theta : \mathbb{R}^N \times \mathbb{R} \rightarrow \mathbb{M}_N$. An important result is that the reverse conditional distribution is tractable when we condition on both $\mathbf{x}^{(k)}$ and $\mathbf{x}^{(0)}$:

$$\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}, \mathbf{x}^{(0)} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}_k(\mathbf{x}^{(k)}, \mathbf{x}^{(0)}), \tilde{\beta}_k I_N), \quad (3.9)$$

¹We keep the same notation as in standard DDPM papers (Ho et al. 2020; Song et al. 2021a) and we use $p(\cdot|\mathbf{x}^{(k)})$ for the reverse process and $q(\cdot|\mathbf{x}^{(k-1)})$ for the forward process

where

$$\tilde{\boldsymbol{\mu}}_k(\mathbf{x}^{(k)}, \mathbf{x}^{(0)}) = \frac{\sqrt{\bar{\alpha}_{k-1}}\beta_k}{1 - \bar{\alpha}_k}\mathbf{x}^{(0)} + \frac{\sqrt{\bar{\alpha}_k}(1 - \bar{\alpha}_{k-1})}{1 - \bar{\alpha}_k}\mathbf{x}^{(k)}, \quad \tilde{\beta}_k = \frac{1 - \bar{\alpha}_{k-1}}{1 - \bar{\alpha}_k}\beta_k. \quad (3.10)$$

More detailed derivations of this result can be found in Appendix A.

Training. The problem of sampling from a density function $q(\mathbf{x})$ is equivalent to the problem of learning the reverse process. In fact, if one can know the exact reverse $p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$, it would be possible to invert the forward process and obtain samples from $q(\mathbf{x})$. This can be done, as in Variational Autoencoder, by using the loss from the *variational lower bound* (VLB) derived from the negative log-likelihood of the target distribution (Weng 2021):

$$-\log(p(\mathbf{x}^{(0)})) \leq \mathbb{E}_{q(\mathbf{x}^{(1:K)})} \left[\log \frac{q(\mathbf{x}^{(1:K)} | \mathbf{x}^{(0)})}{p_\theta(\mathbf{x}^{(0:K)})} \right] = L_{\text{VLB}}(\theta). \quad (3.11)$$

For the reader familiar with standard econometrics models this can be interpreted through likelihood-based methods. DDPM tries to learn the reverse process in a way to optimize the log-likelihood of the observed data as in standard maximum likelihood. However, since we do not have a model for the DGP and we use some latent variables $\{\mathbf{x}^{(k)}\}_{k=1}^K$, a direct evaluation of the likelihood $p_\theta(\mathbf{x}^{(0)})$ is not possible. For this reason, DDPMs objective is the maximization of the variational lower bound on the log-likelihood, also known as the evidence lower bound (ELBO) (Luo 2022).² Sohl-Dickstein et al. (2015) shows that from the lower bound we can get a tractable loss:

$$\begin{aligned} L_{\text{VLB}}(\theta) = \mathbb{E}_q \left[\underbrace{\mathcal{D}_{\text{KL}}(q(\mathbf{x}^{(K)}|\mathbf{x}^{(0)})||p_\theta(\mathbf{x}^{(K)}))}_{=:L_K} \right. \\ \left. + \sum_{k=2}^K \underbrace{\mathcal{D}_{\text{KL}}(q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}, \mathbf{x}^{(0)})||p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}))}_{=:L_k} - \underbrace{\log p_\theta(\mathbf{x}^{(0)}|\mathbf{x}^{(1)})}_{=:L_0} \right], \end{aligned} \quad (3.12)$$

where the $\mathcal{D}_{\text{KL}}$ operators indicate the *Kullback-Leibler* (KL) divergence. The KL divergence of a distribution $p(x)$ relative to $q(x)$ is defined as:

$$\mathcal{D}_{\text{KL}}(p||q) = \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{x \sim p(x)} \left[\log \frac{p(x)}{q(x)} \right] = \mathbb{E}_{x \sim p(x)} [\log p(x) - \log q(x)],$$

Note that $\mathcal{D}_{\text{KL}}$ is a non-symmetric operator and in general $\mathcal{D}_{\text{KL}}(q||p) \neq \mathcal{D}_{\text{KL}}(p||q)$. We can rewrite the variational lower bound which can therefore be written as a sum of terms:

$$L_{\text{VLB}}(\theta) = \sum_{k=0}^K L_k(\theta), \quad (3.13)$$

where all KL terms, except the last one $L_0(\theta)$, compare two Gaussian distributions, and as a result they can be computed in closed form. The *prior matching* term $L_K(\theta)$ is fixed since both $q(\mathbf{x}^{(K)}|\mathbf{x}^{(0)})$ and $p_\theta(\mathbf{x}^{(K)})$ are defined from our fixed forward process

²The ELBO is just the negative version of the VLB and it is used in Variational Autoencoders (Kingma and Welling 2019)

and $(p_\theta(\mathbf{x}^{(K)}))$ converges to a $\mathcal{N}(\mathbf{0}, I_N)$ under our constraints on the variance schedule (Luo 2022). The denoising matching terms $L_k(\theta)$, for $2 < k \leq K$, compares two Gaussian distributions $q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}, \mathbf{x}^{(0)})$ and $p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$. The *reconstruction loss* $L_0(\theta)$ is often small and ignored or modeled separately as in Ho et al. (2020).

For two Gaussian random variables $d_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1)$ and $d_2 \sim \mathcal{N}(\boldsymbol{\mu}_2, \Sigma_2)$, the KL divergence admits the closed-form expression

$$\mathcal{D}_{\text{KL}}(d_1 \| d_2) = \frac{1}{2} \left[\log \frac{|\Sigma_2|}{|\Sigma_1|} - N + \text{tr}(\Sigma_2^{-1}\Sigma_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \Sigma_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \right]. \quad (3.14)$$

Under the common assumption that the covariance is fixed, $\Sigma_\theta(\mathbf{x}^{(k)}, k) = \tilde{\beta}_k I_N$, the KL divergence reduces (up to constants) to a squared error between means. As a result, the optimization problem can be written in the compact form (Luo 2022):

$$\min_{\theta} \mathbb{E}_{q(\mathbf{x}^{(0:K)}), k} \left[\frac{1}{2\tilde{\beta}_k} \left\| \tilde{\boldsymbol{\mu}}_k(\mathbf{x}^{(k)}, \mathbf{x}^{(0)}) - \boldsymbol{\mu}_\theta(\mathbf{x}^{(k)}, k) \right\|^2 \right], \quad (3.15)$$

where $\tilde{\boldsymbol{\mu}}_k(\mathbf{x}^{(k)}, \mathbf{x}^{(0)})$ denotes the mean of the true posterior $q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}, \mathbf{x}^{(0)})$. See the appendix for more details.

Loss as noise prediction. When we fix the variance, we have three equivalent ways to parametrize the variational distribution $p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$:

1. In terms of the mean predictor $\mu_\theta(\mathbf{x}^{(k)}, k)$ as we have done until now we have:

$$\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)} \stackrel{\text{approx}}{\sim} \mathcal{N}(\mu_\theta(\mathbf{x}^{(k)}, k), \tilde{\beta}_k I_N). \quad (3.16)$$

2. Another way would be to predict $\mathbf{x}^{(0)}$ from $\mathbf{x}^{(k)}$ and k . We refer to this as the “clean-sample” predictor $\hat{\mathbf{x}}_\theta(\mathbf{x}^{(k)}, k)$ and we can rewrite the variational distribution as:

$$\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)} \stackrel{\text{approx}}{\sim} \mathcal{N} \left(\frac{\sqrt{\bar{\alpha}_{k-1}}\beta_k}{1 - \bar{\alpha}_k} \hat{\mathbf{x}}_\theta(\mathbf{x}^{(k)}, k) + \frac{\sqrt{\alpha_k}(1 - \bar{\alpha}_{k-1})}{1 - \bar{\alpha}_k} \mathbf{x}^{(k)}, \tilde{\beta}_k I_N \right).$$

3. A third option is to predict the noise ϵ that we add to the clean sample $\mathbf{x}^{(0)}$. To do this we can train a *noise predictor* $\hat{\epsilon}_\theta(\mathbf{x}^{(k)}, k) : \mathbb{R}^N \times \mathbb{R} \rightarrow \mathbb{R}^N$ that for each diffusion step k estimates the total noise added to the original data. The variational distribution in term of the noise predictor can be written as:³

$$\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)} \stackrel{\text{approx}}{\sim} \mathcal{N} \left(\frac{1}{\sqrt{\alpha_k}} \left(\mathbf{x}^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \hat{\epsilon}_\theta(\mathbf{x}^{(k)}, k) \right), \tilde{\beta}_k I_N \right). \quad (3.17)$$

Since the input $\mathbf{x}^{(k)}$ is typically high-dimensional and the output depends on the diffusion step k , a common choice is to parameterize the predictor using a deep neural network. Moreover, predicting the noise is often preferred in practice, as it is normalized and has the same scale across diffusion steps, which helps stabilize training (Ho et al. 2020). For this reason, the third representation of the variational distribution $p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$ is

³The derivation on how to get expression (3.17) can be found in the appendix A

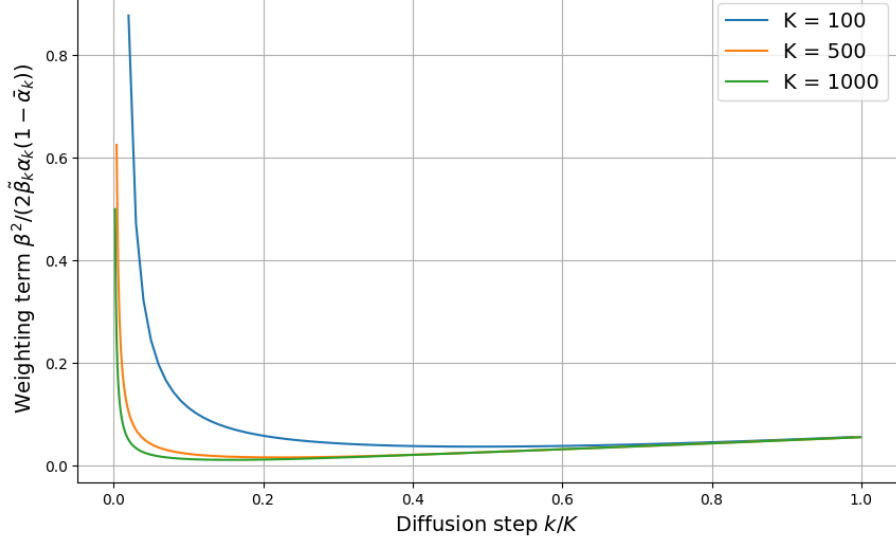


Figure 3: Weighting term of the loss (3.18) across different values of k and K . Own illustration

most commonly used. When predicting the noise, the loss becomes the following:

$$L_{\text{VLB}}(\theta) = \mathbb{E}_{\mathbf{x}^{(0)}, \epsilon, k} \left[\frac{\beta_k^2}{2\tilde{\beta}_k \alpha_k (1 - \bar{\alpha}_k)} \|\epsilon - \hat{\epsilon}_\theta(\mathbf{x}^{(k)}, k)\|^2 \right]. \quad (3.18)$$

The loss is a weighted mean squared error between the true noise added and the noise estimated by the neural network. An important consideration to keep in mind is that the noise predicting network takes both the noisy input $\mathbf{x}^{(k)}$ and the diffusion step k as inputs. If it is straightforward to see why the first is important, it is not as trivial for the latter. The role of the diffusion step k can be understood intuitively in the case of images: starting from Gaussian noise, the model must initially remove large amounts of noise, focusing on recovering the main structure of the image.⁴ As the process progresses and the noise level decreases, the denoising network shifts its focus toward refining finer details. Conditioning the network on k therefore allows it to adapt its behavior across different noise regimes throughout the reverse diffusion process. This same intuition can be applied to all other types of data, explaining the importance of correctly feeding k into the neural network.

In practice, [Ho et al. \(2020\)](#) train the network using a simplified version of the loss that omits the variance weighting term. As shown in Figure 3, this weighting term assigns relatively larger weights to small diffusion steps and smaller weights to large diffusion steps. By removing this weighting, all diffusion steps are treated uniformly, allowing the neural network to focus more effectively on the harder task of denoising highly perturbed samples corresponding to large values of k . Therefore, we use this simplified version to train our neural network:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{\mathbf{x}^{(0)}, \epsilon, k} [\|\epsilon - \hat{\epsilon}_\theta(\mathbf{x}^{(k)}, k)\|^2]. \quad (3.19)$$

⁴Generative diffusion models were originally developed for image synthesis. [Ho et al. \(2020\)](#) and [Nichol and Dhariwal \(2021\)](#) apply DDPM for image generation, by expanding the general framework of [Sohl-Dickstein et al. \(2015\)](#)

Algorithm 1 : Training of DDPM (Ho et al. 2020)

Require: sample $\{\mathbf{x}_i\}_{i=1}^n \sim p(\mathbf{x})$, variance schedule $\{\bar{\alpha}_k\}_{k=1}^K$, batch size B , learning rate η , convergence criteria $\text{crit}(\cdot)$ and c_{crit} .

- 1: Initialize parameters θ
- 2: **while** $\text{crit}(\mathcal{L}) > c_{\text{crit}}$ **do**
- 3: Draw a mini-batch of indices $\mathcal{B} \subset \{1, \dots, N\}$ of size B
- 4: Draw $\{k_i\}_{i \in \mathcal{B}} \sim \mathcal{U}(\{1, \dots, K\})$
- 5: Draw $\{\epsilon_i\}_{i \in \mathcal{B}}$, with $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I_N)$
- 6: Construct perturbed observations:

$$\mathbf{x}_i^{(k_i)} = \sqrt{\bar{\alpha}_{k_i}} \mathbf{x}_i + \sqrt{1 - \bar{\alpha}_{k_i}} \epsilon_i \quad \forall i \in \mathcal{B}$$

- 7: Compute the mini-batch loss:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \left\| \epsilon_i - \hat{\epsilon}_{\theta}(\mathbf{x}_i^{(k_i)}, k_i) \right\|^2$$

- 8: Update parameters:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

- 9: **end while**
-

Algorithm 1 describes the training procedure for the denoising network based on (3.19). Training is performed in mini-batches by sampling B observations from the dataset $\{\mathbf{x}_i\}_{i=1}^n$. For each data point $\mathbf{x}_i$, a diffusion timestep k_i is drawn from a discrete uniform distribution, and a corresponding noise vector $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I_N)$ is sampled from a standard Gaussian distribution. The loss in Algorithm 1 is an empirical approximation of (3.19). The algorithm terminates according to the convergence criteria described in Appendix A.

Learning the variance. In Ho et al. (2020) the variance $\Sigma_{\theta}(\mathbf{x}^{(k)}, k) = \sigma_k^2 I_N$ is fixed to $\sigma_k^2 = \tilde{\beta}_k$. However, by running some experiments, the authors found that fixing $\sigma_k^2 = \beta_k$ had similar results to using $\sigma_k^2 = \tilde{\beta}_k$. This happens because β_k and $\tilde{\beta}_k$ are very similar for all diffusion steps k except around $k = 0$. Increasing the number of steps K decreases the difference between the ratio between the two, as shown in Figure 4. Nichol and Dhariwal (2021) suggest that it is possible to achieve a lower training loss by learning the variance $\Sigma_{\theta}(\mathbf{x}^{(k)}, k)$ of (3.8). This can decrease the loss around the diffusion step $k = 0$, but can also lead to unstable training since it is generally not easy to learn a diagonal matrix $\Sigma_{\theta}(\mathbf{x}^{(k)}, k)$ (Ho et al. 2020). To solve the issue, they use an interpolation between β_k and $\tilde{\beta}_k$:

$$\Sigma_{\theta}(\mathbf{x}^{(k)}, k) = \text{diag}(\exp(\mathbf{v} \log \beta_k + (1 - \mathbf{v}) \log \tilde{\beta}_k)),$$

where $\mathbf{v}$ is a learned vector. Since L_{simple} does not depend on Σ_{θ} Nichol and Dhariwal (2021) uses a modified loss:

$$L_{\text{hybrid}}(\theta) = L_{\text{simple}}(\theta) + \lambda L_{\text{VLB}}(\theta),$$

where $\lambda = 0.001$. There are two main drawbacks to this approach: first, optimizing L_{VLB} is computationally expensive, and second, it leads to noisy gradients and a larger number

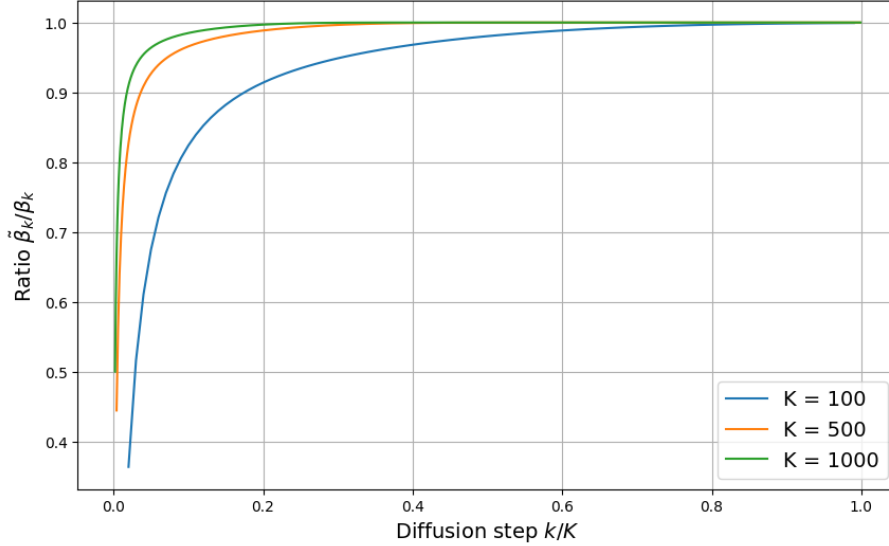


Figure 4: Ratio $\tilde{\beta}_k/\beta_k$ for the linear schedule with different diffusion steps. Own Illustration

of hyperparameters to tune (Nichol and Dhariwal 2021). For these reasons, it is often better and more practical to adopt the simplified loss (3.19), as we do in this work.

Inference. The goal of a DDPM model is to sample $\{\hat{\mathbf{x}}_i\}_{i=1}^m$ from the distribution $q(\mathbf{x})$ using the learned reverse process. This process of generating new samples is referred to as *inference* and is done according to Algorithm 2. We start by drawing $\hat{\mathbf{x}}^{(K)}$ from pure Gaussian noise $\mathcal{N}(\mathbf{0}, I_N)$, and then we sample the learned variational distribution $p_\theta(\mathbf{x}^{(K-1)}|\mathbf{x}^{(K)})$. Since the reverse process is Gaussian and takes the form of expression (3.17), we need to run the forward pass of our noise predicting network $\hat{\epsilon}_\theta(\cdot)$ evaluated in $(\hat{\mathbf{x}}^{(K)}, K)$ to invert the noise added between the diffusion step $K-1$ and K . This process is done iteratively until we get the clean sample $\hat{\mathbf{x}}^{(0)}$. Note that Ho et al. (2020) adjust the sampling procedure in step $k=1$ to account for the loss $L_0(\theta)$ in equation (3.12).

Inference. The goal of a DDPM is to generate samples $\{\hat{\mathbf{x}}_i\}_{i=1}^m$ from the learned data distribution $q(\mathbf{x})$ using the reverse diffusion process. This sampling procedure, referred to as *inference*, is described in Algorithm 2. We begin by drawing $\hat{\mathbf{x}}^{(K)} \sim \mathcal{N}(\mathbf{0}, I_N)$ and iteratively sample from the learned reverse transition distribution $p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)})$. Since this distribution is Gaussian and given by (3.17), each step requires a forward pass of the noise-predicting network $\hat{\epsilon}_\theta(\hat{\mathbf{x}}^{(k)}, k)$ to estimate the noise added at diffusion step k . This procedure is repeated for $k = K, \dots, 1$ until the final clean sample $\hat{\mathbf{x}}^{(0)}$ is obtained. Note that Ho et al. (2020) modifies the sampling procedure at $k=1$ to account for the $L_0(\theta)$ term in the decomposition of the variational bound in Eq. (3.12).

3.2 Denoising Networks

The literature on diffusion models has explored a wide range of architectures for the denoising network $\hat{\epsilon}_\theta(\cdot)$, including convolutional neural networks, U-Nets, and transformer-based models (Weng 2021). Each of these architectures excels in specific applications, and the choice of model typically depends on the structure of the learning problem. For the

Algorithm 2 : Sampling from DDPM (Ho et al. 2020)

Require: Schedules $\{\alpha_k\}_{k=1}^K$ and $\{\bar{\alpha}_k\}_{k=1}^K$, variances $\{\sigma_k\}_{k=1}^K$, number of diffusion steps K , trained noise predicting network $\hat{\epsilon}_\theta(\cdot)$ according the Algorithm 1, and number of sample to draw m

Ensure: Generate samples $\{\hat{\mathbf{x}} \in \mathbb{R}^N\}_{j=1}^m$ from $q(\mathbf{x})$

```
1: for  $j = 1, \dots, m$  do
2:   Draw  $\hat{\mathbf{x}}_j^{(K)} \sim \mathcal{N}(\mathbf{0}, I_N)$ 
3:   for  $k = K, \dots, 1$  do
4:     if  $k > 1$  then:
5:       Draw  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_N)$ 
6:     else
7:       Set  $\mathbf{z} = \mathbf{0}$ 
8:     end if
9:     Sample from  $p_\theta(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)})$ :
```

$$\hat{\mathbf{x}}_j^{(k-1)} = \frac{1}{\sqrt{\alpha_k}} \left(\hat{\mathbf{x}}_j^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \hat{\epsilon}_\theta(\hat{\mathbf{x}}_j^{(k)}, k) \right) + \sigma_k \mathbf{z}$$

```
10:   end for
11: end for
12: return  $\{\hat{\mathbf{x}}_j^{(0)}\}_{j=1}^m$ 
```

purposes of this thesis, we adopt a simple fully-connected architecture. In this subsection, we illustrate two main features of denoising network, while leaving the detailed discussion of our specific architectures for time series models to later chapters.

Positional Embedding. As already mentioned, a key challenge in diffusion models is the incorporation of the diffusion step k . Since k is a discrete index, it must be mapped into a continuous representation that can be processed by a neural network. To do this, we use a sinusoidal time embedding inspired by the Fourier positional encoding introduced in Vaswani et al. (2017). In our case, positional embedding refers to a function $e(k) : \mathbb{R} \rightarrow \mathbb{R}^E$ that maps each diffusion step k to a vector representation of the selected dimension E .

$$e(k) = [\cos(\omega_1 k), \dots, \cos(\omega_{K/2} k), \sin(\omega_1 k), \dots, \sin(\omega_{K/2} k)]^\top, \quad (3.20)$$

where $\omega_j = 10000^{-\frac{2j}{K}}$ for $j = 1, \dots, K/2$ defines an equally spaced sequence. This embedding is then passed through a small multilayer perceptron to produce a learned representation of the diffusion step.

FiLM Layers. Each layer is conditioned on the diffusion step through a *feature-wise affine transformation* (FiLM). Specifically, let $\mathbf{z}$ denote the intermediate representation at a given layer. Then, the transformation takes the form (Perez et al. 2018):

$$\mathbf{z} \mapsto (1 + \boldsymbol{\alpha}_k) \odot \mathbf{z} + \boldsymbol{\beta}_k, \quad (3.21)$$

where $(\boldsymbol{\alpha}_k, \boldsymbol{\beta}_k)$ are functions of the time embedding and $\odot$ denotes element-wise multiplication. This mechanism allows the network to adapt its behavior across different noise levels.

3.3 Score-Based Generative Models

Before diffusion models, a first class of generative models aimed at learning the data distribution $q(\mathbf{x})$ by estimating its *score function* (Hyvärinen 2005). The score is defined as the gradient of the log-density:

$$s(\mathbf{x}) = \nabla_{\mathbf{x}} \log q(\mathbf{x}), \quad (3.22)$$

and can be interpreted as a vector field that points in the direction of increasing probability mass. Intuitively, the score provides local information on how to move a point $\mathbf{x}$ toward regions of the domain of higher density.

Sampling using the score. The importance of the score function lies in the fact that it allows one to generate samples from $q(\mathbf{x})$ without explicitly knowing the density itself. Starting from a prior distribution, one can construct a Markov chain using *Langevin Dynamics* (Welling and Teh 2011):

$$\hat{\mathbf{x}}^{(k-1)} = \hat{\mathbf{x}}^{(k)} + \frac{\eta}{2} \nabla_{\mathbf{x}} \log q(\hat{\mathbf{x}}^{(k)}) + \sqrt{\eta} \mathbf{z}^{(k-1)}, \quad (3.23)$$

where $\mathbf{z}^{(k)} \sim \mathcal{N}(\mathbf{0}, I_N)$ and $\hat{\mathbf{x}}^{(K)}$ is sampled from a prior distribution (usually Gaussian or uniform). The idea of equation (3.23) is to guide the sampled points $\hat{\mathbf{x}}^{(K)}$ in a way that at step 0, $\hat{\mathbf{x}}^{(0)}$ is distributed according to the law $q(\mathbf{x})$. In Langevin Dynamics, this “transportation” is guided by the vector field defined by the score function while the $\sqrt{\eta} \mathbf{z}^{(k)}$ ensures that the process do not collapse to any local mode of the distribution. When η is small enough and K is large enough, Welling and Teh (2011) shows that $\hat{\mathbf{x}}^{(0)} \sim q(\mathbf{x})$.

Score Matching Langevin Dynamics. *Score Matching Langevin Dynamics* (SMLD) models aim to estimate the score of the data distribution $q(\mathbf{x})$ in order to construct sampling procedures based on Langevin Dynamics, as in (3.23). Since the true density $q(\mathbf{x})$ is unknown, these approaches focus on directly learning its gradient, $s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log q(\mathbf{x})$, without requiring explicit likelihood evaluation (Vincent et al. 2011). Once the score function is estimated, it can be plugged into the Langevin dynamics recursion to generate samples from the target distribution. In this sense, the quality of the generated samples depends on the accuracy of the estimated score, especially in regions of low probability where data are scarce (Song and Ermon 2019).

Connection with DDPM. Langevin Dynamics resembles the reverse process of DDPMs, as both methods rely on a recursion that starts from Gaussian noise. Indeed, it is possible to show that the objective used in score-based generative modeling is closely related to the DDPM training loss in Eq. (3.19). A key link between these frameworks is given by Tweedie’s formula (Efron 2011), which relates posterior means to score functions. For a random variable $\boldsymbol{\mu}$ and observation $\mathbf{y}$, with prior $p(\boldsymbol{\mu})$ and likelihood $\mathbf{y}|\boldsymbol{\mu} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$, the posterior mean satisfies

$$\mathbb{E}[\boldsymbol{\mu}|\mathbf{y}] = \mathbf{y} + \Sigma \nabla_{\mathbf{y}} \log p(\mathbf{y}), \quad (3.24)$$

where $p(\mathbf{y}) = \int p(\mathbf{y}|\boldsymbol{\mu})p(\boldsymbol{\mu})d\boldsymbol{\mu}$. In DDPMs, the reparametrized forward process takes the form

$$\mathbf{x}^{(k)} | \mathbf{x}^{(0)} \sim \mathcal{N}(\sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)}, (1 - \bar{\alpha}_k)I_N).$$

By setting $\boldsymbol{\mu} = \sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)}$, $\mathbf{y} = \mathbf{x}^{(k)}$, and $\Sigma = (1 - \bar{\alpha}_k)I_N$, Tweedie's formula yields

$$\mathbb{E}[\sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)} | \mathbf{x}^{(k)}] = \mathbf{x}^{(k)} + (1 - \bar{\alpha}_k)\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}).$$

Rearranging this expression we get that

$$\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}) = -\frac{\mathbf{x}^{(k)} - \sqrt{\bar{\alpha}_k}\mathbb{E}[\mathbf{x}^{(0)}|\mathbf{x}^{(k)}]}{1 - \bar{\alpha}_k}. \quad (3.25)$$

Using the DDPM reparameterization

$$\mathbf{x}^{(k)} = \sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)} + \sqrt{1 - \bar{\alpha}_k}\boldsymbol{\epsilon},$$

and substituting the model prediction $\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}^{(k)}, k)$, we obtain (Sung 2025)

$$\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}) = -\frac{\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}^{(k)}, k)}{\sqrt{1 - \bar{\alpha}_k}}. \quad (3.26)$$

This shows that DDPM noise prediction is equivalent, up to a scaling factor, to the score estimation in SMLD. Consequently, both approaches can be interpreted as different parameterizations of the same underlying diffusion process (see Section 3.5). This connection also allows DDPMs to be viewed as score-based models, which will be the perspective adopted in the following section.

3.4 Conditional Denoising Diffusion Probabilistic Models

Until now, we have seen how to sample from a probability distribution $q(\mathbf{x})$. However, we are actually interested in forecasting a time series process by approximating the conditional distribution $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \dots, \mathbf{x}_1)$. The literature on generative models exploits the properties of the score function to sample from a conditional distribution $q(\mathbf{x}|\mathbf{y})$ while only making small adjustments to the DDPM framework.

Guidance. In score matching, sampling from $q(\mathbf{x}|\mathbf{y})$, where $\mathbf{y} \in \mathbb{R}^M$ is closely related to sampling from the marginal distribution, as it suffices to train a network that predicts the conditional score (Song et al. 2020). However, a key limitation of this approach is that score estimation is often noisy and unreliable in certain regions of the distribution's support. Dhariwal and Nichol (2021) adapt DDPM to conditional generation using the following decomposition of the score of the conditional distribution $q(\mathbf{x}^{(k)}|\mathbf{y})$:

$$\begin{aligned} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}|\mathbf{y}) &= \nabla_{\mathbf{x}^{(k)}} \log \frac{q(\mathbf{x}|\mathbf{y}^{(k)})q(\mathbf{x}^{(k)})}{q(\mathbf{y})} \\ &= \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)}) + \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}) - \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}) \\ &= \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)}) + \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}). \end{aligned} \quad (3.27)$$

By substituting $\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)})$ with equation (3.26) we get (Dhariwal and Nichol 2021):

$$\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}|\mathbf{y}) = -\frac{(\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k) - \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)}))}{\sqrt{1 - \bar{\alpha}_t}}. \quad (3.28)$$

This result is very elegant and shares the same structure of (3.26). Therefore, we can interpret the numerator as an estimator of the *conditional noise* added to $\mathbf{x}$ given a particular conditioning $\mathbf{y}$. The *guidance term* $\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)})$ can be seen as the net noise difference between the unconditional and conditional generation. To estimate the conditional noise, Dhariwal and Nichol (2021) trains a second neural network to predict $\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)})$ and uses a parameter to control the strength of the guidance:

$$\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y}) = \hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k) - w\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)}), \quad (3.29)$$

where $\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y})$ is the conditional noise and $w \in [0, 1]$ is the guidance parameter. A simpler approach is to use a *guidance-free* estimation, as in Ho and Salimans (2021). Guidance-free generation estimates a different noise :

$$\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y}) = w\tilde{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y}) + (1 - w)\tilde{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{o}), \quad (3.30)$$

where $\tilde{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{c})$ is a network that predicts noise based on a conditioning $\mathbf{c}$ and $w \in [0, 1]$. The conditioning $\mathbf{c}$ can be either $\mathbf{y}$ or a learned vector $\mathbf{o} \in \mathbb{R}^M$ that represents the unconditional generation. We can read the expression above as $\tilde{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{o}) = -\sqrt{1 - \bar{\alpha}_k} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)})$ being the unconditional noise predictor and $\tilde{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y}) = -\sqrt{1 - \bar{\alpha}_k} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{y}|\mathbf{x}^{(k)})$ the conditional noise predictor. The advantage of this approach is that we just need to train one denoising network, and while training we just pass the vector $\mathbf{o}$ with probability p_{uncond} . Inference is done in the same way as in the unconditional diffusion model (Algorithm 2), with the only difference that we use conditional noise $\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y})$ instead of $\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k)$.

Conditional Generation. Classifier-free guidance allows for both unconditional and conditional generation. However, when we are solely interested in conditional generation we can use a simplification that arises by the fact that the score of a conditional distribution $q(\mathbf{x}|\mathbf{y})$ is equivalent to the score of the joint $q(\mathbf{x}, \mathbf{y})$:

$$\nabla_{\mathbf{x}} \log q(\mathbf{x}|\mathbf{y}) = \nabla_{\mathbf{x}} \log \frac{q(\mathbf{x}, \mathbf{y})}{q(\mathbf{y})} = \nabla_{\mathbf{x}} \log q(\mathbf{x}, \mathbf{y}) - \nabla_{\mathbf{x}} \log q(\mathbf{y}) = \nabla_{\mathbf{x}} \log q(\mathbf{x}, \mathbf{y}).$$

This implies that:

$$\nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}|\mathbf{y}) = \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}, \mathbf{y}).$$

Therefore, one can sample from a conditional distribution using DDPM by training a noise predicting network $\hat{\epsilon}_{\theta}(\mathbf{x}^{(k)}, k, \mathbf{y})$ directly. Training a conditional DDPM framework is very similar as the unconditional generation, as shown by the training Algorithm 3. Inference is the same as in Algorithm 2, with the only change that we predict the conditional noise.

3.5 Diffusion Models in Continuous Time

SDE Formulation. The DDPM specification of the forward process represents a particular discretization within a broader class of diffusion models. Consider $\mathbf{x}(\kappa)$ to be a

Algorithm 3 : Training conditional DDPM

Require: sample $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^n \sim q(\mathbf{x}, \mathbf{y})$, variance schedule $\{\bar{\alpha}_k\}_{k=1}^K$, batch size B , learning rate η , number of diffusion steps K , convergence criteria $\text{crit}(\cdot)$ and c_{crit}

- 1: Initialize parameters θ
- 2: **while** $\text{crit}(\mathcal{L}) > c_{\text{crit}}$ **do**
- 3: Draw a mini-batch of indices $\mathcal{B} \subset \{1, \dots, N\}$ of size B
- 4: Draw $k_i \sim \mathcal{U}(\{1, \dots, K\})$, $\forall i \in \mathcal{B}$
- 5: Draw $\{\epsilon_i\}_{i \in \mathcal{B}}$, with $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I_N)$
- 6: Construct perturbed observations:

$$\mathbf{x}_i^{(k_i)} = \sqrt{\bar{\alpha}_{k_i}} \mathbf{x}_i + \sqrt{1 - \bar{\alpha}_{k_i}} \epsilon_i, \quad \forall i \in \mathcal{B}$$

- 7: Compute the mini-batch loss:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \left\| \epsilon_i - \hat{\epsilon}_{\theta}(\mathbf{x}_i^{(k_i)}, \mathbf{y}_i, k_i) \right\|^2$$

- 8: Update parameters:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

- 9: **end while**
-

continuous time diffusion process for $\kappa \in [0, 1]$. Song et al. (2021b) show that many diffusion-based generative models can be unified under a continuous-time framework. In this setting, the forward diffusion process is described by a *stochastic differential equation* (SDE) of the form (Song et al. 2021b):

$$d\mathbf{x}(\kappa) = f(\kappa)\mathbf{x} d\kappa + g(\kappa) d\mathbf{w}, \quad (3.31)$$

where $\mathbf{x}(0) \sim q(\mathbf{x})$, $f(\kappa)$ is the drift coefficient, $g(\kappa)$ is the diffusion coefficient, and $d\mathbf{w}$ is a Wiener process. The reverse process can be formulated as an SDE in the following way:

$$d\mathbf{x}(\kappa) = [f(\kappa)\mathbf{x}(\kappa) - g(\kappa)^2 \nabla_{\mathbf{x}(\kappa)} \log p_{\kappa}(\mathbf{x}(\kappa))] d\kappa + g(\kappa) d\bar{\mathbf{w}}, \quad (3.32)$$

where $\mathbf{x}(\kappa) \sim \mathcal{N}(\mathbf{0}, I_N)$ and $\nabla_{\mathbf{x}(\kappa)} \log p_{\kappa}(\mathbf{x}(\kappa))$ is the score function of the marginal distribution $p_{\kappa}(\mathbf{x}(\kappa))$. In the literature, this expression is also referred as the *reverse-time SDE*.

Equation (3.31) looks very similar to the DDPM forward process and the NCSN perturbation. Here, we focus only on DDPM, referring the reader to Song et al. (2021b) for more information on the formulation of continuous time *Score Matching Langevin Dynamic* (SMLD) diffusion models.

DDPM in continuous time. Since DDPM used the forward Markov process $\mathbf{x}^{(k)} = \sqrt{1 - \beta_k} \mathbf{x}^{(k-1)} + \sqrt{\beta_k} \mathbf{z}^{(k)}$ (see (3.1)) we can draw an analogy with equation (3.31). To match the continuous time index one can simply write $\kappa = \frac{k}{K}$ and normalize the DDPM index between 0 and 1. In the DDPM forward chain we can substitute the discrete variables with a continuous time version and instead of a one step we consider a small step $\Delta\kappa$. Using:

$$\mathbf{x}(\kappa) = \mathbf{x}^{(k)}, \quad \mathbf{z}(\kappa) = \mathbf{z}^{(k)}, \quad \beta(\kappa) = \beta_k \Delta\kappa,$$

we can write the next latent random variable in the continuous time Markov chain $\mathbf{x}(\kappa + \Delta\kappa)$ as (Song et al. 2021b):

$$\mathbf{x}(\kappa + \Delta\kappa) = \sqrt{1 - \beta(\kappa + \Delta\kappa)\Delta\kappa} \cdot \mathbf{x}(\kappa) + \sqrt{\beta(\kappa + \Delta\kappa)\Delta\kappa} \cdot \mathbf{z}(\kappa).$$

We can employ a first order Taylor approximation of the function $f(y) = \sqrt{1 - y} \approx 1 - \frac{1}{2}y$ around $y_0 = 0$ and write:

$$\mathbf{x}(\kappa + \Delta\kappa) \approx \mathbf{x}(\kappa) - \frac{1}{2}\beta(\kappa)\Delta\kappa \mathbf{x}(\kappa) + \sqrt{\beta(\kappa)\Delta\kappa} \mathbf{z}(\kappa),$$

when $\Delta\kappa \rightarrow 0$, $\mathbf{x}(\kappa + \Delta\kappa)$ converges to (Song et al. 2021b):

$$d\mathbf{x}(\kappa) = -\frac{1}{2}\beta(\kappa)\mathbf{x}(\kappa)d\kappa + \sqrt{\beta(\kappa)}d\mathbf{w}.$$

This expression mirrors (3.31) and is the *continuous time forward process for DDPM*, where $f(\kappa) = -\frac{1}{2}\beta(\kappa)$ and $g(\kappa) = \sqrt{\beta(\kappa)}$.⁵

Variational Diffusion Models. We illustrated that diffusion process are a general category of generative models, and DDPM and SMLD are just some particular cases. Kingma et al. (2021) introduces Variational Diffusion Models (VDMs), a common framework for diffusion models that includes SMLD and DDPM. VDMs are continuous time diffusion models that are described by equations (3.31)-(3.32) and have a general form for the forward diffusion kernel

$$\mathbf{x}(\kappa)|\mathbf{x}(0) \sim \mathcal{N}(\alpha(\kappa)\mathbf{x}(0), \sigma(\kappa)I_N), \quad (3.33)$$

where the quantity $\frac{\alpha(\kappa)^2}{\sigma(\kappa)^2}$ is called the Signal-to-Noise-Ratio (SNR) and it is monotonically decreasing over the diffusion process. Karras et al. (2022) shows that the conditional mean and variance are related to the parameters of the continuous time SDE

$$\alpha(\kappa) = \exp\left(\int_0^\kappa f(s)ds\right), \quad (3.34)$$

$$\sigma^2(\kappa) = \alpha(\kappa)^2 \int_0^\kappa \frac{g(s)^2}{\alpha(s)^2} ds. \quad (3.35)$$

For DDPM it can be shown that $\alpha(\kappa) = \exp\left(-\frac{1}{2}\int_0^\kappa \beta(s)ds\right) \approx \sqrt{\bar{\alpha}_\kappa}$ and $\sigma(\kappa)^2 = 1 - \alpha(\kappa)^2$ (Karras et al. 2022). Moreover, since DDPM imposes that the $\{\beta_k\}_{k=1}^K$ sequence is strictly increasing, it holds that the signal-to-noise ratio is decreasing monotonically.⁶ As for discrete time diffusion models, the forward process is predefined by the choice of the parameters. Therefore, the only unknown term in equation (3.32) is the score function. To be able to generate samples from $q(\mathbf{x}(0))$ it is necessary to train a score predicting network $s_\theta(\cdot)$ and then run the reverse time SDE. In order to do that, we can use the Euler-Maruyama discretization of the (Karras et al. 2022):

$$\mathbf{x}^{(\kappa-\Delta\kappa)} = \mathbf{x}^{(\kappa)} - [f(\kappa)\mathbf{x}^{(\kappa)} - g(\kappa)^2\nabla_{\mathbf{x}^{(\kappa)}} \log p_\kappa(\mathbf{x}^{(\kappa)})] \Delta\kappa + g(\kappa)\sqrt{\Delta\kappa}\boldsymbol{\zeta}, \quad (3.36)$$

⁵Note that this representation of DDPM forward SDE is a Variance Preserving SDE (VP-SDE) opposite to SMLD, where the SDE representation is a Variance-Exploding SDE.

⁶Note that $\bar{\alpha}_k$ is not the same as $\alpha(k)$ in the VDM formulation.

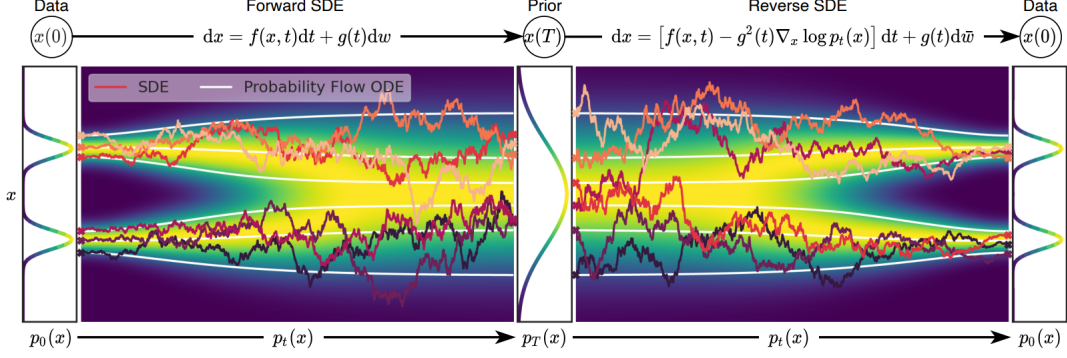


Figure 5: The reverse-time SDE has an ODE representation that deterministically maps a point in $q(\mathbf{x}^{(K)})$ to $q(\mathbf{x}^{(0)})$. Illustration from Song et al. (2021b), Figure 2, Page 4

where $\boldsymbol{\zeta} \sim \mathcal{N}(\mathbf{0}, I_N)$ and $\Delta\kappa > 0$.

Probability Flow ODE. Diffusion models have recently achieved state-of-the-art performance in image generation due to their ability to produce high-quality samples. However, a key limitation is that both training and sampling are computationally expensive. As a result, much of the literature has focused on accelerating the inference procedure by reducing the number of sampling steps and introducing deterministic or more efficient reverse processes (Song et al. 2021a; Nichol and Dhariwal 2021; Lu et al. 2022). This faster generation can be achieved by using *Ordinary Differential Equations* (ODE) representation of the reverse-time SDE (3.32) that shares the same marginal densities $\{p_\kappa(\mathbf{x})\}_{\kappa=0}^1$ (Song et al. 2021b). The *reverse-time ODE* process has the following form:

$$d\mathbf{x}(\kappa) = \left[f(\kappa)\mathbf{x}(\kappa) - \frac{1}{2}g(\kappa)^2\nabla_{\mathbf{x}(\kappa)}\log(p_\kappa(\mathbf{x})) \right] d\kappa. \quad (3.37)$$

Figure 5 shows the relationship between the SDE representation of the reverse process and the deterministic path of the ODE. ODE solvers have been studied for a long time and can be applied to solve the reverse time deterministic process (3.37) (Lu et al. 2022). For example, a first-order discretization of the reverse-time ODE (often referred to as *DPM-Solver-1* since it is the first-order approximation (Sung 2025)) applied to DDPM is (Lu et al. 2022):

$$\mathbf{x}^{(k)} = \frac{\alpha_k}{\alpha_{k+1}}\mathbf{x}^{(k+1)} - \left(\alpha_k \frac{\sigma_{k+1}}{\alpha_{k+1}} - \sigma_t \right) \hat{\epsilon}_\theta(\mathbf{x}^{(k+1)}, k+1), \quad (3.38)$$

where $\hat{\epsilon}_\theta(\mathbf{x}^{(k+1)}, k+1)$ is the estimated noise at the diffusion step $k+1$ and σ_k is the variance of the reverse process.⁷ The first-order solution of the reverse process is the same as suggested by Song et al. (2021a) in *Denoising Diffusion Implicit Models* (DDIM), a non-Markovian extension of DDPM. DDIM shares the same marginal distributions as DDPM, but uses a deterministic map between the noise and the original data (Weng 2021).

⁷It is also possible to use the DPM-Solver-1 or higher order approximation for the case of conditional generation by estimating the conditional noise instead of the unconditional one.

4 Time Series Diffusion

In this Section, we discuss how diffusion models can be applied to time series probabilistic forecasting. In Section 4.1, we formulate the problem of time series diffusion models and characterize the target distribution we aim to learn. Then, in Section 4.2, we present TimeGrad (Rasul et al. 2021), a state of the art probabilistic model based on DDPM that employs an autoregressive recurrent component to capture the temporal structure of the data. Finally, we briefly discuss our own implementation of TimeGrad.

4.1 Problem Formulation

Consider a multivariate time series $\{\mathbf{x}_t \in \mathbb{R}^N\}_{t=1}^T$. We want to learn the conditional distribution $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \dots, \mathbf{x}_1)$ to be able to perform probabilistic forecasting and generation. We start by dividing our time series into a *lookback window* for $t \in [1, t_0)$ and a *prediction window* $t \in [t_0, T]$ (Rasul et al. 2021). Time-series diffusion models are interested in modeling the distribution of the prediction window conditional on the lookback window (Lin et al. 2024)

$$q(\mathbf{x}_{t_0}, \dots, \mathbf{x}_T | \mathbf{x}_1, \dots, \mathbf{x}_{t_0-1}) = \prod_{t=t_0}^T q(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1}). \quad (4.1)$$

As in Rasul et al. (2021) we can consider the case where some relevant covariates $\{\mathbf{c}_t \in \mathbb{R}^C\}_{t=1}^T$ of dimension $C \in \mathbb{N}$ are included in the forecasting problem.⁸ In such a case, we can rewrite equation (4.1) as:

$$q(\mathbf{x}_{t_0}, \dots, \mathbf{x}_T | \mathbf{x}_1, \dots, \mathbf{x}_{t_0-1}, \mathbf{c}_1, \dots, \mathbf{c}_T) = \prod_{t=t_0}^T q(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{c}_1, \dots, \mathbf{c}_{t-1}). \quad (4.2)$$

The problem of time series generation is then to learn $q(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{c}_1, \dots, \mathbf{c}_{t-1})$. Since this is a problem of conditional generation, the results of Section 3.4 apply.

4.2 Time Series Autoregressive Denoising Diffusion Models (TimeGrad)

Rasul et al. (2021) propose *TimeGrad*, a time series diffusion model based on DDPM that exploits Recurrent Neural Networks (RNNs) to generate samples from conditional distributions. TimeGrad can be used for both time series forecasting and generation.

Recurrent Neural Networks. Recurrent Neural Networks (RNNs) are a class of models designed to process sequential data by defining a latent state that evolves over time. At each time step, the hidden state is updated as a function of the current input and the previous state, allowing the network to capture temporal dependencies. Formally, an RNN defines a recursive mapping of the form $\mathbf{s}_t = \Phi_{\theta_1}(\mathbf{x}_t, \mathbf{s}_{t-1})$, where $\mathbf{s}_t$ summarizes the information contained in past observations up to time t . For readers familiar with state-space methods, an RNN can be interpreted as a nonlinear *State Space Model* (SSM), where $\mathbf{s}_t$ plays the role of the latent state variable. Long Short-Term Memory (LSTM)

⁸Rasul et al. (2021) consider the covariates $\mathbf{c}_t$ as known at all timesteps. However this assumption is not very realistic in a forecasting setting. Therefore, in this work we restrict it to have the covariates as contemporaneously known.

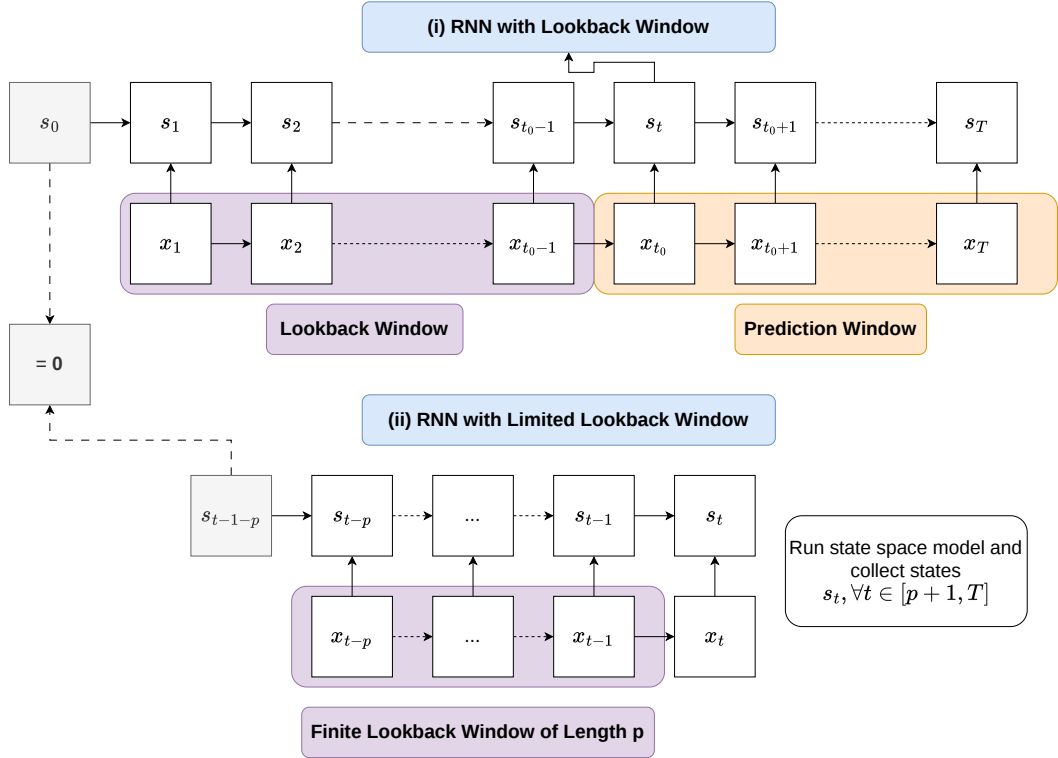


Figure 6: RNN hidden state computation with (i) full lookback window and (ii) finite lookback window of length p (proposed approach), where states $\mathbf{s}_t$ are collected for $t \in [p+1, T]$. Own illustration.

networks (Hochreiter and Schmidhuber 1997) and Gated Recurrent Units (GRUs) (Cho et al. 2014) are widely used variants of RNNs that exhibit improved learning capabilities, particularly in capturing long sequence dependencies.

Conditional modeling via hidden states. To model the conditional distribution $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \dots, \mathbf{x}_1, \mathbf{c}_{t-1}, \dots, \mathbf{c}_1)$, we require a compact and efficient representation of the conditioning information. TimeGrad addresses this by using the hidden state produced by an RNN to encode the temporal dependence of the data. The hidden state is computed recursively as:

$$\mathbf{s}_t = \Phi_{\theta_1}([\mathbf{x}_t^\top, \mathbf{c}_t^\top]^\top, \mathbf{s}_{t-1}), \quad (4.3)$$

where $\mathbf{s}_t \in \mathbb{R}^H$, θ_1 are the parameters of the RNN and the initial state is set to $\mathbf{s}_0 = \mathbf{0}$. The hidden state $\mathbf{s}_t$ can be interpreted as a summary of the past trajectory, containing all the relevant information needed to predict future observations. Using this representation, the joint conditional distribution over a prediction window can be approximated as:

$$q(\mathbf{x}_{t_0}, \dots, \mathbf{x}_T \mid \mathbf{x}_1, \dots, \mathbf{x}_{t_0-1}, \mathbf{c}_1, \dots, \mathbf{c}_T) \approx \prod_{t=t_0}^T p_{\theta}(\mathbf{x}_t \mid \mathbf{s}_{t-1}), \quad (4.4)$$

where each conditional distribution $p_\theta(\mathbf{x}_t \mid \mathbf{s}_{t-1})$ is modeled using a DDPM-based sampler. Therefore, the learned reverse process assumes a similar form as in (3.17):

$$p_\theta(\mathbf{x}_t^{(k-1)} \mid \mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}) = \mathcal{N}\left(\frac{1}{\sqrt{\alpha_k}} \left(\mathbf{x}_t^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \alpha_k}} \hat{\epsilon}_\theta(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}, k)\right), \tilde{\beta}_k I_N\right). \quad (4.5)$$

Note that $\theta = [\theta_1^\top, \theta_2^\top]^\top$, where θ_2 represents both the parameters of the noise-predicting network and θ_1 the ones of the RNN. The role of the lookback window is to ensure that the RNN has sufficient temporal context to produce a meaningful hidden state. However, it is not necessary to initialize the recursion in (4.3) from the beginning of the sample. In practice, one can define a finite sequence of length p and construct the hidden state $\mathbf{s}_t$ using only the most recent observations $\{\mathbf{x}_{t-p}, \dots, \mathbf{x}_t\}$. This second approach reduces the computational burden and can improve robustness, at the cost of requiring multiple initializations of the recurrent state $\mathbf{s}_{t-p-1}$. This additional cost can be mitigated by reusing the hidden state computed in the previous window, thereby limiting the number of required re-initializations. Figure 6 shows the two approaches.

Training. Similarly to DDPM, training is performed by minimizing the negative log-likelihood of the approximate model in (4.4). This minimization problem resembles a maximum-likelihood estimation of a time series model, with the distinction that we are not postulating a specific model for $\mathbf{x}_t$:

$$\mathcal{L}(\theta) = \sum_{t=t_0}^T -\log p_\theta(\mathbf{x}_t^{(0)} \mid \mathbf{s}_{t-1}). \quad (4.6)$$

This objective is closely related to (3.11), with the key difference that it corresponds to a sum of variational lower bounds (VLBs) for conditional distributions. Since each conditional distribution $\log p_\theta(\mathbf{x}_t^{(0)} \mid \mathbf{s}_{t-1})$ can be modeled using the conditional generation framework introduced in Section 3.4, Rasul et al. (2021) extend this conditional diffusion formulation to the TimeGrad setting. The resulting training loss corresponds to estimating the conditional noise added at each realization of the time series, as in (3.19):

$$L_{\text{simple}}(\theta) = \sum_{t=t_0}^T \mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon, k} \left[\|\epsilon_t - \hat{\epsilon}_\theta(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}, k)\|^2 \right]. \quad (4.7)$$

The training procedure for TimeGrad closely follows the conditional DDPM framework outlined in our discussion of conditional generation. However, it is worth noting that training is performed jointly for both the noise-predicting network and the RNN. In other words, the parameters of the denoising network θ_2 and the recurrent component θ_1 are updated simultaneously at each training iteration. This formulation highlights the importance of the lookback window in the standard TimeGrad setting for obtaining a balanced loss across time steps. Since the recurrent hidden state is initialized with a fixed value ($\mathbf{s}_0 = \mathbf{0}$ in our case), the hidden states $\mathbf{s}_t$ for early time steps ($t \approx 0$) are poorly representative of the conditioning history. As a result, the corresponding conditional distributions are estimated less accurately, which introduces a systematic bias in the loss contribution of early time steps and may negatively affect optimization. In the case of our finite lookback window approach, this issue is avoided by default since each state $\mathbf{s}_t$ has p timesteps to produce relevant conditioning.

Algorithm 4 : Estimation of TimeGrad (Rasul et al. 2021)

Require: time series $\{\mathbf{x}_t\}_{t=1}^T \sim q(\mathbf{x}_1, \dots, \mathbf{x}_T)$, covariates $\{\mathbf{c}_t\}_{t=1}^T$, variance schedule $\{\bar{\alpha}_k\}_{k=1}^K$, batch size B , learning rate η , initial state $\mathbf{s}_0$, length of the lookback window t_0 , number of diffusion steps K , and convergence criteria $\text{crit}(\cdot)$ and c_{crit}

1: Initialize parameters θ

2: **while** $\text{crit}(\mathcal{L}) > c_{\text{crit}}$ **do**

3: Draw a mini-batch of time indices $\mathcal{T} \subset \{t_0, \dots, T\}$ of size B

4: Compute hidden states $\{\mathbf{s}_{t-1}\}_{t \in \mathcal{T}}$ recursively:

$$\mathbf{s}_{t-1} = \Phi_{\theta}([\mathbf{x}_{t-1}^{\top}, \mathbf{c}_{t-1}^{\top}]^{\top}, \mathbf{s}_{t-2}), \quad \forall t \in \mathcal{T}$$

5: Draw $k_t \sim \mathcal{U}(\{1, \dots, K\})$, $\forall t \in \mathcal{T}$

6: Draw $\{\epsilon_t\}_{t \in \mathcal{T}}$, with $\epsilon_t \sim \mathcal{N}(\mathbf{0}, I_N)$

7: Construct perturbed observations:

$$\mathbf{x}_t^{(k_t)} = \sqrt{\bar{\alpha}_{k_t}} \mathbf{x}_t + \sqrt{1 - \bar{\alpha}_{k_t}} \epsilon_t, \quad \forall t \in \mathcal{T}$$

8: Compute the mini-batch loss:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{t \in \mathcal{T}} \left\| \epsilon_t - \hat{\epsilon}_{\theta}(\mathbf{x}_t^{(k_t)}, \mathbf{s}_{t-1}, k_t) \right\|^2$$

9: Update parameters:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

10: **end while**

Inference. Given the hidden state $\mathbf{s}_T$ produced by the RNN over the observed window, TimeGrad uses the learned reverse diffusion process to sample from the conditional distribution $p_{\theta}(\mathbf{x}_{T+1} \mid \mathbf{s}_T)$. This enables two related tasks: *probabilistic forecasting* and *time series generation*. For probabilistic forecasting, a one-step-ahead forecast is obtained by drawing m samples from $p_{\theta}(\mathbf{x}_{T+1} \mid \mathbf{s}_T)$, yielding an empirical predictive distribution. Multi-step forecasting proceeds autoregressively: each sampled value $\hat{\mathbf{x}}_{T+j}$ is fed back into the RNN to update the hidden state $\mathbf{s}_{T+j}$, from which the next predictive distribution is obtained. From the approximate conditional distribution, one can estimate the conditional mean $\mathbb{E}[\mathbf{x}_t \mid \mathbf{s}_{t-1}] \approx \mathbb{E}[\mathbf{x}_t \mid \mathbf{x}_{t-1}, \dots, \mathbf{x}_1]$ as well as higher order conditional moments. Furthermore, we can also compute the quantiles of our approximate conditional distribution directly.

Time Series Generation. Another appealing feature of time series diffusion models is their flexibility. Models such as TimeGrad are not only able to produce probabilistic forecasts, but can also generate synthetic trajectories that share the same statistical properties as the observed time series. In particular, one can generate a trajectory of arbitrary length by iteratively sampling from the estimated conditional distribution and recursively updating the hidden state using the previously generated observation. The resulting synthetic path shares key features of the data, such as its autocorrelation structure, since the hidden state encodes the underlying temporal dynamics of the process. This represents an important advantage over quantile regression approaches, which focus on estimating specific conditional quantiles but do not provide a coherent generative mechanism for

Algorithm 5 : Sampling $x_t^{(0)}$ via Langevin dynamics (Rasul et al. 2021)

Require: hidden state $\mathbf{s}_{t-1}$ from a trained RNN $\Phi_\theta(\cdot)$ according to Algorithm 4, trained denoising network $\epsilon_\theta(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}, k)$ from Algorithm 4 and schedules $\{\beta_k, \alpha_k, \bar{\alpha}_k, \tilde{\beta}_k\}_{k=1}^K$

Ensure: Sample $\mathbf{x}_t \in \mathbb{R}^N$ from $q(\mathbf{x}_t | \mathbf{s}_{t-1})$

- 1: Draw $\hat{\mathbf{x}}_t^{(K)} \sim \mathcal{N}(\mathbf{0}, I_N)$
- 2: **for** $k = K, \dots, 1$ **do**
- 3: **if** $k > 1$ **then**
- 4: Draw $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_N)$
- 5: **else**
- 6: Draw $\mathbf{z} = \mathbf{0}$
- 7: **end if**
- 8: Sample from $p_\theta(\mathbf{x}_t^{(k-1)} | \mathbf{x}_t^{(k)})$:

$$\hat{\mathbf{x}}_t^{(k-1)} = \frac{1}{\sqrt{\alpha_k}} \left(\hat{\mathbf{x}}_t^{(k)} - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta(\hat{\mathbf{x}}_t^{(k)}, \mathbf{s}_{t-1}, k) \right) + \tilde{\beta}_k \mathbf{z}$$

- 9: **end for**
 - 10: **return** $\hat{\mathbf{x}}_t^{(0)}$
-

simulating full trajectories.⁹

Model Architecture. In this paper, we consider a simplified version of the denoising network proposed in the original TimeGrad framework. Following Rasul et al. (2021), we set the number of diffusion steps to $K = 100$ and model the temporal dynamics using a two-layer LSTM. The model is trained via stochastic gradient-based optimization using the Adam optimizer with a learning rate of 10^{-3} . Our denoising network differs from the convolutional architecture proposed in Rasul et al. (2021). The primary reason is computational: we focus on relatively simple applications for which a large convolutional network would be unnecessarily costly to train.¹⁰ For this reason, we adopt a simple and flexible fully-connected architecture.¹¹ The resulting model can be interpreted as a time-conditioned multilayer perceptron (MLP), where the dependence on the diffusion step is introduced through feature-wise affine modulation at each layer (see Section 3.2 for details on FiLM and MLP layers). Nonlinear activation functions (SiLU) are applied between layers. Figure C1 in the Appendix illustrates the structure of the denoising network $\hat{\epsilon}_\theta(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}, k)$, while Figure C2 presents the overall temporal architecture of the time series diffusion model.

⁹It is worth noting, however, that TimeGrad is primarily designed as a forecasting model. Generating a full trajectory requires an initial hidden state $\mathbf{s}_T$, meaning that the model is conditioned on an observed lookback window rather than generating entirely from an unconditional distribution.

¹⁰The original TimeGrad architecture is designed for multivariate time series with "hundreds of thousands of correlated dimensions" (Rasul et al. 2021)

¹¹The idea for the network comes from Sung (2025) and has been adapted to our particular case

5 Diffusion Models and Reservoir Computing

We introduce two novel time-series diffusion models by building on the TimeGrad framework and leveraging the computational efficiency of Reservoir Computing (RC). First, in Section 5.1, we present Echo State Networks (ESNs) and show how their learning mechanism can capture the autoregressive structure of time series without relying on backpropagation. Second, in Section 5.2, we demonstrate how ESNs can be integrated into the TimeGrad framework while preserving the main structure of Denoising Diffusion Probabilistic Models (DDPMs). Finally, in Section 5.3, we propose a fully reservoir-based diffusion model in which the denoising network is replaced by a Random-Weight Neural Network (RWNN). Both models rely on a smaller number of trainable parameters than TimeGrad while exhibiting similar characteristics. Distributional forecasting and sample generation can be performed in a manner analogous to TimeGrad and other time-series diffusion models.

5.1 Echo State Networks

State Space Models. State Space Models (SSMs) are a family of time series models that can be written according to the following representation (Ballarin et al. 2024):

$$\mathbf{s}_t = \Phi_\theta(\mathbf{x}_t, \mathbf{s}_{t-1}), \quad (5.1)$$

$$\mathbf{y}_{t+1} = g_\vartheta(\mathbf{s}_t) + \boldsymbol{\varepsilon}_t, \quad (5.2)$$

where $\mathbf{s}_t \in \mathbb{R}^H$ is a latent *state vector*, $\mathbf{x}_t \in \mathbb{R}^N$ and $\mathbf{y}_t \in \mathbb{R}^M$ are the input and the output of the system, $\Phi_{\theta_1} : \mathbb{R}^H \times \mathbb{R}^N \rightarrow \mathbb{R}^H$ is the *state map*, $g_\vartheta : \mathbb{R}^H \rightarrow \mathbb{R}^M$ is the *observation map* and $\boldsymbol{\varepsilon}_t \in \mathbb{R}^M$ is an $\mathcal{N}(\mathbf{0}, \sigma_\varepsilon^2 I_M)$ i.i.d. noise process. A large range of classical time-series models, such as the Box-Jenkins models and multivariate GARCH can be casted in state space form (Durbin and Koopman 2012). Recurrent neural networks are also a class of state space models and are generally trained using backpropagation through time. Reservoir computing models are a particular class of state space models in which the parameters θ_1 of the state map are randomly initialized and kept fixed, while only the parameters of the observation map are trained. This approach leads to substantial computational gains, as it avoids backpropagation through time as in RNNs. Moreover, many RC models, and in particular, Echo State Networks, enjoy strong theoretical guarantees, including universal approximation properties and favorable learning bounds (Gonon et al. 2020; Grigoryeva and Ortega 2018).

Echo State Networks. Echo State Networks are a subclass of RC models with a particular representation of (5.1)-(5.2):

$$\mathbf{s}_t = \alpha \mathbf{s}_{t-1} + (1 - \alpha) \sigma(A \mathbf{s}_{t-1} + C \mathbf{x}_t + \boldsymbol{\zeta}), \quad (5.3)$$

$$\mathbf{y}_{t+1} = \mathbf{a} + W^\top \mathbf{s}_t + \boldsymbol{\varepsilon}_t, \quad (5.4)$$

where $A \in \mathbb{M}_H$ is the *reservoir matrix*, $C \in \mathbb{M}_{H,N}$ is the *input matrix*, $\boldsymbol{\zeta} \in \mathbb{R}^H$ is the *input shift*, $\alpha \in [0, 1)$ is the *leak rate* and $W \in \mathbb{M}_{H,M}$ is the *readout matrix*. The function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is an activation function applied element by element and is often chosen to be the hyperbolic tangent. The parameters $(A, C, \boldsymbol{\zeta})$ are called *state parameters* and are randomly drawn and never trained. Instead, the parameters $\vartheta = (\mathbf{a}, W)$ are usually

estimated by penalized least squares (Ballarin et al. 2024).

State parameters. Simply drawing the state parameters from arbitrary distributions does not necessarily yield an effective ESN model (Ballarin et al. 2025). For this to hold, the ESN must satisfy the so-called *Echo State Property* (ESP). The ESP states that, for a given input sequence $\{\mathbf{x}_t\}_{t=1}^T$, there exists a unique state sequence $\{\mathbf{s}_t\}_{t=0}^T$ such that (5.3)–(5.4) are satisfied. A sufficient condition for the ESP is $\|A\|_2 L_\sigma < 1$, where L_σ denotes the Lipschitz constant of the activation function (Grigoryeva and Ortega 2018). In particular, for a large class of activation functions, such as the sigmoid or the hyperbolic tangent, this condition is satisfied if the spectral radius of the reservoir matrix A is strictly less than one. Moreover, this condition implies that the influence of the initial state $\mathbf{s}_0$ vanishes for sufficiently long sequences, a property known as the *fading memory property* (Grigoryeva and Ortega 2018).

To appropriately select the state parameters, Ballarin et al. (2024) propose to first sample them from standard distributions (e.g., Gaussian or uniform) and then to perform a normalization. Specifically, let:

$$\bar{A} = \frac{\tilde{A}}{\rho(\tilde{A})}, \quad \bar{C} = \frac{\tilde{C}}{\|\tilde{C}\|}, \quad \bar{\zeta} = \frac{\tilde{\zeta}}{\|\tilde{\zeta}\|}, \quad (5.5)$$

where $\rho(\cdot)$ denotes the spectral radius, and $\|\cdot\|$ is a chosen norm (e.g., the Euclidean norm for ζ and ℓ_1 , ℓ_2 , or ℓ_∞ norms for C). Setting $A = \rho\bar{A}$, $C = \gamma\bar{C}$, and $\zeta = \omega\bar{\zeta}$, the state equation of the ESN becomes:

$$\mathbf{s}_t = \alpha\mathbf{s}_{t-1} + (1 - \alpha)\sigma(\rho\bar{A}\mathbf{s}_{t-1} + \gamma\bar{C}\mathbf{x}_t + \omega\bar{\zeta}), \quad (5.6)$$

where $\rho \in [0, 1)$ is the *spectral radius* of the reservoir matrix, $\gamma \in \mathbb{R}_+$ is the *input scaling*, and $\omega \in \mathbb{R}_+$ is the *shift scaling*. The tuple $\varphi = (\alpha, \rho, \gamma, \omega)$ is called the hyperparameters of the state dynamics (Ballarin et al. 2024).

In practice, each entry of the matrix $\tilde{A}$ is typically drawn from a Gaussian distribution, while $\tilde{C}$ and $\tilde{\zeta}$ are sampled element-wise from uniform distributions. To mitigate overfitting and improve computational efficiency, a predefined level of *sparsity* is imposed on the state parameters (Ballarin et al. 2025). Sparsity is introduced by randomly setting a fraction of the entries of these matrices to zero according to a specified probability. This procedure is analogous to dropout in recurrent neural networks, although it is applied directly to the fixed reservoir weights rather than during training.

5.2 Time Series Diffusion Model and ESNs (ESNGrad)

We propose to replace the RNN of TimeGrad with an Echo State Network. In this way, we avoid the issues related to RNNs and long sequences (Orvieto et al. 2023), while reducing the complexity of the model. We refer to this model as *ESNGrad* since it is the combination of the ESN for temporal modeling and a score estimating network (recall that the denoising network indirectly estimates the score of the variational distribution). Formally, for the ESNGrad diffusion model, we model the conditional distribution as

$$q(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{c}_1, \dots, \mathbf{c}_{t-1}) \approx p_\theta(\mathbf{x}_t | \mathbf{s}_{t-1}),$$

Algorithm 6 : Estimation of ESNGrad

Require: time series $\{\mathbf{x}_t\}_{t=1}^T \sim q(\mathbf{x}_1, \dots, \mathbf{x}_T)$, covariates $\{\mathbf{c}_t\}_{t=1}^T$, variance schedule $\{\bar{\alpha}_k\}_{k=1}^K$, batch size B , learning rate η , length of the lookback window t_0 , initial ESN state $\mathbf{s}_0$, hyperparameters φ , number of diffusion steps K , convergence criteria $\text{crit}(\cdot)$ and c_{crit}

- 1: Initialize parameters θ .
- 2: Draw entry-wise $\bar{A}$, $\bar{C}$ and $\bar{\zeta}$ from appropriate distributions independently from the data
- 3: Define $\mathbf{y}_t := [\mathbf{x}_t^\top, \mathbf{c}_t^\top]^\top$ and compute the ESN states:

$$\mathbf{s}_{t-1} = \alpha \mathbf{s}_{t-2} + (1 - \alpha) \sigma(\rho \bar{A} \mathbf{s}_{t-2} + \gamma \bar{C} \mathbf{y}_{t-1} + \omega \bar{\zeta}), \quad \forall t \in [1, \dots, T]$$

- 4: **while** $\text{crit}(\mathcal{L}) > c_{\text{crit}}$ **do**
- 5: Draw a batch of time indices $\mathcal{T} \subset \{t_0, \dots, T\}$ of size B
- 6: Draw $k_t \sim \mathcal{U}(\{1, \dots, K\}) \quad \forall t \in \mathcal{T}$
- 7: Draw $\{\epsilon_t\}_{t \in \mathcal{T}}$, with $\epsilon_t \sim \mathcal{N}(\mathbf{0}, I_N)$
- 8: Construct perturbed observations:

$$\mathbf{x}_t^{(k_t)} = \sqrt{\bar{\alpha}_{k_t}} \mathbf{x}_t + \sqrt{1 - \bar{\alpha}_{k_t}} \epsilon_t, \quad \forall t \in \mathcal{T}$$

- 9: Compute the batch loss:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{t \in \mathcal{T}} \left\| \epsilon_t - \hat{\epsilon}_\theta(\mathbf{x}_t^{(k_t)}, \mathbf{s}_{t-1}, k_t) \right\|^2$$

- 10: Update parameters:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$$

- 11: **end while**
-

where $\mathbf{s}_{t-1}$ comes from the recursion (5.6). Since the model shares the same setting as a conditional diffusion model and we also minimize the negative log-likelihood, we can use the same arguments of Section 4 to derive the following loss:

$$L_{\text{simple}}(\theta) = \sum_{t=t_0}^T \mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon, k} \left[\left\| \epsilon_t - \hat{\epsilon}_\theta(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}, k) \right\|^2 \right]. \quad (5.7)$$

Training is performed according to Algorithm 6. The need for a lookback window in ESNGrad is justified by the fading memory property: to have a relevant state $\mathbf{s}_t$ that summarizes the history of the time series, we need to allow the model to have enough timesteps to forget the starting condition $\mathbf{s}_0$. Moreover, we need to impose that $\rho < 1$ to ensure that the spectral radius of A lies in the unit circle.

Computational Gains. The main advantage of this approach is that unlike TimeGrad, in ESNGrad we only need to train the denoising network. In fact, given some particular state parameters and ESN hyperparameters, the states of the ESN are computed once and kept fixed during training. This brings clear computational advantages at the cost of correctly selecting appropriate law for the state parameters and choosing the hyperparameters φ . Indeed, for long sequences with a large cross section, the training of the

RNN can suffer from vanishing or exploding gradients. Moreover, since we need to train two models jointly, we also need to keep track of the gradient flow through the lookback window. Since the denoising network is often large, it amplifies the issues associated with the training of the RNN. Recall that our partial solution to this problem in TimeGrad could be to produce each $\mathbf{s}_t$ by using a sequence of limited length p , as we suggested in Section 4.

Inference. Generation and forecasting are done the same way as in TimeGrad. Starting from $\hat{\mathbf{x}}_t^{(K)} \sim \mathcal{N}(\mathbf{0}, I_N)$, we can generate one observation from $p_\theta(\mathbf{x}_t | \mathbf{s}_{t-1})$ by iteratively sampling from $p_\theta(\mathbf{x}_t^{(k-1)} | \mathbf{x}_t^{(k)}, \mathbf{s}_{t-1})$. This is done as in Algorithm 5 for TimeGrad with the only difference that we use the ESN states instead of a trained RNN state.

5.3 Full Reservoir Computing Time Series Diffusion

We introduce a second time-series diffusion model fully based on reservoir computing. While ESNGrad replaces the trainable autoregressive component with a fixed Echo State Network to model temporal dependencies, we now take a step further by replacing the denoising network $\hat{\epsilon}_\theta(\cdot)$ with a reservoir-based architecture.

Random Weight Neural Networks. *Random Weight Neural Networks* (RWNNs) can be viewed as the RC counterpart of *Artificial Neural Networks* (ANNs) and as a *static* version of random recurrent architectures such as ESNs. In particular, RWNNs are a class of feed-forward neural networks in which only a subset of the parameters is trained, while the remaining parameters are randomly initialized and kept fixed. A single-hidden-layer RWNN defines a mapping $f(\mathbf{u}) : \mathbb{R}^M \rightarrow \mathbb{R}^N$ of the form (Gonon et al. 2023):

$$f(\mathbf{u}) = W^\top \sigma(A\mathbf{u} + b), \quad (5.8)$$

where $\mathbf{u} \in \mathbb{R}^M$, $W \in \mathbb{M}_{R,N}$, $A \in \mathbb{M}_{R,M}$, and $b \in \mathbb{R}^R$ denote the parameters of the network, and $\sigma(\cdot)$ is a nonlinear activation function applied element-wise. The parameters A and b are sparse and randomly drawn from simple regular distributions (such as Gaussian or uniform) and kept fixed, while the output weight matrix W is estimated conditional on the realizations of A and b . Figure 7 offers a visual comparison of the different training procedure of RWNNs relative to ANNs. The dimension R , corresponding to the number of hidden neurons, is typically chosen to be large in order to embed the input into a high-dimensional feature space. This enables the model to approximate complex nonlinear functions through a simple linear combination of nonlinear features. Under suitable conditions, RWNNs enjoy universal approximation properties and favorable generalization bounds (Gonon et al. 2023). Moreover, random feature models with ridge regularization for W can achieve optimal generalization using a number of random features substantially smaller than the training sample size (Rudi and Rosasco 2017).

Random Weight Denoising Network. Our goal is to replace the trainable noise-predicting network $\hat{\epsilon}_\theta(\mathbf{x}^{(k)}, k, \mathbf{s}_{t-1})$ of ESNGrad with a static reservoir-based architecture. Recall that, through noise estimation, we implicitly parameterize the reverse diffusion process:

$$p_\theta(\mathbf{x}_t^{(0:K)} | \mathbf{s}_{t-1}) = p(\mathbf{x}_t^{(K)} | \mathbf{s}_{t-1}) \prod_{k=1}^K p_\theta(\mathbf{x}_t^{(k-1)} | \mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}). \quad (5.9)$$

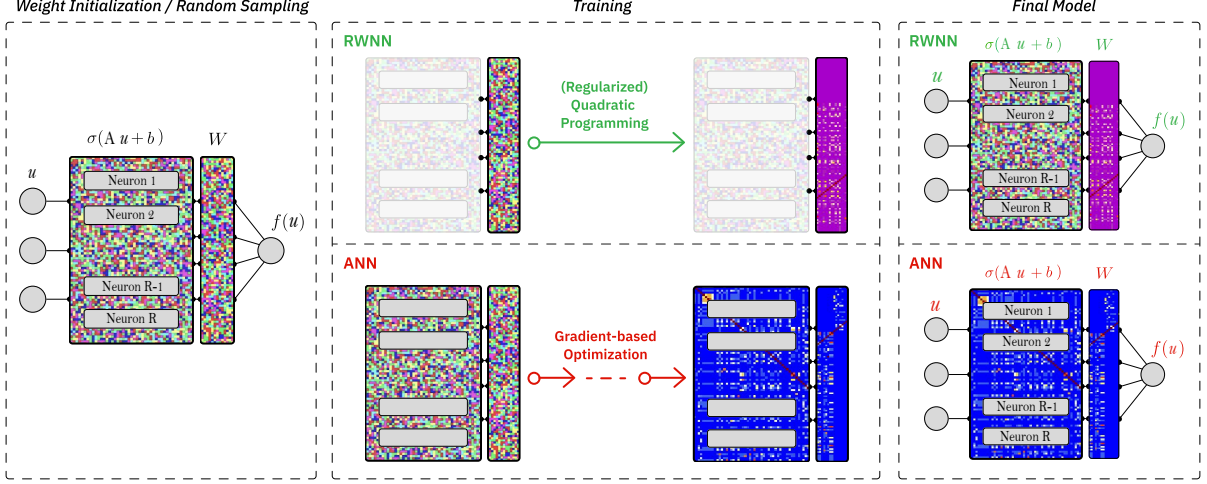


Figure 7: Comparison between Random Weight Neural Networks (RWNNs) and standard Artificial Neural Networks (ANNs)

As discussed previously, correctly accounting for the diffusion step k is crucial, since it determines the noise level in the perturbed data. To capture this dependence, we propose to assign a specific Random Weight Neural Network to each diffusion step k . We refer to the resulting chain as a *Random Weight Denoising Network* (RWDN). Formally, a RWDN consists of K RWNNs, where each network defines a map $\hat{\epsilon}_{\theta_k} : \mathbb{R}^N \times \mathbb{R}^H \rightarrow \mathbb{R}^N$ of the form:

$$\mathbf{u}_t^{(k)} = [(\mathbf{x}_t^{(k)})^\top, \mathbf{s}_{t-1}^\top]^\top,$$

$$\hat{\epsilon}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}) = W_k^\top \sigma(A_k \mathbf{u}_t^{(k)} + b_k), \quad (5.10)$$

where $A_k \in \mathbb{M}_{R, N+H}$ and $b_k \in \mathbb{R}^R$ are randomly drawn, $\mathbf{s}_{t-1}$ is the hidden state of the ESN, $\mathbf{x}_t^{(k)}$ is the noisy input of the diffusion step k , $W_k \in \mathbb{M}_{R, N}$ is the readout matrix and $\sigma(\cdot)$ is an activation function. θ_k represents the parameters of the network k , that in this case is just the readout matrix W_k , while $\theta = [\theta_1^\top, \dots, \theta_K^\top]^\top$ are the parameters of the RWDN. The reverse transition kernel follows directly from the standard DDPM formulation, with the only difference that the noise is predicted by the k -th RWNN:

$$p_\theta(\mathbf{x}_t^{(k-1)} | \mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}) = \mathcal{N}\left(\frac{1}{\sqrt{\alpha_k}} \left(\mathbf{x}_t^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \hat{\epsilon}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1})\right), \tilde{\beta}_k I_N\right). \quad (5.11)$$

We adopt the same conditional DDPM objective as in ESNGrad, which can be written as:

$$L_{\text{simple}}(\theta) = \sum_{k=1}^K \mathcal{L}_k(\theta_k), \quad (5.12)$$

where each loss $\mathcal{L}_k(\theta)$ takes the form of:

$$\mathcal{L}_k(\theta_k) = \mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon} \left[\sum_{t=t_0}^T \|\epsilon_t - \hat{\epsilon}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1})\|^2 \right]. \quad (5.13)$$

We rewrite the ESNGrad loss in a slightly different way to show that we are actually optimizing a sequence of K different neural networks. Therefore, minimizing (5.12) is the same as independently minimizing each one of the $\mathcal{L}_k(\theta_k)$ in (5.13). The only trained parameter of an RWDN is the readout matrix associated with each step k . Since for time series diffusion models the total number of diffusion step is generally around $K \approx 100$, the estimation of a RWDN is equivalent to the estimation of the parameters $\{W_k\}_{k=1}^K$ of (5.10). Although this may appear computationally demanding at first, since we need to estimate K matrices of R rows and N columns, the problem is in fact straightforward. Let us define $\mathbf{z}_t^{(k)} \in \mathbb{R}^H$ as the output of the hidden layer of a unit of the RWDN:

$$\mathbf{z}_t^{(k)} := \sigma(A_k \mathbf{u}_t^{(k)} + b_k). \quad (5.14)$$

Conditional on random features $(A_k, \mathbf{b}_k)$ of the hidden-layer of the RWNN and on the state parameters (A, C, ζ) of the ESN, the model is linear in the parameters W_k :

$$\hat{\epsilon}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}) = W_k^\top \mathbf{z}_t^{(k)}. \quad (5.15)$$

Therefore, the network can be efficiently estimated using standard least-squares or stochastic gradient methods. In fact, if we plug (5.15) in our loss (5.13), the minimization problem looks exactly like a least squares problem. This significantly reduces the computational burden compared to fully trainable deep neural networks, while retaining sufficient flexibility to approximate complex nonlinear denoising functions.

Estimation of W_k . The readout matrix W_k can be estimated either via gradient-based methods or by least squares. Here, we focus on the closed form solution for W_k , conditional on the random features $\mathbf{z}_t^{(k)}$. Since we apply a selected degree of sparsity to the parameters $(A_k, \mathbf{b}_k)$ the computation of the inverse in a standard least squares solution can be problematic. For this reason, we use as in Ballarin et al. (2024) a regularized regression by rewriting the objective (5.13) (van de Geer 2000):

$$\arg \min_{W_k} \sum_{t=t_0}^T \|\epsilon_t - W_k^\top \mathbf{z}_t^{(k)}\|^2 + \lambda p(W_k), \quad (5.16)$$

where λ is the ridge penalization coefficient and $p(W_k) = \|W_k\|^2$ is the ridge penalty. To get the closed form solution, let us define:

$$U_k = \begin{bmatrix} (\mathbf{u}_{t_0}^{(k)})^\top \\ \vdots \\ (\mathbf{u}_T^{(k)})^\top \end{bmatrix}, \quad Z_k = \begin{bmatrix} (\mathbf{z}_{t_0}^{(k)})^\top \\ \vdots \\ (\mathbf{z}_T^{(k)})^\top \end{bmatrix}, \quad E_k = \begin{bmatrix} (\epsilon_{t_0}^{(k)})^\top \\ \vdots \\ (\epsilon_T^{(k)})^\top \end{bmatrix}, \quad (5.17)$$

where $Z_k \in \mathbb{M}_{T-t_0+1, R}$ collects the hidden-layer outputs, $U_k \in \mathbb{M}_{T-t_0+1, N+H}$ contains the inputs at diffusion step k , and $E_k \in \mathbb{M}_{T-t_0+1, N}$ stacks the corresponding noise realizations $\epsilon_t^{(k)}$. Note that, in contrast to previous sections, we explicitly index the noise by the diffusion step k . This choice reflects the fact that the estimation problem decomposes across diffusion steps, and each noise predictor $\hat{\epsilon}_{\theta_k}$ is trained independently. Moreover, we still consider the division between the lookback window and the prediction window to ensure that the ESN state $\mathbf{s}_{t-1}$ provides a meaningful summary of the past according to

Algorithm 7 : Estimating a Random Weight Denoising Network

Require: time series $\{\mathbf{x}_t\}_{t=1}^T \sim q(\mathbf{x}_1, \dots, \mathbf{x}_T)$, covariates $\{\mathbf{c}_t\}_{t=1}^T$, variance schedule $\{\bar{\alpha}_k\}_{k=1}^K$, initial ESN state $\mathbf{s}_0$, length of the lookback window t_0 , ESN hyperparameters φ , RWDN hyperparameters ψ , ridge penalty λ and number of diffusion steps K

- 1: Draw ESN state parameters $\bar{A}$, $\bar{C}$ and $\bar{\zeta}$ from appropriate normalized distributions
- 2: Draw RWDN parameters $\{\bar{A}_k\}_{k=1}^K$ and $\{\bar{b}_k\}_{k=1}^K$ from appropriate normalized distributions
- 3: Define $\mathbf{y}_t := [\mathbf{x}_t^\top, \mathbf{c}_t^\top]^\top$ and compute the ESN states:

$$\mathbf{s}_{t-1} = \alpha \mathbf{s}_{t-2} + (1 - \alpha) \sigma(\rho \bar{A} \mathbf{s}_{t-2} + \gamma \bar{C} \mathbf{y}_{t-1} + \omega \bar{\zeta}), \quad \forall t \in [1, \dots, T]$$

- 4: **for** $k = 1, \dots, K$ **do**
- 5: **for** $t = t_0, \dots, T$ **do**
- 6: Draw $\{\boldsymbol{\epsilon}_t^{(k)}\}_{t=t_0}^T$ with $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, I_N)$
- 7: Construct perturbed observations:

$$\mathbf{x}_t^{(k)} = \sqrt{\bar{\alpha}_k} \mathbf{x}_t + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}_t^{(k)}$$

- 8: Define:

$$\mathbf{u}_t^{(k)} = [(\mathbf{x}_t^{(k)})^\top, \mathbf{s}_{t-1}^\top]^\top$$

- 9: Take the forward pass:

$$\mathbf{z}_t^{(k)} = \sigma(A_k \mathbf{u}_t^{(k)} + b_k)$$

- 10: **end for**
- 11: Construct E_k and Z_k as in definitions (5.17)
- 12: Estimate the readout matrix in closed form:

$$\widehat{W}_k = (Z_k^\top Z_k + \lambda I_R)^{-1} Z_k^\top E_k$$

- 13: **end for**
-

the fading memory property. Under this notation, the closed form solution of W_k is:

$$\widehat{W}_k = (Z_k^\top Z_k + \lambda I_R)^{-1} Z_k^\top E_k, \quad (5.18)$$

and for $\lambda \rightarrow 0$ this solution converges to the standard OLS solution. Ridge regression provides a good regularization when both $T - t_0 > R$ and $T - t_0 \leq R$ (Ishwaran and Rao 2014). Other details about the loss in RWDN can be found in the Appendix A.

Our closed form estimation relies on the use of fixed ESN states as conditioning variables. If the autoregressive component were modeled using a trainable RNN (as in TimeGrad), the parameters of the recurrent network and the readout matrices would need to be jointly optimized. This would reintroduce the computational burden of TimeGrad since we would need to optimize K observation maps while estimating each observation map of the RWDN. For this reason, we restrict our analysis to the full reservoir computing diffusion architecture combining a Random Weight Denoising Network with ESN-based temporal encoding. Hereafter, we refer to this specification simply as *RWDN*, with the

understanding that the model also incorporates an Echo State Network for the autoregressive component. Indeed, the proposed RWDN estimation procedure is specifically designed for the case in which the temporal conditioning is represented through fixed ESN states. Algorithm 7 summarizes the sequential estimation procedure used to train each random-weight neural network composing the RWDN.

Inference. Sampling is done in the same way as in ESNGrad and TimeGrad (see Algorithm 4). The only major difference is that we use a different noise estimating network at each diffusion step of the process.

RWDN Architecture. The choice of random parameters $(A_k, \mathbf{b}_k)$ is crucial for the performance of RWDN. For this reason, we adopt the sampling scheme of Ballarin et al. (2025). We start by sampling $\tilde{A}_k$ and $\tilde{b}_k$ from the appropriate law: in our case, each entry is $\mathcal{N}(0, 1)$ and $\mathcal{U}(-1, 1)$ respectively. We then define $\bar{A}_k$ and $\bar{b}_k$ as the regularized versions of $\tilde{A}_k$ and $\tilde{b}_k$, in the same way as we did for the ESN (see (5.5)). Finally, we define:

$$A_k = \nu \bar{A}_k, \quad b_k = \varrho \bar{b}_k, \quad (5.19)$$

where $\psi = (\nu, \varrho)$ are the hyperparameters of our RWDN and are shared among the diffusion steps. As an activation function, we use $\tanh(\cdot)$, while the number of diffusion steps K is the same as in TimeGrad and ESNGrad. The parameters of the ESN are drawn in a similar way as explained in Section 5.2.

6 Evaluation on Simulated Time Series

In this Section, we evaluate the performance of the proposed reservoir-based diffusion models, ESNGrad and RWDN, across a set of time series experiments. The objective is to assess both their ability to approximate conditional distributions and their capacity to generate realistic trajectories.

We begin in Section 6.1 with a simple autoregressive AR(1) process, which provides a benchmark setting where the conditional distribution is known in closed-form. This allows for a direct evaluation of the models using Wasserstein distances and standard time series diagnostics. In Section 6.2 we then consider a GARCH(1,1)-like process, which introduces nonlinear dynamics, making the modeling task substantially more challenging.

Across both settings, we compare the generative and predictive performance of ESNGrad and RWDN against the true data-generating process. The evaluation is conducted using distributional metrics, such as the Wasserstein distance, as well as classical tools including autocorrelation analysis and residual diagnostics.

Overall, the results in this Section provide evidence that reservoir-based diffusion models can approximate complex time series dynamics with good accuracy, while maintaining a significantly lower computational cost compared to fully trainable architectures.

Evaluating Conditional Generation. We want to evaluate how well we are able to approximate the conditional distribution $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \dots, \mathbf{x}_1)$ with $p_\theta(\mathbf{x}_t | \mathbf{s}_{t-1})$ for ESNGrad and RWDN with ESN autoregressive component. A natural way to compare distributions is to use their *Wasserstein distance*, also known as Earth Mover distance. Let $x \sim \mu$ and $y \sim \nu$ be two random variables in $\mathbb{R}$, the Wasserstein distance measures how difficult it is to reconfigure the probability mass of μ to obtain ν (Panaretos and Zemel 2019). The Wasserstein-1, W_1 , takes the form of:

$$W_1(\mu, \nu) = \int_0^1 |F_\mu^{-1}(\tau) - F_\nu^{-1}(\tau)| d\tau, \quad (6.1)$$

where F_μ^{-1} and F_ν^{-1} are the quantile functions of μ and ν respectively (Panaretos and Zemel 2019). This closed-form makes W_1 particularly tractable in our setting, as we work mainly with univariate distributions. In the case where the conditional distribution $q(x_t | x_{t-1} = \bar{x}_{t-1}, \dots, x_1 = \bar{x}_1)$ is known, with $\{\bar{x}_j\}_{j=1}^{t-1}$ denoting the observed realizations of the stochastic process, we can directly evaluate W_1 between this conditional distribution and $p_\theta(x_t | \mathbf{s}_{t-1})$, since we can draw an arbitrary number of samples from the DDPM reverse sampler.

In other settings, however, the data-generating process is unknown, or we are interested in assessing whether a trajectory generated by a time series diffusion model is statistically similar to one generated from the true process. Let $\{x_t\}_{t=1}^T$ denote a trajectory from the DGP and $\{\hat{x}_t\}_{t=1}^T$ a trajectory generated by the model. Our objective is to compare the conditional dynamics of the two processes by conditioning on similar values. For both trajectories, we construct tuples of history and next observation of length $V \in \mathbb{N}$:

$$(\mathbf{x}^{(t)}, x_t), \quad (\hat{\mathbf{x}}^{(t)}, \hat{x}_t),$$

for $t = V + 1, \dots, T$, where

$$\mathbf{x}^{(t)} = [x_{t-V}, \dots, x_{t-1}]^\top, \quad \hat{\mathbf{x}}^{(t)} = [\hat{x}_{t-V}, \dots, \hat{x}_{t-1}]^\top.$$

The vectors $\mathbf{x}^{(t)}$ and $\hat{\mathbf{x}}^{(t)}$ represent the historical contexts associated with the subsequent observations x_t and $\hat{x}_t$, respectively. We then define the corresponding conditional distributions $\mu_x(\mathbf{b})$ and $\mu_{\hat{x}}(\mathbf{b})$, with associated conditional densities $p(x_t \mid \mathbf{x}^{(t)} = \mathbf{b})$ and $p(\hat{x}_t \mid \hat{\mathbf{x}}^{(t)} = \mathbf{b})$, respectively. We aim to evaluate how different these distributions are for the two trajectories. To make histories comparable, all windows are standardized using the mean μ_x and standard deviation σ_x of the real series. We then draw M conditioning points from the real trajectory. For each sampled conditioning history $\mathbf{b}^m$, we search for the k closest histories in both the real and generated trajectories using the Euclidean distance. This is equivalent to find the k nearest neighbors:

$$\mathcal{N}_x^m = k\text{-argmin}_{t \neq i} \|\mathbf{b}^m - \mathbf{x}^{(t)}\|_2, \quad \mathcal{N}_{\hat{x}}^m = k\text{-argmin}_t \|\mathbf{b}^m - \hat{\mathbf{x}}^{(t)}\|_2,$$

where $k\text{-argmin}$ indicates an operator that takes the k argument with the smallest distance $\|\mathbf{b}^m - \mathbf{x}^{(t)}\|_2$. The sampled history itself is excluded from the search in the real trajectory to avoid the trivial zero-distance match. The observations associated with the selected histories are then collected to form empirical conditional samples:

$$\mathcal{S}_x^m = \{x_t : t \in \mathcal{N}_x^m\}, \quad \mathcal{S}_{\hat{x}}^m = \{\hat{x}_t : t \in \mathcal{N}_{\hat{x}}^m\}.$$

These empirical samples provide a nonparametric approximation of the conditional distributions for the same conditioning context $\mathbf{b}^m$. Finally, we compute the W_1 distance between the two empirical distributions and average across the M iterations:

$$\widehat{W} = \frac{1}{M} \sum_{m=1}^M W_1(\mathcal{S}_x^m, \mathcal{S}_{\hat{x}}^m). \quad (6.2)$$

This procedure is adapted from the classical *k-Nearest Neighbors algorithm* (kNN) and allows us to compare the conditional behavior of the generated and real trajectories without requiring knowledge of the closed form of the conditional distribution. The complete procedure is summarized in Algorithm 8 in Appendix A.

6.1 A Generative Study of an AR(1) Process

We start with the simple case of an autoregressive process of order one with Gaussian innovations (Hamilton 1994):

$$x_t = \phi x_{t-1} + \varepsilon_t, \quad \varepsilon_t \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2). \quad (6.3)$$

We consider the case where $|\phi| < 1$. For a stationary AR(1) model, the conditional distribution is normal and depends only on the previous observation.

$$q(x_t | x_{t-1}, \dots, x_1) = \mathcal{N}(\phi x_{t-1}, \sigma^2). \quad (6.4)$$

Our goal is to approximate the conditional distribution using our reservoir-based time series diffusion models. We also evaluate the generative ability of ESNGrad and our full-RC model by sampling trajectories and checking the autocorrelation structure. We use $\phi = 0.8$ and $\sigma = 1$, as an example.

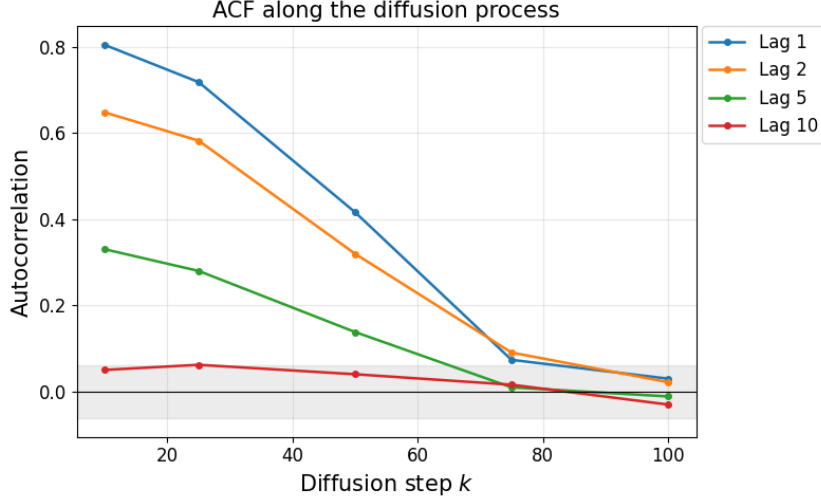


Figure 8: Autocorrelation between x_t and $x_{t-\text{lag}}$ along the diffusion process. For $K = 100$ the autocorrelation structure of $x_t^{(K)}$ resembles the one of an i.i.d. white noise process.

Grey bands indicate 95% confidence intervals under the white noise null hypothesis.

Setting. For each time series diffusion model we need to define our fix forward process. We adopt the linear schedule of Rasul et al. (2021) from $\beta_1 = 10^{-4}$ to $\beta_K = 0.1$. The number of diffusion steps can be arbitrarily chosen, with the only requirement that it should be large enough such that the reverse process is approximately normal and that the trajectory x_t is white noise at the end of the diffusion process. We adopt $K = 100$ as in the original TimeGrad paper, since it works well as a rule of thumb (Yan et al. 2021). Figure 8 shows that $K = 100$ seems to work well in our particular case while Table B1 in the Appendix displays the values of the Ljung-Box test for different lags and diffusion steps. We trained our models using $T = 1000$ observations from the AR(1) model and considered a lookback window of $t_0 = 100$ observations. The parameterization of the models can be found in the Appendix.

Model	Wasserstein-1 Distance
DGP vs True	0.041
ESNGrad vs DGP	0.045
RWDN vs DGP	0.053

Table 1: Estimated average Wasserstein-1 distances between the model-implied conditional distributions and the true conditional distribution $\mathcal{N}(\phi x_{t-1}, \sigma^2)$

Conditional distributions. We begin by comparing the model-implied conditional distributions $p_\theta(x_t | \mathbf{s}_{t-1})$ of ESNGrad and RWDN with the corresponding theoretical conditional distribution, for a fixed conditioning. recall that we can generate from the learned conditional distributions by running the reverse process of Algorithm 5. Table 1 reports the average Wasserstein-1 distance between samples drawn from the estimated conditional distribution (for $n = 1000$ draws from the DDPM reverse sampler) and samples drawn from the true distribution.

A key feature of our models is their path dependence: since the ESN state $\mathbf{s}_{t-1}$ is constructed from the entire past trajectory, sampling from $p_\theta(x_t | \mathbf{s}_{t-1})$ is only meaningful

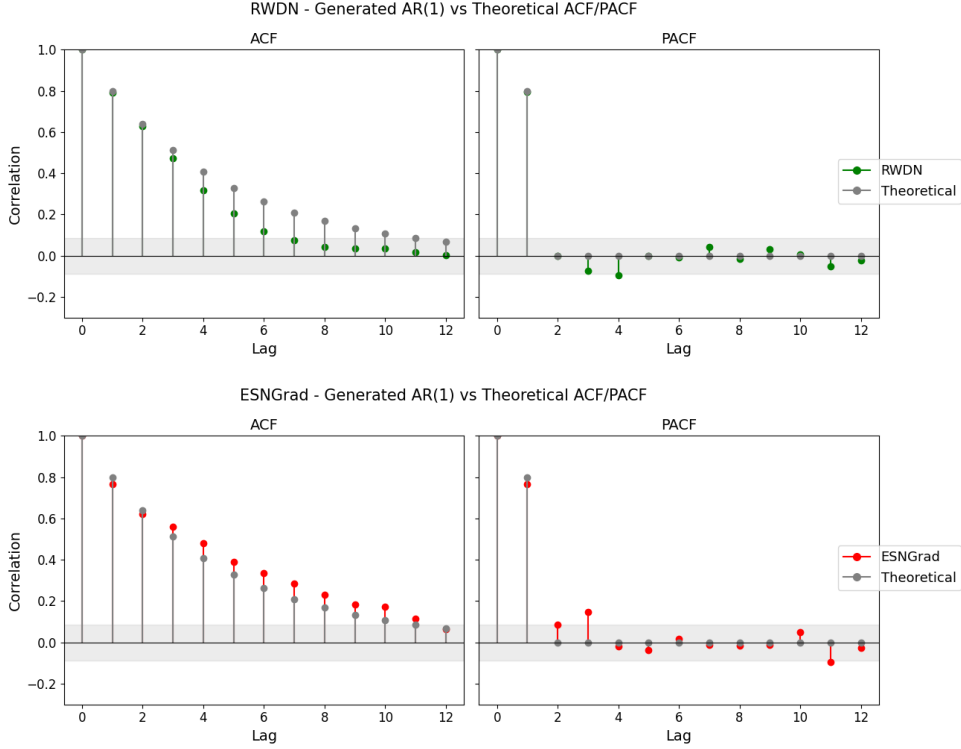


Figure 9: ACF and PACF plots for the trajectories generated by our time series diffusion models against the true autocorrelation structure of an AR(1) model with $\phi = 0.8$. The gray area shows the confidence bounds.

conditional on a given realization of the state. Moreover, because the Wasserstein-1 distance depends on the support, the comparison must be made under the same conditioning value, here fixed at x_{t-1} . Overall, the results indicate that both models provide a good approximation of the true conditional distribution of the autoregressive process. This finding is further supported by a visual inspection: Figure B1 in Appendix B compares $n = 1000$ samples from the estimated conditional distributions with the corresponding theoretical density, showing a good fit of our models.

Generated trajectories. We generate synthetic trajectories of length $n = 1000$ by iteratively sampling from the reverse diffusion process described in Section 4. Figure 9 shows that both ESNGrad and RWDN are able to reproduce the autocorrelation structure of the underlying AR(1) process, even in the presence of strong persistence. However, because trajectories are generated recursively and each new observation is conditioned on previously simulated values, estimation errors may accumulate over time. This limitation is inherent in the TimeGrad framework, which is designed primarily for probabilistic forecasting rather than for direct joint sampling. In contrast, alternative diffusion-based approaches that explicitly model the joint distribution can offer improved performance for long-term trajectory generation (Lin et al. 2024).

To further assess the fidelity of the generated series, we estimate the autoregressive parameter ϕ via maximum likelihood on the simulated data. The results indicate a close match with the true data-generating process: we obtain $\hat{\phi} = 0.75$ for ESNGrad and $\hat{\phi} = 0.78$ for RWDN. These estimates are close to the true parameter value, suggesting that both models are able to capture the underlying linear dependence structure of the process with

good accuracy.

Finally, we compute the conditional Wasserstein-1 distance using the kNN-based procedure described in Algorithm 8 in Appendix A. Table 2 reports the average W_1 distance and its standard deviation, obtained with a lookback window $V = 12$, $M = 100$ conditioning points, and $k = 20$ nearest neighbors, comparing generated trajectories to a reference trajectory from the true data-generating process. The choice of the history length V is guided by the largest significant lag in the autocorrelation function (ACF). Unlike the W_1 reported in Table 1, which compares conditional distributions under a fixed conditioning, this metric evaluates similarity by conditioning on *comparable histories* across the two processes. For an AR(1) process, such a procedure is technically superfluous, as the conditional distribution is known in closed-form. Nevertheless, we report this measure to illustrate that this evaluation strategy works for a general setting.

Model	$W_1(S_x(\mathbf{b}^m), S_{\hat{x}}(\mathbf{b}^m))$	
	Mean	Std
DGP vs DGP	0.47	0.20
RWDN vs DGP	0.45	0.21
ESNGrad vs DGP	0.56	0.25

Table 2: Wasserstein-1 distance using $M = 100$ and $k = 20$ closest histories on generated trajectories of $n = 1000$ points

Residual Diagnostics. Another way to assess whether the models adequately capture the AR(1) structure is to analyze the residuals. Using the true autoregressive parameter, we construct residuals from the generated trajectories and evaluate whether they behave as an i.i.d white noise process. As reported in Table 3, the Wasserstein-1 distance between the unconditional distribution of residuals obtained from our generative models and that derived from a counterfactual AR(1) trajectory indicates a close match to the underlying data-generating process. Additional evidence is provided by the residual histogram, shown in Figure B2 and the QQ-Plots in Figure B4. Moreover, Figure B3 shows that the residuals are essentially uncorrelated over time.

Model	Wasserstein-1 Distance
DGP vs DGP	0.041
ESNGrad vs DGP	0.036
RWDN vs DGP	0.039

Table 3: Estimated average Wasserstein-1 distances between the model residuals and true AR(1) residuals

Computational Efficiency. We evaluate the computational efficiency of the proposed models by comparing their training convergence with TimeGrad. Specifically, we consider a TimeGrad baseline consisting of a single LSTM layer with a finite lookback window of size $p = 13$. For ESNGrad and TimeGrad, we measure the training time required to reach a fixed training loss threshold. Since the RWDN model is trained in closed form and follows a different convergence behavior, we instead use its average training loss,

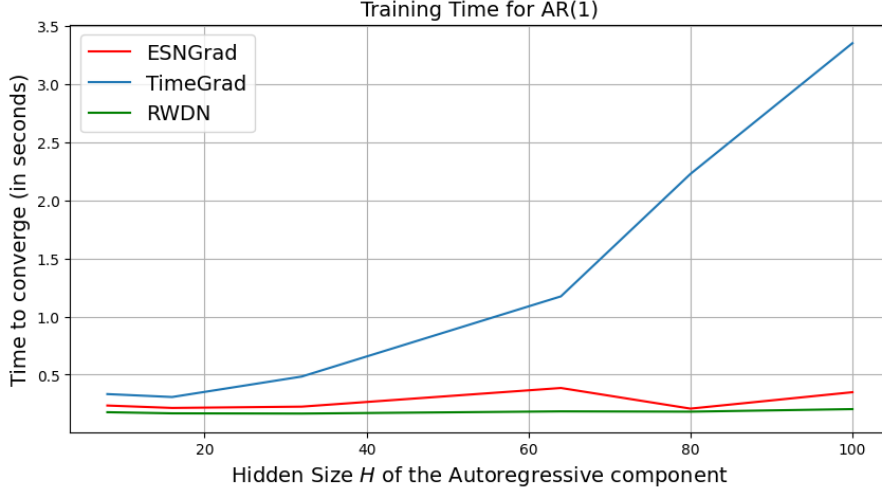


Figure 10: Training time for different hidden sizes H of the temporal encoding $\mathbf{s}_t$.

$L_{\text{simple}} = 0.375$, as a reference threshold. We then report the time required for ESNGrad and TimeGrad to reach this value. In these models, the diffusion loss is computed as an equally weighted sum over all diffusion steps, which allows for a consistent comparison of convergence speed across methods. However, it is important to note that training loss alone does not directly reflect forecasting or generation quality. In particular, diffusion models may achieve low training loss while still producing poor samples. As a consequence, comparing models solely based on convergence time does not provide a fully comprehensive evaluation, especially when contrasting RWDN with ESNGrad and TimeGrad. For this purposes, we will adopt a comparison that considers both forecasting quality and training time in Section 7.

Figure 10 shows that the reservoir-based models achieve faster convergence across all values of the hidden state dimension $\mathbf{s}_t$. Moreover, the training time of TimeGrad increases significantly as the hidden size H grows, whereas the reservoir-based models are less affected by this issue. This behavior is consistent with a well-known limitation of recurrent neural networks, namely their poor scalability with respect to model size due to the cost of backpropagation through time and optimization instability in long sequences (Orvieto et al. 2023). Note that these results are reported for a univariate forecasting setting; in higher-dimensional, where the effective sequence complexity and parameter interactions increase, these scalability issues are expected to become even more pronounced.

6.2 A Generative Study of a GARCH(1,1) Process

As a second example, we consider a stationary GARCH(1,1) process (Hamilton 1994):

$$r_t = \sigma_t z_t, \quad \sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2, \quad (6.5)$$

where $\alpha + \beta < 1$ and $z_t \sim \mathcal{N}(0, 1)$. The unconditional variance is given by $\mathbb{E}[\sigma_t^2] = \frac{\omega}{1 - (\alpha + \beta)}$. We set $\alpha = 0.5$ and $\beta = 0.45$, and choose ω such that the unconditional variance is equal to one.

Setting. We define a training sample of 2000 data points and an out-of-sample of 500 points. We use the same parameterization of the forward process and the same hyperpa-

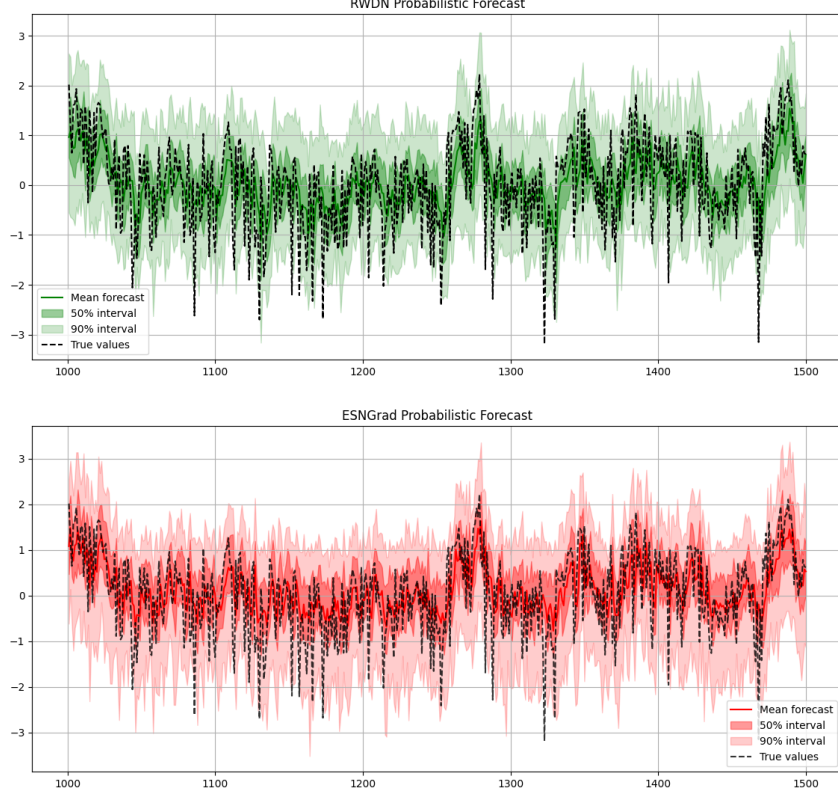


Figure 11: Rolling window probabilistic forecast of ESNGrad and RWDN

rameters for our networks as in the AR(1) case.

Our objective is to model the process r_t^2 using our random weight time series diffusion models. This task is challenging because r_t^2 is constrained to be non-negative and its distribution is typically highly skewed, with a large mass concentrated close to zero. These features create difficulties for diffusion models based on Gaussian perturbations. In particular, our implementation relies on the standard noise-prediction parameterization, where the network estimates the noise added at each diffusion step. While this choice stabilizes training, it does not enforce any constraint on the support of the reconstructed samples. As a result, the reverse process on r_t^2 produces negative values and accumulates errors near the boundary, especially when the signal is close to zero. One possible alternative would be to directly predict the clean sample $\mathbf{x}^{(0)}$ from the noisy input $\mathbf{x}^{(k)}$, as proposed in Karras et al. (2022), which allows for more control over the support of the generated values. However, in this thesis we adopt a simpler approach and address the issue through a transformation of the data. Specifically, we consider the logarithm of squared returns, $y_t = \log r_t^2$, which maps the positive domain to the real line and reduces the skewness of the distribution. We further standardize y_t to improve the conditioning of the neural network. This leads to more stable and accurate predictions.

Generated trajectories. We generate synthetic trajectories of length $\tilde{T} = 1000$ by iteratively sampling from the reverse diffusion process, as in the AR(1) setting. Figure B5 in the Appendix compares the autocorrelation function of the generated series with that of the true GARCH(1,1) process, which indicates that both ESNGrad and RWDN are able to reproduce the persistence of the process dynamics quite well.

We also illustrate how to perform probabilistic forecasting using ESNGrad and RWDN.

Figure 11 presents a one step ahead rolling window probabilistic forecasts obtained from the two models. The forecast is done according to Algorithm 9. We iteratively sample $n = 100$ observations from the conditional distribution and construct empirical quantiles and the conditional mean. We iteratively feed the new realization of our $\log r_t^2$ in our model and proceed until the end of the test sample. In this way, we obtained 500 out-of-sample predictions. Both models closely track the conditional mean of the process and produce well-calibrated predictive distributions, even during periods of high volatility.

To provide a quantitative assessment of the generative performance, we compute the conditional Wasserstein-1 distance using the kNN-based procedure described in Algorithm 8. Table 4 reports the average distance and its standard deviation, comparing trajectories generated by our models with those from the true data-generating process.

Model	$W_1(S_x(\mathbf{b}^m), S_{\hat{x}}(\mathbf{b}^m))$	
	Mean	Std
DGP vs DGP	0.3192	0.1210
RWDN vs DGP	0.3361	0.1461
ESNGrad vs DGP	0.4114	0.1553

Table 4: Conditional Wasserstein-1 distance for the GARCH(1,1) process using $L = 12$, $M = 100$, and $k = 20$.

The results indicate that both models provide a reasonable approximation of the conditional distribution of the $\log r_t^2$ process, with RWDN achieving a closer match to the true data-generating process. As in the AR(1) case, the benchmark highlights the intrinsic variability of the metric due to finite-sample effects.

7 An Application to Economic Data

In this section, we evaluate the performance of the proposed reservoir-based generative models on real-world data. As a benchmark, we compare our approaches with TimeGrad, which provides a well-established reference framework for diffusion-based time series modeling. Our primary objective is to assess whether the computational gains of ESNGrad and RWDN can be achieved without sacrificing predictive and generative accuracy. We consider a standard application in macroeconomics: probabilistic forecasting of consumer price inflation, a problem known as measuring the Inflation-at-Risk (López-Salido and Loria 2024). We evaluate in-sample performance by comparing conditional distributions using the Wasserstein distance and by analyzing the implied correlation structure. In addition, we conducted an out-of-sample analysis to assess the forecasting performance of our models relative to TimeGrad and a simple autoregressive model.

7.1 Data and Setting

We use the monthly seasonally adjusted Consumer Price Index for All Urban Consumers: All Items in U.S. City Average (CPIAUCSL) from the Federal Reserve Bank of St. Louis database.¹² The sample spans January 1960 to December 2023 and includes episodes of pronounced inflation dynamics, such as the Great Inflation (1965–1982) and the post-COVID inflation surge. Following , we work with the first difference of annualized inflation.¹³ The dataset is split into a training sample of $t_n = 651$ months (approximately 85% of the data) and a test sample of 115 months. Both series are standardized using the mean and standard deviation computed on the training set. We evaluate ESNGrad and RWDN and compare them to a TimeGrad model with LSTM as autoregressive component. Model parameterizations are reported in the Appendix: for TimeGrad, we follow Rasul et al. (2021), while for ESN-based models we adopt standard choices from the reservoir computing literature (Ballarin et al. 2025).

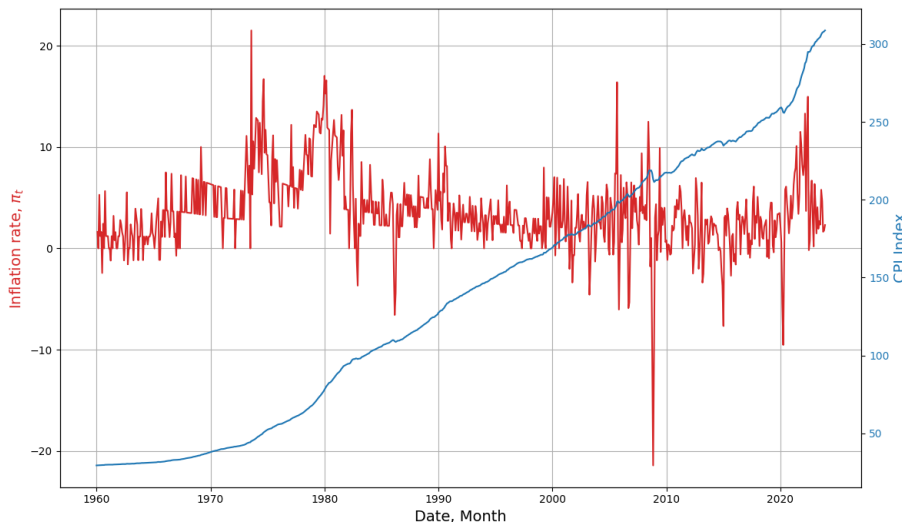


Figure 12: Consumer Price for All Urban Consumers: All Items in U.S. City Average. Index and inflation rate. The data source is the Federal Reserve of St. Louis Database.

¹²The data are available at: <https://fred.stlouisfed.org/series/CPIAUCSL>.

¹³See Stock and Watson (2007) for an ARIMA-based analysis of U.S. inflation.

We conduct a rolling-window probabilistic forecasting exercise based on one-step-ahead predictions. For each time point t in the test set, we draw $n = 100$ samples from the conditional distribution $p(y_t \mid \mathbf{s}_{t-1})$. From these samples, we estimate the conditional mean as well as empirical quantiles, including 50% and 90% prediction intervals. This procedure is described in Algorithm 9 in Appendix A.¹⁴ As a simple benchmark, we also include an AR(1) model.

To assess computational efficiency, we compare training times across models. Since all approaches rely on the same DDPM-based sampling procedure (Algorithm 5), differences at inference time are limited, aside from minor gains due to reduced architectural complexity.¹⁵ We also consider alternative RNN architectures to benchmark efficiency.

Finally, we evaluate in-sample performance by comparing the autocorrelation structure (ACF and PACF) of trajectories generated by the models with that of the observed CPI series. In addition, we compute the empirical conditional Wasserstein-1 distance using Algorithm 8 to compare the similarity between generated and observed trajectories. Since the true data-generating process is unknown, this metric provides a relative measure to assess how closely the different models reproduce the observed dynamics.

Models. The parameterization of the models can be found in Table B3 in the Appendix. Our RC based models share similar features as in Section 6. For TimeGrad, we used an LSTM as an autoregressive component. Moreover for TimeGrad we adopt a limited lookback component as described in Section 4.2: the state $\mathbf{s}_t$ is generated only using the past p values $\{x_{t-p}, \dots, x_t\}$. The length p is selected by using the last significant lag of the ACF of our training data.

Forecast Assessment. Forecast performance is assessed using standard loss metrics such as Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) for our conditional mean prediction. We also use the Continuous Ranked Probability Score (CRPS), a standard metric in probabilistic forecasting. For a predictive cumulative distribution function F and realization x , the CRPS is defined as

$$\text{CRPS}(F, y) = \int_{-\infty}^{+\infty} (F(z) - \mathbb{1}\{z \geq x\})^2 dz \quad (7.1)$$

where z is the integration variable and F denotes the predictive cumulative distribution function (Rasul et al. 2021). In practice, F is replaced by its empirical counterpart $\hat{F}$ obtained from samples generated by the diffusion model. Given n samples $\{\hat{x}_t^{(i)}\}_{i=1}^n$ drawn from the predictive distribution at time t , the CRPS can be empirically computed as

$$\text{CRPS}_t = \frac{1}{n} \sum_{i=1}^n \left| \hat{x}_t^{(i)} - x_t \right| - \frac{1}{2n^2} \sum_{i=1}^n \sum_{j=1}^n \left| \hat{x}_t^{(i)} - \hat{x}_t^{(j)} \right| \quad (7.2)$$

$$\text{CRPS} = \frac{1}{T} \sum_{t=1}^T \text{CRPS}_t. \quad (7.3)$$

¹⁴In Algorithm 9, the superscript indexes Monte Carlo samples and does not refer to the diffusion step.

¹⁵Acceleration at inference time could be achieved using DDIM sampling (Song et al. 2021a). However, in time series applications the number of diffusion steps is typically modest, and in our experiments accurate reconstructions are obtained early in the reverse process.

Note that for an AR(1) the conditional distribution is Gaussian $\mathcal{N}(\hat{\phi}y_{t-1}, \hat{\sigma}^2)$. In such a case, we can use the result from [Gneiting et al. \(2005\)](#) that for a Gaussian distribution, the CRPS can be computed in a close form:

$$\text{CRPS}_t(\mathcal{N}(\mu_t, \sigma^2), x_t) = \sigma \cdot \left(\frac{x_t - \mu_t}{\sigma} \left[2\Phi\left(\frac{x_t - \mu_t}{\sigma}\right) - 1 + 2\phi\left(\frac{x_t - \mu_t}{\sigma}\right) - \frac{1}{\sqrt{\pi}} \right] \right) \quad (7.4)$$

where μ_t and σ are the first two moments of the conditional distribution $p(x_t|x_{t-1})$ of an AR(1) process, $\Phi(\cdot)$ and $\phi(\cdot)$ are, respectively, the cdf and pdf of a $\mathcal{N}(0, 1)$.

7.2 Results

We begin by discussing the results of the analysis of the in-sample generative performance of the models and then turn to the rolling-window one-step-ahead forecasting exercise.

In-Sample Performance. We trained our diffusion models on a relatively large training sample, especially considering the low-frequency nature of macroeconomic data. At convergence, all models achieved a similar value of the DDPM training objective, with $L_{\text{simple}} \approx 0.35$, indicating that the denoising networks were able to accurately estimate the noise added during the forward diffusion process. Recall that in the DDPM framework a value of $L_{\text{simple}} = 1$ corresponds to the case in which the model is unable to learn the noise structure and effectively returns random predictions. Indeed, since the objective is based on the mean squared error and the target noise has zero mean and unit variance, a model that outputs only zeros would achieve an expected loss equal to one. Note again that since for all models, except ESNGrad and our RWDN, we are only passing a limited history and the issue vanishing or exploding gradient is implicitly avoided. From various experiments, we noted that the performance of training is unstable and the fit is low when using a large sample size and a plain vanilla RNN to encode the full history.

Metric	TG(LSTM)	TG(RNN)	TG(GRU)	ESNGrad	RWDN
Relative Training Time	1.0	1.37	2.77	0.21	0.03
Conditional W_1	1.35	1.36	1.39	1.35	1.40

Table 5: Performance of different TimeGrad-like models using different recurrent neural network. The relative training time compares the models with TimeGrad with LSTM temporal encoding. The W_1 metric is computed according to algorithm 8 for trajectories of $n = 650$ observations using $V = 11$, $k = 30$ and $M = 250$

We considered several TimeGrad specifications employing different recurrent architectures for temporal encoding, including standard RNNs, GRUs, and LSTMs, and generated synthetic trajectories with the same length as the training sample. We performed the same experiment for the proposed reservoir computing models, ESNGrad and RWDN. Figure B6 shows that all diffusion-based specifications are able to reproduce reasonably well the autocorrelation structure of the CPI series, both in terms of the ACF and PACF. In particular, the reservoir-based models appear to retain the relevant persistence dynamics of the data despite relying on fixed random recurrent representations.

To quantitatively assess the similarity between generated and observed trajectories, we employ the empirical conditional Wasserstein metric introduced in Algorithm 8. Table 5

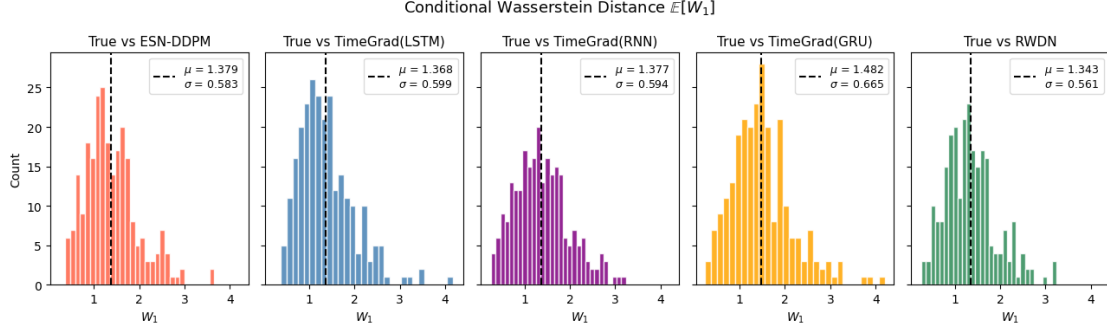


Figure 13: Distribution of the conditional W_1 metrics for different diffusion probabilistic models

reports the average conditional Wasserstein-1 distance computed on generated trajectories of length $n = 650$, using a lookback window $V = 11$, $k = 30$ nearest neighbors, and $M = 250$ conditioning points. The results indicate that ESNGrad achieves a conditional Wasserstein distance comparable to the LSTM-based TimeGrad specifications, while requiring only a fraction of the training time. The fully reservoir-based RWDN exhibits a slightly larger distance. Overall, the results suggest that reservoir-based diffusion models are able to approximate the conditional dynamics of the CPI process with an accuracy comparable to fully trainable recurrent architectures.

Computational Efficiency. The proposed reservoir-based models are the clear winners in terms of training efficiency. We compare the computational time required for the different models to converge under the same convergence criterion. In our experiments, ESNGrad required 80% less time to train relative to TimeGrad, while RWDN required only a few seconds to train, thus achieving a speed up of more than 95%. As already mentioned before, for the RWDN specification, however, the notion of convergence is somewhat different, since the model parameters are obtained through a chain of closed-form regularized least-squares problems rather than through iterative gradient-based optimization. For this reason, the reported training time for RWDN corresponds to the total time required to estimate the K ridge regressions. Consequently, this comparison is not entirely homogeneous with the gradient-trained architectures.

In principle, we could have compared the time required for TimeGrad and ESNGrad to reach the same average training loss achieved by RWDN, as done in Section 6. However, in our forecasting setting this criterion led to substantially different outcomes in terms of predictive performance, making it less suitable as a basis for comparison. For this reason, we adopt a balanced evaluation protocol based on early stopping with a patience of $P = 25$ epochs and a batch size of $B = 64$, ensuring that each model is allowed to observe at least one full pass over the training data before termination.

Overall, the results indicate that replacing fully trainable recurrent components with fixed reservoir representations yields a substantial reduction in computational cost, while maintaining comparable predictive and generative accuracy. Notably, these gains are already significant in the univariate setting considered here and are expected to become even more pronounced in higher-dimensional multivariate scenarios.

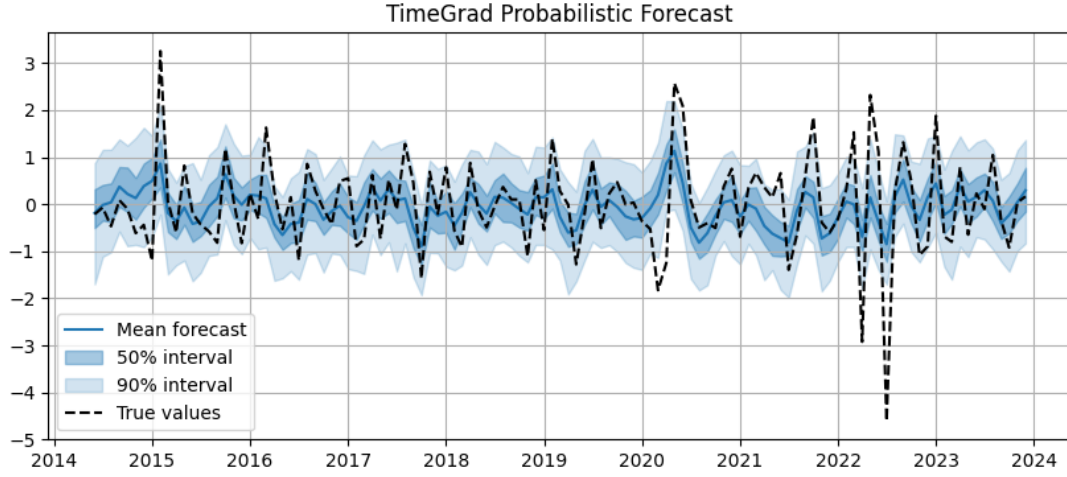
Forecasting performance. Table 6 reports the out-of-sample forecasting results for the recursive one-step-ahead exercise. Overall, the diffusion models outperform the simple

AR(1) benchmark across all point forecasting metrics. In particular, ESNGrad, TimeGrad with LSTM temporal encoding, and RWDN all achieve lower RMSE and MAE values, indicating a better ability to capture the nonlinear dynamics of the inflation process. Among the proposed specifications, RWDN achieves the lowest RMSE, while TimeGrad achieves the best CRPS, suggesting a slightly more accurate calibration of the full predictive distribution. Nevertheless, the differences between the reservoir-based models and the LSTM-based specification remain very small, showing that the proposed reservoir architectures are able to match the forecasting accuracy of fully trainable recurrent models. We believe that, since we have not extensively tuned hyperparameters and sampling distributions properties (such as sparsity) of our RC-based models, the accuracy of the distributional forecast could be further improved.

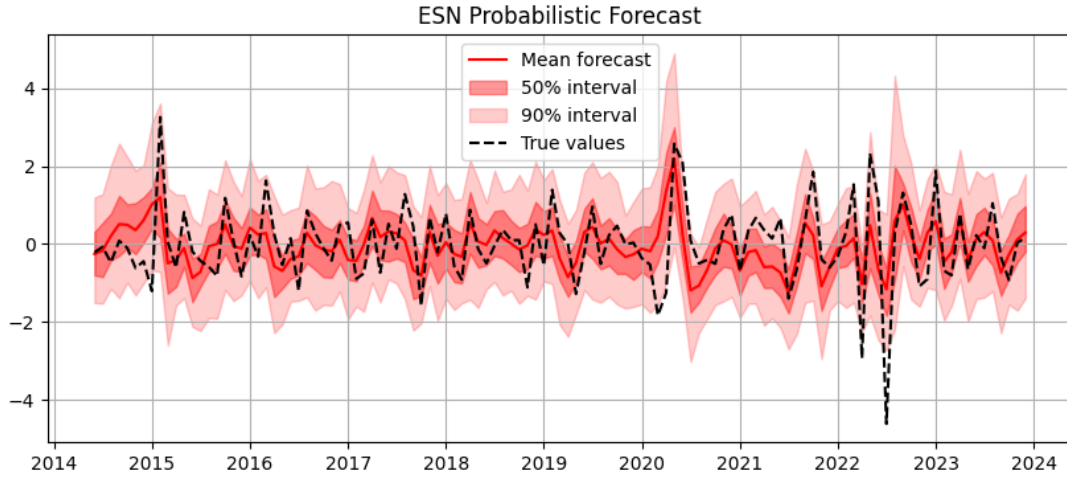
Metric	AR(1)	TimeGrad	ESNGrad	RWDN
RMSE	1.0011	0.88	0.88	0.87
MAE	0.7321	0.64	0.64	0.66
50% Coverage	–	50%	55%	39%
90% Coverage	–	90%	92%	83%
Average CRPS	0.54	0.47	0.49	0.48
Relative CRPS (%)	–	12.2	9.9	11.4

Table 6: Performance metrics for the recursive one-step ahead on the test set. 50% and 90% Coverage refer to the share of the realizations that are inside the 50% and 90% intervals. Relative CRPS indicates how much better, in percentage points, the model performs relative to the AR(1) forecast.

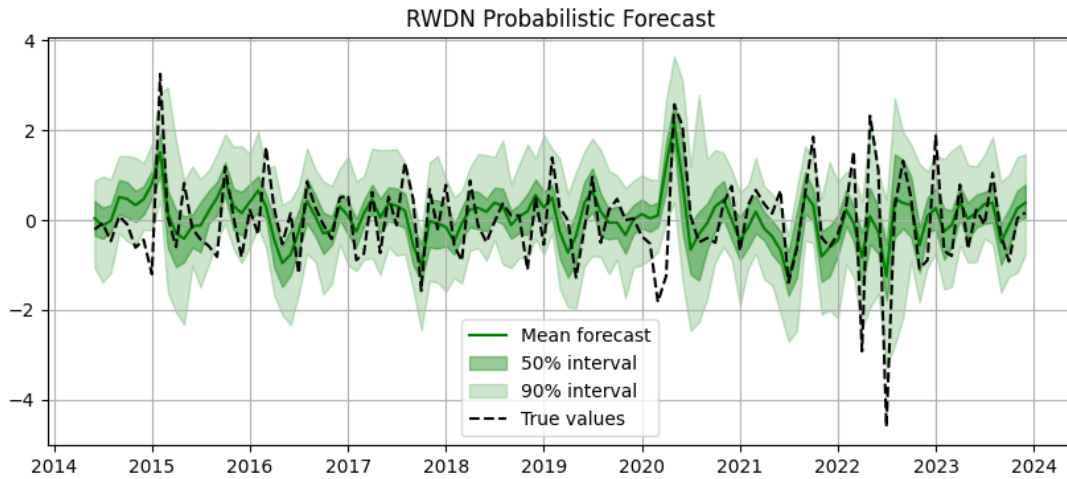
The coverage probabilities further confirm the quality of the probabilistic forecasts. ESNGrad and TimeGrad produce empirical coverage rates close to the nominal levels for both the 50% and 90% prediction intervals, whereas RWDN tends to generate slightly narrower predictive distributions, leading to some undercoverage, especially for the 50% interval. Note that this does not necessarily mean that our empirical forecasting distribution matches the empirical quantiles of the true conditional distribution. Figure 14 complements these quantitative results by displaying the probabilistic forecasts over the test period. All models are able to track the conditional mean dynamics of the inflation process reasonably well, including periods of increased volatility. The main discrepancies occur at the beginning of the test sample, where the models tend to temporarily overestimate the inflation change. Overall, the results suggest that the proposed reservoir-based diffusion models provide probabilistic forecasts comparable to those obtained with TimeGrad, while relying on a substantially simpler and computationally cheaper architecture.



(a) TimeGrad with LSTM temporal encoding ($n = 100$ forecasts per step).



(b) ESNGrad probabilistic forecast ($n = 100$ forecasts per step).



(c) Random Weight Denoising Network with ESN temporal encoding ($n = 100$ forecasts per step).

Figure 14: One-step-ahead probabilistic forecasts for U.S. CPI for TimeGrad, ESNGrad and RWDN

8 Conclusion

This work introduced ESNGrad and RWDN, two novel time series diffusion models that combine the reservoir computing paradigm with the TimeGrad framework, one of the most well-known architectures for diffusion-based probabilistic forecasting. Building on Denoising Diffusion Probabilistic Models, our models replace the fully trainable recurrent component by a state space model with random parameters. In the case of RWDN, we also replace the denoising network itself with a sequence of random-weight neural networks. In doing so, the proposed models avoid backpropagation through time and substantially reduce the computational burden associated with training diffusion models for time series data. To the best of our knowledge, this is the first work to integrate reservoir computing with time series diffusion models and one of the first to provide a quantitative evaluation of random-weight architectures in a diffusion-based generative setting.

The empirical results on simulated data show that ESNGrad and RWDN are able to closely approximate the target conditional distributions, achieving Wasserstein distances comparable to those obtained from the true DGP. We further demonstrated that both models can successfully perform probabilistic forecasting and synthetic trajectory generation. In the empirical application to U.S. inflation data, the proposed reservoir-based diffusion models achieved forecasting and generative performance comparable to TimeGrad with LSTM temporal encoding, while requiring only a fraction of the computational cost. In particular, the estimation of RWDN reduces to a sequence of closed-form ridge regression problems, leading to training times that are approximately two orders of magnitude lower than those of fully trainable recurrent architectures. Although the present work focused on univariate macroeconomic time series, these computational gains are expected to become even more relevant in high-dimensional settings.

More broadly, the results suggest that diffusion models represent a promising alternative to traditional approaches for probabilistic macroeconomic forecasting, such as parametric density models or quantile regression techniques. In particular, the proposed framework provides a flexible nonparametric approach for studying problems such as Inflation-at-Risk or Growth-at-Risk without imposing restrictive assumptions on the shape of the conditional distribution. While diffusion models have recently attracted growing attention in finance and econometrics, their application to economic time series remains relatively limited compared to their widespread success in image, audio, and text generation. We believe that this represents a particularly promising direction for future research.

At the same time, both reservoir computing and diffusion-based generative modeling remain relatively new fields of study. The objective of this thesis is to demonstrate the potential of combining the computational efficiency of reservoir computing with the generative capabilities of diffusion models, but many important theoretical and methodological questions remain open. On the architectural side, a more principled analysis of hyperparameter selection for the ESN and RWDN components (e.g. reservoir size, spectral radius, sparsity, and input scaling) could lead to significant improvements, especially in multivariate and high-dimensional environments where the curse of dimensionality plays a significant role. Extending the framework to large multivariate systems may also require more structured reservoir architectures or alternative random features schemes.

From a methodological perspective, integrating reservoir computing with the stochastic differential equation (SDE) formulation of diffusion models appears particularly promising (Yan et al. 2021; Meijer and Chen 2023). In particular, the use of ODE-based or accelerated reverse process solvers could substantially reduce inference time, complementing the training-time gains already achieved in this work and potentially enabling highly efficient end-to-end generative models for time series. More generally, we hope that this thesis contributes to laying the foundations for a broader exploration of random-weight methods in generative modeling, an area that remains largely unexplored despite the strong theoretical guarantees and simplicity offered by reservoir computing.

References

- Acciaio, B., Eckstein, S., and Hou, S. (2024). Time-causal vae: Robust financial time series generator. *arXiv preprint arXiv:2411.02947*.
- Adrian, T., Boyarchenko, N., and Giannone, D. (2019). Vulnerable growth. *American Economic Review*, 109(4):1263–1289.
- Adrian, T., Sasaki, Y., and Wang, Y. (2026). Robust econometrics for growth-at-risk. *Journal of Econometrics*, 255:106235.
- Ballarin, G., Capra, J., and Dellaportas, P. (2025). Multi-horizon echo state network prediction of intraday stock returns. *arXiv preprint arXiv:2504.19623*.
- Ballarin, G., Dellaportas, P., Grigoryeva, L., Hirt, M., van Huellen, S., and Ortega, J.-P. (2024). Reservoir computing for macroeconomic forecasting with mixed-frequency data. *International Journal of Forecasting*, 40(3):1206–1237.
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734.
- Coletta, A., Jerome, J., Savani, R., and Vyetenko, S. (2023). Conditional generators for limit order book environments: Explainability, challenges, and robustness. *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 27–35.
- Dhariwal, P. and Nichol, A. (2021). Diffusion models beat gans on image synthesis. *arXiv preprint arXiv:2105.05233*.
- Diebold, F. X., Gunther, T. A., and Tay, A. S. (1998). Evaluating density forecasts with applications to financial risk management. *International Economic Review*, 39(4):863–883.
- Durbin, J. and Koopman, S. J. (2012). *Time Series Analysis by State Space Methods*. Oxford University Press, 2 edition.
- Efron, B. (2011). Tweedie’s formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614.
- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4):987–1007.
- Geweke, J. (1989). Bayesian inference in econometric models using Monte Carlo integration. *Econometrica*, 57(6):1317–1339.
- Geweke, J. and Amisano, G. (2010). Comparing and evaluating Bayesian predictive distributions of asset returns. *International Journal of Forecasting*, 26(2):216–230.

- Gneiting, T., Raftery, A. E., III, A. H. W., and Goldman, T. (2005). Calibrated probabilistic forecasting using ensemble model output statistics and minimum crps estimation. *Monthly Weather Review*, 133(5):1098–1118.
- Gonon, L., Grigoryeva, L., and Ortega, J.-P. (2020). Risk bounds for reservoir computing. *Journal of Machine Learning Research*, 21(240):1–61.
- Gonon, L., Grigoryeva, L., and Ortega, J.-P. (2023). Approximation bounds for random neural networks and reservoir systems. *The Annals of Applied Probability*.
- Gonon, L. and Ortega, J.-P. (2020). Reservoir computing universality with stochastic inputs. *IEEE Transactions on Neural Networks and Learning Systems*, 31(1):100–112.
- Gordon, N. J., Salmond, D. J., and Smith, A. F. M. (1993). Novel approach to nonlinear and non-Gaussian Bayesian state estimation. *IEE Proceedings F – Radar and Signal Processing*, 140(2):107–113.
- Grigoryeva, L. and Ortega, J.-P. (2018). Echo state networks are universal. *Neural Networks*, 108:495–508.
- Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press, Princeton, NJ.
- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851.
- Ho, J. and Salimans, T. (2021). Classifier-free diffusion guidance. *OpenReview*. Available at OpenReview.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3 edition.
- Hyvärinen, A. (2005). Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709.
- Ishwaran, H. and Rao, J. (2014). Geometry and properties of generalized ridge regression in high dimensions. In Ahmed, S., editor, *Contemporary Mathematics*, volume 622, pages 81–93. American Mathematical Society, Providence, Rhode Island.
- Karras, T., Aittala, M., Aila, T., and Laine, S. (2022). Elucidating the design space of diffusion-based generative models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:26565–26577.
- Kingma, D. P., Salimans, T., Poole, B., and Ho, J. (2021). Variational diffusion models. *Advances in Neural Information Processing Systems*, 34:2192–2205.
- Kingma, D. P. and Welling, M. (2019). An introduction to variational autoencoders. *Foundations and Trends in Machine Learning*, 12(4):307–392.
- Lin, L., Li, Z., Li, R., et al. (2024). Diffusion models for time-series applications: a survey. *Frontiers of Information Technology & Electronic Engineering*, 25:19–41.

- Litterman, R. B. (1986). Forecasting with bayesian vector autoregressions: Five years of experience. *Journal of Business Economic Statistics*, 4(1):25–38.
- Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. (2022). Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:5775–5787.
- Luo, C. (2022). Understanding diffusion models: A unified perspective. *arXiv preprint arXiv:2208.11970*.
- López-Salido, D. and Loria, F. (2024). Inflation at risk. *Journal of Monetary Economics*, 145:103570. Inflation: Expectations Dynamics October 14-15, 2022.
- Meijer, C. and Chen, L. Y. (2023). The rise of diffusion models in time-series forecasting. *arXiv preprint. EEMCS Distributed Systems*.
- Nagy, P., Frey, S., Sapora, S., Li, K., Calinescu, A., Zohren, S., and Foerster, J. (2023). Generative ai for end-to-end limit order book modelling: A token-level autoregressive generative model of message flow using a deep state space network. *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 91–99.
- Nakkiran, P., Bradley, A., Zhou, H., and Advani, M. (2024). Step-by-step diffusion: An elementary tutorial. *ArXiv*, abs/2406.08929.
- Nichol, A. Q. and Dhariwal, P. (2021). Improved denoising diffusion probabilistic models. *Proceedings of the 38th International Conference on Machine Learning*, 139:8162–8171.
- Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gülçehre, Ç., Pascanu, R., and De, S. (2023). Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*.
- Panaretos, V. M. and Zemel, Y. (2019). Statistical aspects of wasserstein distances. *Annual Review of Statistics and Its Application*, 6:405–431.
- Pascual, L., Romo, J., and Ruiz, E. (2006). Bootstrap prediction for returns and volatilities in GARCH models. *Computational Statistics & Data Analysis*, 50(9):2293–2312.
- Perez, E., Strub, F., de Vries, H., Dumoulin, V., and Courville, A. (2018). Film: Visual reasoning with a general conditioning layer. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*.
- Rasul, K., Seward, C., Schuster, I., and Vollgraf, R. (2021). Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. *Proceedings of the 38th International Conference on Machine Learning*, 139:8857–8868.
- Rudi, A. and Rosasco, L. (2017). Generalization properties of learning with random features. *Proceedings of the 31st International Conference on Neural Information Processing Systems*, page 3218–3228.
- Saha, E. and Tran, G. (2025). Generalization bound for diffusion models using random features. *arXiv preprint arXiv:2310.04417*.

- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. (2015). Deep unsupervised learning using nonequilibrium thermodynamics. *Proceedings of the 32nd International Conference on Machine Learning*, 37:2256–2265.
- Song, J., Meng, C., and Ermon, S. (2021a). Denoising diffusion implicit models. *International Conference on Learning Representations*.
- Song, Y. and Ermon, S. (2019). Generative modeling by estimating gradients of the data distribution. *arXiv preprint arXiv:1907.05600*.
- Song, Y., Garg, S., Shi, J., and Ermon, S. (2020). Sliced score matching: A scalable approach to density and score estimation. *arXiv preprint arXiv:2006.05957*.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2021b). Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*.
- Stock, J. H. and Watson, M. W. (2007). Why has u.s. inflation become harder to forecast? *Journal of Money, Credit and Banking*, 39(s1):3–33.
- Sung, M. (2025). Course slides on diffusion and flow models. <https://diffusion.kaist.ac.kr/>. Course website, Korea Advanced Institute of Science and Technology.
- Tay, A. S. and Wallis, K. F. (2000). Density forecasting: a survey. *Journal of Forecasting*, 19(4):235–254.
- van de Geer, S. (2000). *Empirical Processes in M-Estimation*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, Cambridge.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Vincent, P., Larochelle, H., Lajoie, I., Bengio, Y., and Manzagol, P.-A. (2011). A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674.
- Wang, A., Tong, T. P., and Pang, J. (2023). Effective and efficient structural inference with reservoir computing. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J., editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 36391–36410. PMLR.
- Welling, M. and Teh, Y. W. (2011). Bayesian learning via stochastic gradient langevin dynamics. *Proceedings of the 28th International Conference on International Conference on Machine Learning*, page 681–688.
- Weng, L. (2021). What are diffusion models? <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>. Accessed: 2026-02-22.
- Yan, T., Zhang, H., Zhou, T., Zhan, Y., and Xia, Y. (2021). Scoregrad: Multivariate probabilistic time series forecasting with continuous energy-based generative models. *arXiv preprint arXiv:2106.10121*.

Appendix A: Additional Material

A.1 Forward and Reverse Processes in DDPM

Sohl-Dickstein et al. (2015) show that the forward posterior $q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}, \mathbf{x}^{(0)})$ defined in (3.9) is Gaussian. Since the full derivation is quite lengthy here we just show the main idea. By applying the Bayes rule to write $q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}, \mathbf{x}^{(0)})$ in terms of the forward processes $q(\mathbf{x}^{(t-1)}|\mathbf{x}^{(t)}, \mathbf{x}^{(0)})$ and $q(\mathbf{x}^{(k)}|\mathbf{x}^{(0)})$ we get:

$$\begin{aligned} q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(k)}, \mathbf{x}^{(0)}) &= \frac{q(\mathbf{x}^{(k-1)}, \mathbf{x}^{(k)}, \mathbf{x}^{(0)})}{q(\mathbf{x}^{(k-1)}, \mathbf{x}^{(0)})} \\ &= \frac{q(\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)}, \mathbf{x}^{(0)})q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(0)})q(\mathbf{x}^{(0)})}{q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(0)})q(\mathbf{x}^{(0)})} \\ &= \frac{q(\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)}, \mathbf{x}^{(0)})q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(0)})}{q(\mathbf{x}^{(k)}|\mathbf{x}^{(0)})} \end{aligned}$$

Since $q(\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)}, \mathbf{x}^{(0)}) = \mathcal{N}(\sqrt{\alpha_k}\mathbf{x}^{(0)}, (1 - \alpha_k)I_N)$, $q(\mathbf{x}^{(k-1)}|\mathbf{x}^{(0)}) = \mathcal{N}(\sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)}, (1 - \bar{\alpha}_k)I_N)$ and $q(\mathbf{x}^{(k)}|\mathbf{x}^{(0)}) = \mathcal{N}(\sqrt{\bar{\alpha}_k}\mathbf{x}^{(0)}, (1 - \bar{\alpha}_k)I_N)$. The results is again a Gaussian distribution with mean and variance specified in (3.9). Luo (2022) offers a complete step-by-step derivation.

A.2 Decomposition of the VLB

The variational lower bound of (3.11) can be obtained by exploiting the the fact that the KL divergence is non-negative (Weng 2021):

$$\begin{aligned} -\log p_\theta(\mathbf{x}^{(0)}) &\leq -\log p_\theta(\mathbf{x}^{(0)}) + D_{\text{KL}}(q(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)})||\log p_\theta(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)})) \\ &= -\log p_\theta(\mathbf{x}^{(0)}) + \mathbb{E}_{\mathbf{x}^{(1:K)} \sim q(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)})} \left[\log \frac{q(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)})}{p_\theta(\mathbf{x}^{(0:K)})/p_\theta(\mathbf{x}^{(0)})} \right] \\ &= -\log p_\theta(\mathbf{x}^{(0)}) + \mathbb{E}_q \left[\log \frac{q(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)})}{p_\theta(\mathbf{x}^{(0:K)})} + \log p_\theta(\mathbf{x}^{(0)}) \right] \\ &= \mathbb{E}_{q(\mathbf{x}^{(0:K)})} \left[\log \frac{q(\mathbf{x}^{(1:K)}|\mathbf{x}^{(0)})}{p_\theta(\mathbf{x}^{(0:K)})} \right] \end{aligned}$$

The following derivation shows the decomposition of the Variational Lower Bound to obtain (3.12) as in Sohl-Dickstein et al. (2015):

$$\begin{aligned}
L_{\text{VLB}}(\theta) &= \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}^{(0:K)})}{q(\mathbf{x}^{(1:K)} | \mathbf{x}^{(0)})} \right] \\
&= \mathbb{E}_q \left[-\log p(\mathbf{x}^{(K)}) - \sum_{k \geq 1} \log \frac{p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)})}{q(\mathbf{x}^{(k)} | \mathbf{x}^{(k-1)})} \right] \\
&= \mathbb{E}_q \left[-\log p(\mathbf{x}^{(K)}) - \sum_{k > 1} \log \frac{p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)})}{q(\mathbf{x}^{(k)} | \mathbf{x}^{(k-1)})} - \log \frac{p_\theta(\mathbf{x}^{(0)} | \mathbf{x}^{(1)})}{q(\mathbf{x}^{(1)} | \mathbf{x}^{(0)})} \right] \\
&= \mathbb{E}_q \left[-\log p(\mathbf{x}^{(K)}) - \sum_{k > 1} \log \frac{p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)})}{q(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}, \mathbf{x}^{(0)})} \frac{q(\mathbf{x}^{(k-1)} | \mathbf{x}^{(0)})}{q(\mathbf{x}^{(k)} | \mathbf{x}^{(0)})} - \log \frac{p_\theta(\mathbf{x}^{(0)} | \mathbf{x}^{(1)})}{q(\mathbf{x}^{(1)} | \mathbf{x}^{(0)})} \right] \\
&= \mathbb{E}_q \left[-\log \frac{p(\mathbf{x}^{(K)})}{q(\mathbf{x}^{(K)} | \mathbf{x}^{(0)})} - \sum_{k > 1} \log \frac{p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)})}{q(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}, \mathbf{x}^{(0)})} - \log p_\theta(\mathbf{x}^{(0)} | \mathbf{x}^{(1)}) \right] \\
&= \mathbb{E}_q \left[D_{\text{KL}}(q(\mathbf{x}^{(K)} | \mathbf{x}^{(0)}) \| p(\mathbf{x}^{(K)})) \right. \\
&\quad + \sum_{k > 1} D_{\text{KL}}(q(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}, \mathbf{x}^{(0)}) \| p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)})) \\
&\quad \left. - \log p_\theta(\mathbf{x}^{(0)} | \mathbf{x}^{(1)}) \right]
\end{aligned}$$

The reduced variational lower bound of the negative log-likelihood can be written in close form by exploiting the properties of the KL divergence. Since $q(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}, \mathbf{x}^{(0)}) := \mathcal{N}(\tilde{\boldsymbol{\mu}}_k, \tilde{\Sigma}_k)$ and $p_\theta(\mathbf{x}^{(k-1)} | \mathbf{x}^{(k)}) := \mathcal{N}(\boldsymbol{\mu}_\theta, \Sigma_\theta)$:

$$\begin{aligned}
\arg \min_{\theta} \mathcal{D}_{\text{KL}}(\mathcal{N}(\tilde{\boldsymbol{\mu}}_k, \tilde{\Sigma}_k) \| \mathcal{N}(\boldsymbol{\mu}_\theta, \Sigma_\theta)) &= \\
&= \arg \min_{\theta} \frac{1}{2} \left[\log \frac{|\Sigma_\theta|}{|\tilde{\Sigma}_k|} - N + \text{tr}(\Sigma_\theta^{-1} \tilde{\Sigma}_k) + (\boldsymbol{\mu}_\theta - \tilde{\boldsymbol{\mu}}_k)^\top \Sigma_\theta^{-1} (\boldsymbol{\mu}_\theta - \tilde{\boldsymbol{\mu}}_k) \right].
\end{aligned}$$

Under the assumption of Ho et al. (2020) the loss is a weighted mean squared error between the mean of the variational distribution and the reverse conditional:

$$\begin{aligned}
\arg \min_{\theta} \mathcal{D}_{\text{KL}}(\cdot) &= \min_{\theta} \frac{1}{2} (\boldsymbol{\mu}_\theta - \tilde{\boldsymbol{\mu}}_k)^\top (\tilde{\beta}_k I_N)^{-1} (\boldsymbol{\mu}_\theta - \tilde{\boldsymbol{\mu}}_k) \\
&= \min_{\theta} \frac{1}{2\tilde{\beta}_k} \left\| \boldsymbol{\mu}_\theta(\mathbf{x}^{(k)}, k) - \tilde{\boldsymbol{\mu}}_k(\mathbf{x}^{(k)}, \mathbf{x}^{(0)}) \right\|^2.
\end{aligned}$$

A.3 Reverse distribution based on the noise

Ho et al. (2020) show that predicting the mean of the reverse conditional distribution is equivalent to predict the noise. By plugging the reparametrized forward representation

into the mean of the reverse distribution (3.10) we get:

$$\begin{aligned}
\mu_q(\mathbf{x}^{(k)}, \mathbf{x}^{(0)}) &= \frac{\sqrt{\alpha_k(1 - \bar{\alpha}_{k-1})} \mathbf{x}^{(k)} + \sqrt{\bar{\alpha}_{k-1}(1 - \alpha_k)} \mathbf{x}^{(0)}}{1 - \bar{\alpha}_k} \\
&= \frac{\sqrt{\alpha_k(1 - \bar{\alpha}_{k-1})} \mathbf{x}^{(k)} + \sqrt{\bar{\alpha}_{k-1}(1 - \alpha_k)} \frac{\mathbf{x}^{(k)} - \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}}{\sqrt{\alpha_k}}}{1 - \bar{\alpha}_k} \\
&= \frac{\sqrt{\alpha_k(1 - \bar{\alpha}_{k-1})} \mathbf{x}^{(k)} + (1 - \alpha_k) \frac{\mathbf{x}^{(k)} - \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}}{\sqrt{\alpha_k}}}{1 - \bar{\alpha}_k} \\
&= \frac{\sqrt{\alpha_k(1 - \bar{\alpha}_{k-1})} \mathbf{x}^{(k)}}{1 - \bar{\alpha}_k} + \frac{(1 - \alpha_k) \mathbf{x}^{(k)}}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} - \frac{(1 - \alpha_k) \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} \\
&= \left(\frac{\sqrt{\alpha_k(1 - \bar{\alpha}_{k-1})}}{1 - \bar{\alpha}_k} + \frac{1 - \alpha_k}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} \right) \mathbf{x}^{(k)} - \frac{(1 - \alpha_k) \sqrt{1 - \bar{\alpha}_k}}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} \boldsymbol{\epsilon} \\
&= \left(\frac{\alpha_k(1 - \bar{\alpha}_{k-1})}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} + \frac{1 - \alpha_k}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} \right) \mathbf{x}^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k} \sqrt{\alpha_k}} \boldsymbol{\epsilon} \\
&= \frac{\alpha_k - \bar{\alpha}_k + 1 - \alpha_k}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} \mathbf{x}^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k} \sqrt{\alpha_k}} \boldsymbol{\epsilon} \\
&= \frac{1 - \bar{\alpha}_k}{(1 - \bar{\alpha}_k) \sqrt{\alpha_k}} \mathbf{x}^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k} \sqrt{\alpha_k}} \boldsymbol{\epsilon} \\
&= \frac{1}{\sqrt{\alpha_k}} \mathbf{x}^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k} \sqrt{\alpha_k}} \boldsymbol{\epsilon}.
\end{aligned}$$

Moreover, since the reverse process mean can be written as a function of the score (Luo 2022) derives (3.26):

$$\boldsymbol{\mu}(\mathbf{x}^{(k)}, \mathbf{x}^{(0)}) = \frac{1}{\sqrt{\alpha_k}} + \frac{1 - \alpha_k}{\sqrt{\alpha_k}} \nabla_{\mathbf{x}^{(k)}} \log q(\mathbf{x}^{(k)}).$$

A.4 Convergence Criteria

The training procedure is terminated according to an early-stopping rule based on the evolution of the objective function. Let $\mathcal{L}^{(i)}$ denote the mini-batch loss at iteration (or epoch) i from our training Algorithms 4 and 6. We define a criterion function $\text{crit}(\mathcal{L})$ that measures the improvement of the loss over iterations:

$$\text{crit}(\mathcal{L}) = \min_{j \in \{1, \dots, P\}} (\mathcal{L}^{(i-j)} - \mathcal{L}^{(i)}),$$

where P is a *patience parameter*. The algorithm proceeds as long as $\text{crit}(\mathcal{L}) > c_{\text{crit}}$, where $c_{\text{crit}} \geq 0$ is a tolerance level. In practice, this corresponds to stopping the training when the loss does not decrease sufficiently for P consecutive iterations.

A.5 Random Weight Denoising Networks

TimeGrad and ESNGrad have the following loss objective:

$$L_{\text{simple}}(\theta) = \sum_{t=t_0}^T \mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon, k} \left[\|\boldsymbol{\epsilon}_t - \widehat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1}, k)\|^2 \right],$$

where k is randomly drawn from discrete uniform $\mathcal{U}(\{1, \dots, K\})$. When we assign a RWNN $\widehat{\boldsymbol{\epsilon}}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1})$ to each diffusion step k the loss can be split into one objective for each neural network:

$$L_{\text{simple}}(\theta) = \sum_{t=t_0}^T \sum_{k=1}^K \mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon} \left[\|\boldsymbol{\epsilon}_t^{(k)} - \widehat{\boldsymbol{\epsilon}}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1})\|^2 \right],$$

where $\boldsymbol{\epsilon}_t^{(k)}$ is just the noise associated with diffusion step k . This yields the representation in (5.12). By taking the sum over the timesteps inside the expectation we get (5.13):

$$L_{\text{simple}}(\theta) = \sum_{k=1}^K \left(\mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon} \left[\sum_{t=t_0}^T \|\boldsymbol{\epsilon}_t^{(k)} - \widehat{\boldsymbol{\epsilon}}_{\theta_k}(\mathbf{x}_t^{(k)}, \mathbf{s}_{t-1})\|^2 \right] \right).$$

The ESN parameters are fixed given a drawn of the state parameters. Therefore, by substituting our denoising network with a RWDN as in (5.10) per each k we get:

$$L_{\text{simple}}(\theta) = \sum_{k=1}^K \left(\mathbb{E}_{\mathbf{x}_t^{(0)}, \epsilon} \left[\sum_{t=t_0}^T \|\boldsymbol{\epsilon}_t^{(k)} - W_k^\top \mathbf{z}_t^{(k)}\|^2 \right] \right).$$

Each problem can be solved individually since $\mathbf{z}_t^{(k)}$ is fixed in our problem. Therefore, using (5.17) we have the following minimization problem of the residuals:

$$\min_{W_k} \sum_{t=t_0}^T \left\| \boldsymbol{\epsilon}_t^{(k)} - W_k^\top \mathbf{z}_t^{(k)} \right\|^2 = \|E_k - Z_k W_k\|^2.$$

Taking the derivative with respect to W yields:

$$\begin{aligned} \frac{\partial}{\partial W_k} \|E_k - Z_k W_k\|^2 &= \mathbf{0}, \\ -2Z_k^\top (E_k - Z_k \widehat{W}_k) &= \mathbf{0}. \end{aligned}$$

This is the standard least squares solution for W_k :

$$\widehat{W}_k = (Z_k^\top Z_k)^{-1} Z_k^\top E_k.$$

The alternative solution is to use a penalized estimator for W_k as we did in (5.18). This offers more stability during training and converges to the standard OLS solution above when $\lambda \rightarrow 0$.

A.6 Additional Algorithms

Algorithm 8 : Conditional Wasserstein via kNN

Require: Real series $\{x_t\}_{t=1}^T$, generated series $\{\hat{x}_t\}_{t=1}^T$, lookback window V , number of conditioning points M , number of neighbors k

- 1: For both series, construct overlapping history-next observation tuples $(\mathbf{x}^{(t)}, x_t)$ and $(\hat{\mathbf{x}}^{(t)}, \hat{x}_t)$ for $t = V + 1, \dots, T$. Where:

$$\mathbf{x}^{(t)} = [x_{t-V}, \dots, x_{t-1}]^\top, \quad \hat{\mathbf{x}}^{(t)} = [\hat{x}_{t-V}, \dots, \hat{x}_{t-1}]^\top$$

- 2: Standardize all histories by the mean μ_x and standard deviation σ_x of the real series:

$$\tilde{\mathbf{x}}^{(t)} = \frac{\mathbf{x}^{(t)} - \mu_x}{\sigma_x}, \quad \tilde{\hat{\mathbf{x}}}^{(t)} = \frac{\hat{\mathbf{x}}^{(t)} - \mu_x}{\sigma_x}$$

- 3: **for** $m = 1, \dots, M$ **do**

- 4: Draw $i \sim \text{Uniform}\{V + 1, \dots, T\}$ and fix a conditioning $\mathbf{b}^m = \mathbf{x}^{(i)}$

- 5: Find the k time indices whose histories are closest to $\mathbf{b}^m$ in Euclidean distance:

$$\mathcal{N}_x^m = k\text{-argmin}_{t \neq i} \|\mathbf{b}^m - \mathbf{x}^{(t)}\|_2, \quad \mathcal{N}_{\hat{x}}^m = k\text{-argmin}_t \|\mathbf{b}^m - \tilde{\hat{\mathbf{x}}}^{(t)}\|_2$$

- 6: Collect the corresponding next observations in the tuple as empirical conditional samples:

$$\mathcal{S}_x^m = \{x_t : t \in \mathcal{N}_x^m\}, \quad \mathcal{S}_{\hat{x}}^m = \{\hat{x}_t : t \in \mathcal{N}_{\hat{x}}^m\}$$

- 7: Compute the Wasserstein-1 distance between the two empirical samples:

$$W^m = W_1(\mathcal{S}_x^m, \mathcal{S}_{\hat{x}}^m)$$

- 8: **end for**

- 9: **return** $\widehat{W} = \frac{1}{M} \sum_{m=1}^M W^m$
-

Algorithm 9 : One-step ahead prediction with Time Series Diffusion Models

Require: Training data $\{x_t\}_{t=1}^{t_s}$, test data $\{x_t\}_{t=t_s+1}^T$, number of samples n per step, trained denoising network $\widehat{\epsilon}(\cdot)$ according to the model specifications (Algorithm 4, 6 or 7), ESN network state $\mathbf{s}_{t_s}$ or RNN states, quantiles $\mathcal{A} = \{0.05, 0.25, 0.75, 0.95\}$

- 1: **for** $t = t_s + 1, \dots, T$ **do**
- 2: **for** $i = 1, \dots, n$ **do**
- 3: Sample $\widehat{x}_t^{(i)} \sim p(x_t \mid \mathbf{s}_{t-1})$ or using Algorithm 5
- 4: **end for**
- 5: Estimate the conditional mean

$$\widehat{\mu}_t = \frac{1}{n} \sum_{i=1}^n \widehat{x}_t^{(i)}$$

- 6: Estimate the empirical distribution:

$$\widehat{F}_t(y) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}\{\widehat{x}_t^{(i)} \leq y\}$$

- 7: **for** $\alpha \in \mathcal{A}$ **do**
- 8: Compute the empirical quantile:

$$\widehat{q}_\alpha^{(t)} = \inf \left\{ \widehat{x} : \widehat{F}_t(\widehat{x}) \geq \alpha \right\}$$

- 9: **end for**
- 10: Update $\mathbf{s}_t$ using the true realization x_t :

$$\mathbf{s}_t = \Phi_\theta(x_t, \mathbf{s}_{t-1})$$

- 11: **end for**
 - 12: **return** $\{\widehat{\mu}_t, \widehat{q}_{0.05}^{(t)}, \widehat{q}_{0.25}^{(t)}, \widehat{q}_{0.75}^{(t)}, \widehat{q}_{0.95}^{(t)}\}_{t=t_s}^T$
-

Appendix B: Figures and Tables

Lag L	Diffusion step k				
	$k = 10$	$k = 25$	$k = 50$	$k = 75$	$k = 100$
1	0.0000	0.0000	0.0000	0.0009	0.8022
5	0.0000	0.0000	0.0000	0.0016	0.9154
10	0.0000	0.0000	0.0000	0.0019	0.6454

Table B1: Ljung-Box p -values at different diffusion steps k and lags L . Values above 0.05 (in bold) indicate failure to reject the white noise hypothesis.

Parameter	ESNGrad	RWDN
<i>Training Parameters</i>		
Diffusion steps K	100	100
Variance schedule $\{\beta_k\}$	linear	linear
Batch size B	64	–
Patience P	50	–
Learning rate η	10^{-3}	–
Lookback window t_0	100	100
<i>Architecture Parameters</i>		
Denoising network	MLP	RWNN
RNN / Reservoir type	ESN	ESN
Hidden size H	25	25
Denoising hidden size R	[32,64,32]	50
<i>Hyperparameters</i>		
Spectral radius ρ	0.9	0.9
Input scaling γ	2.5	0.1
Shift scaling ω	0	0
Bias scaling ϱ	–	1.5
Leak rate α	0.2	0.2
Matrix scaling ν	–	3
Ridge penalty λ	–	10^{-4}
<i>Random Parameters</i>		
A_k	–	$\mathcal{N}(0, 1)$
$\mathbf{b}_k$	–	$\mathcal{U}(-1, 1)$
C	$\mathcal{N}(0, 1)$	$\mathcal{N}(0, 1)$
A	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$
ζ	–	–
<i>Sparsity</i>		
Sparsity of A_k	–	0
Sparsity of $\mathbf{b}_k$	–	0
Sparsity of C	0	0
Sparsity of A	0.05	0.05
Sparsity of ζ	–	–

Table B2: Models configurations and hyperparameter settings for the AR(1) and GARCH(1,1) experiments.

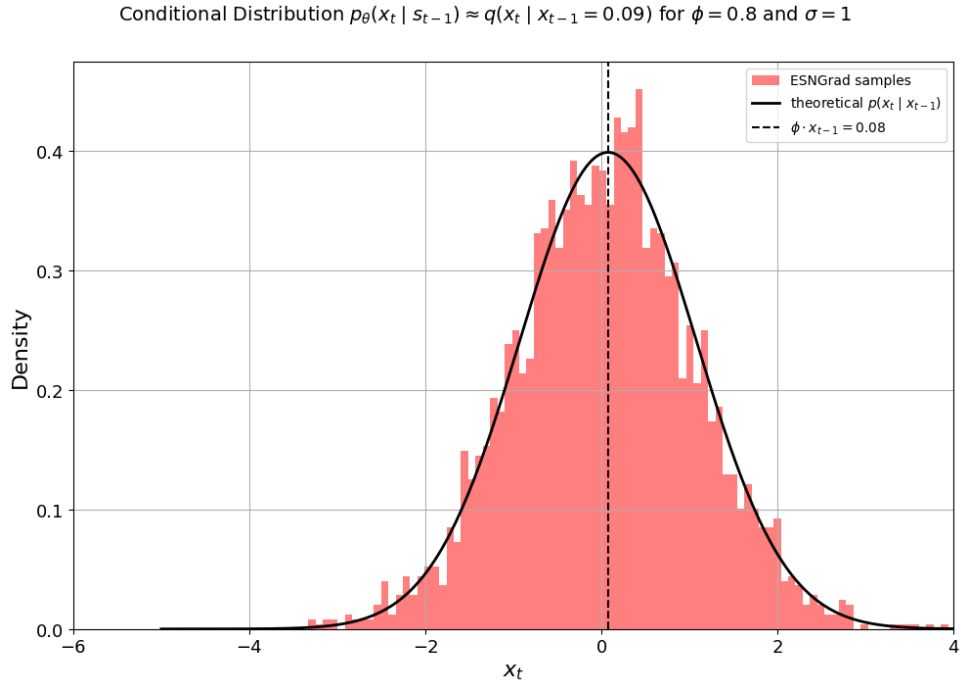
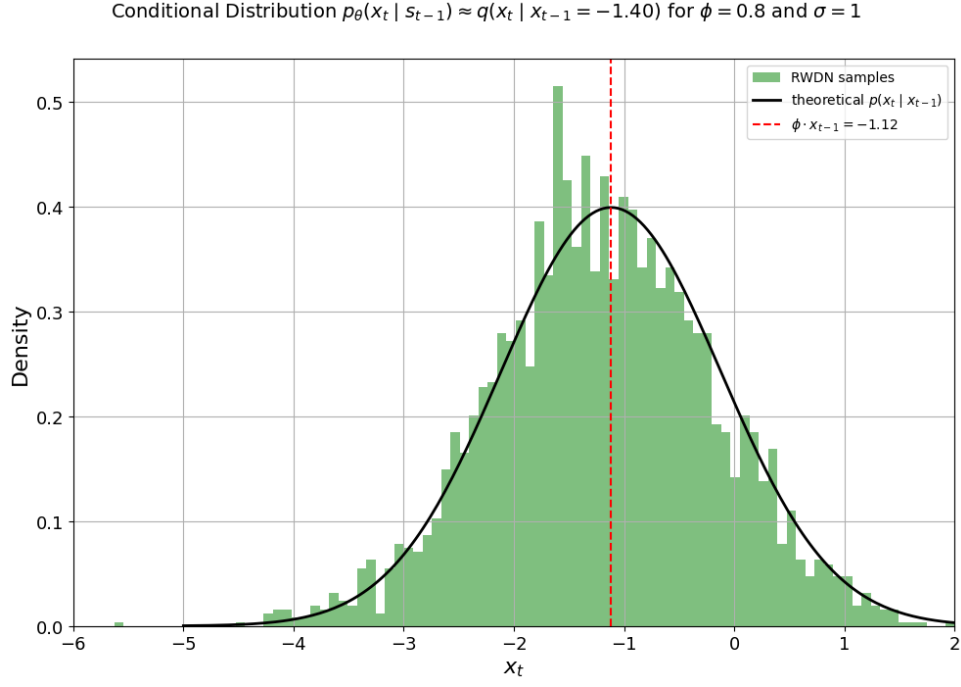


Figure B1: Histograms for ESNGrad (red) and RWDN (green) samples against the theoretical conditional distribution for some fix conditioning.

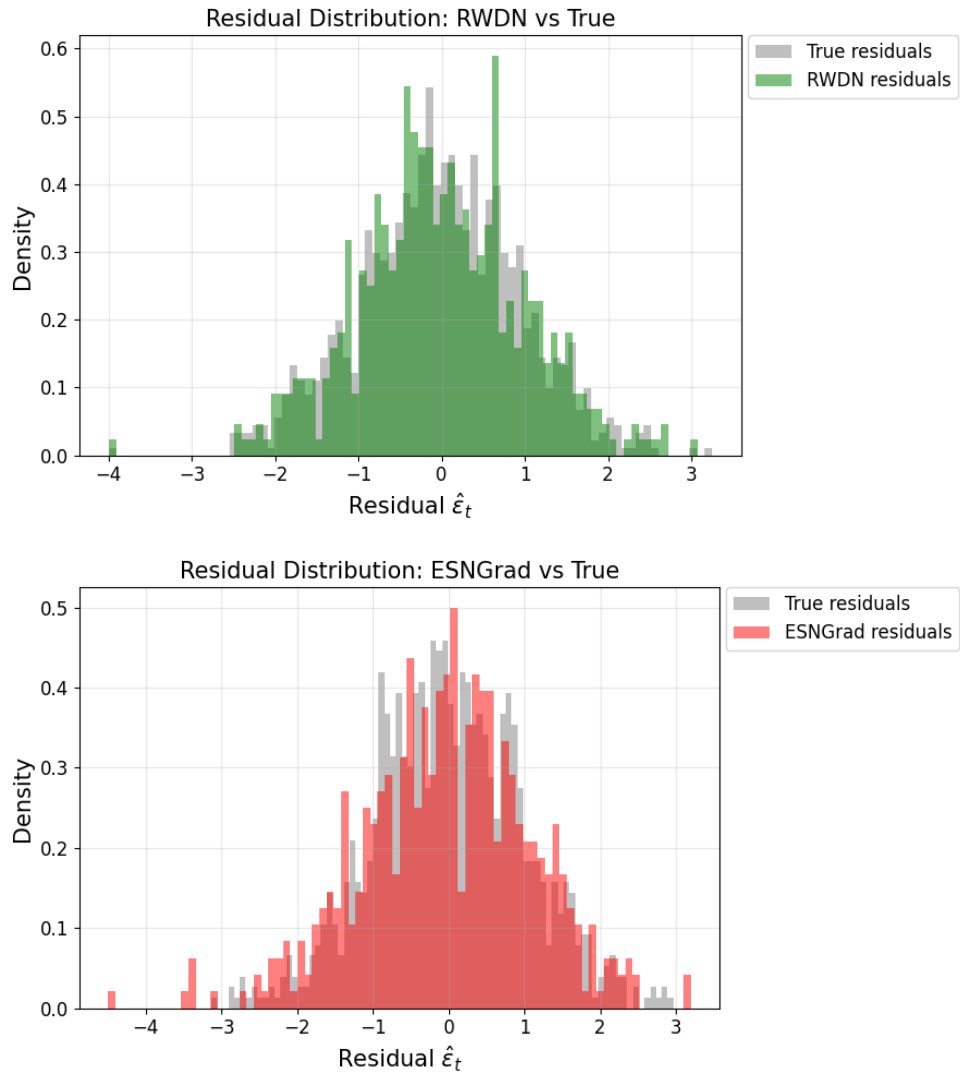


Figure B2: Histograms for ESNGrad (red) and RWDN (green) residuals

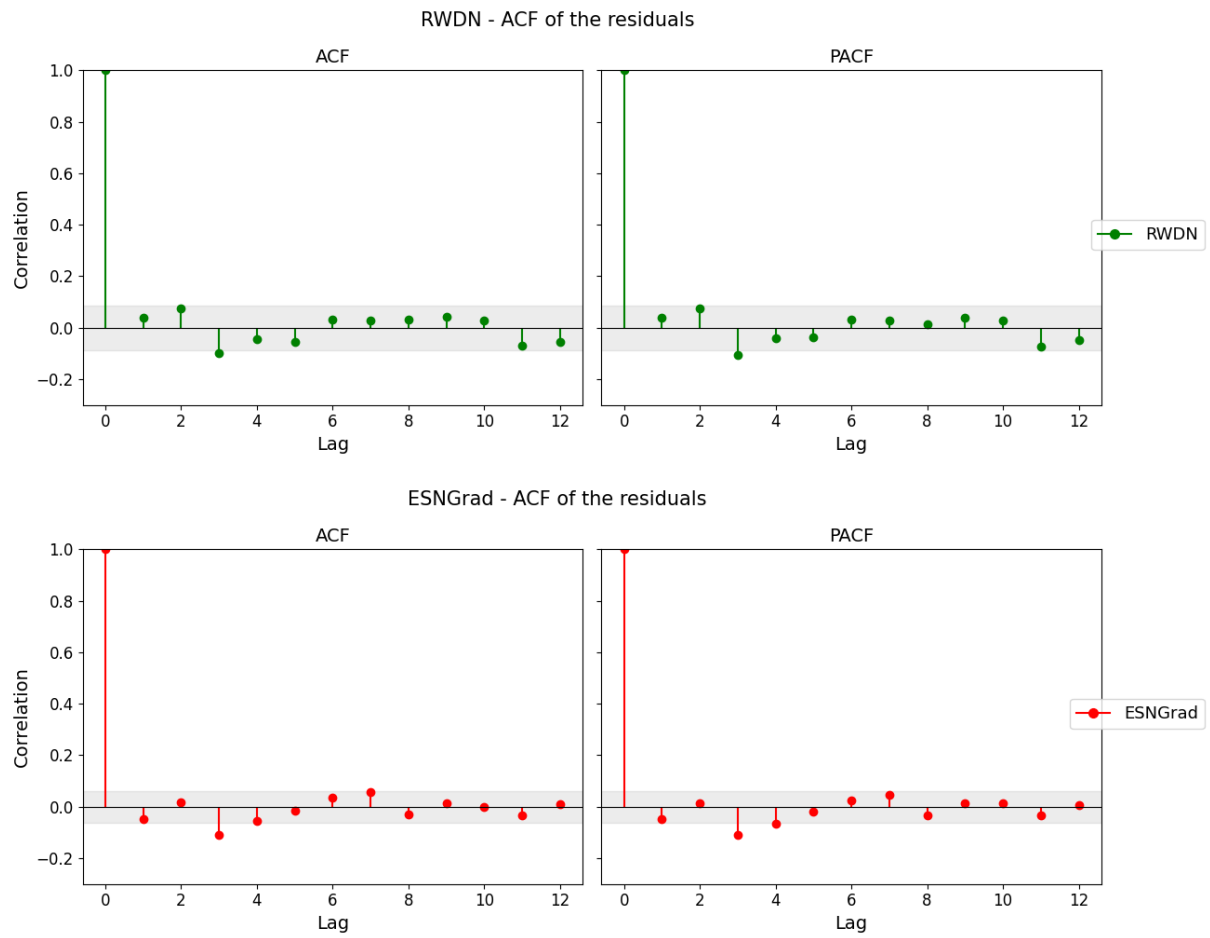


Figure B3: ACF and PACF plots for the AR(1) residuals of our Generative Models. The gray area shows the confidence bounds.

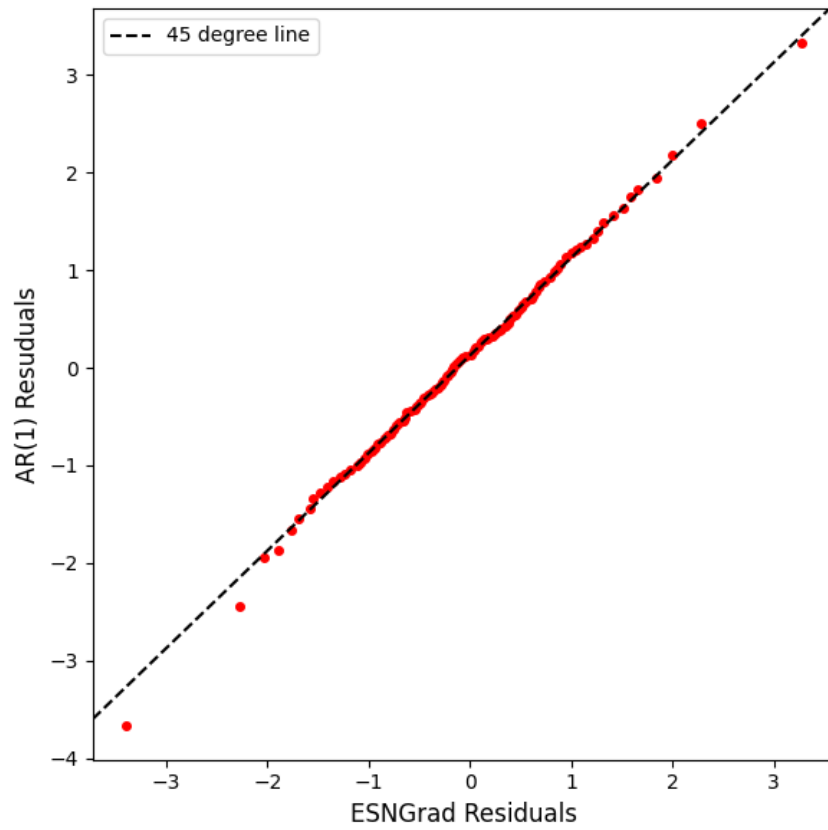
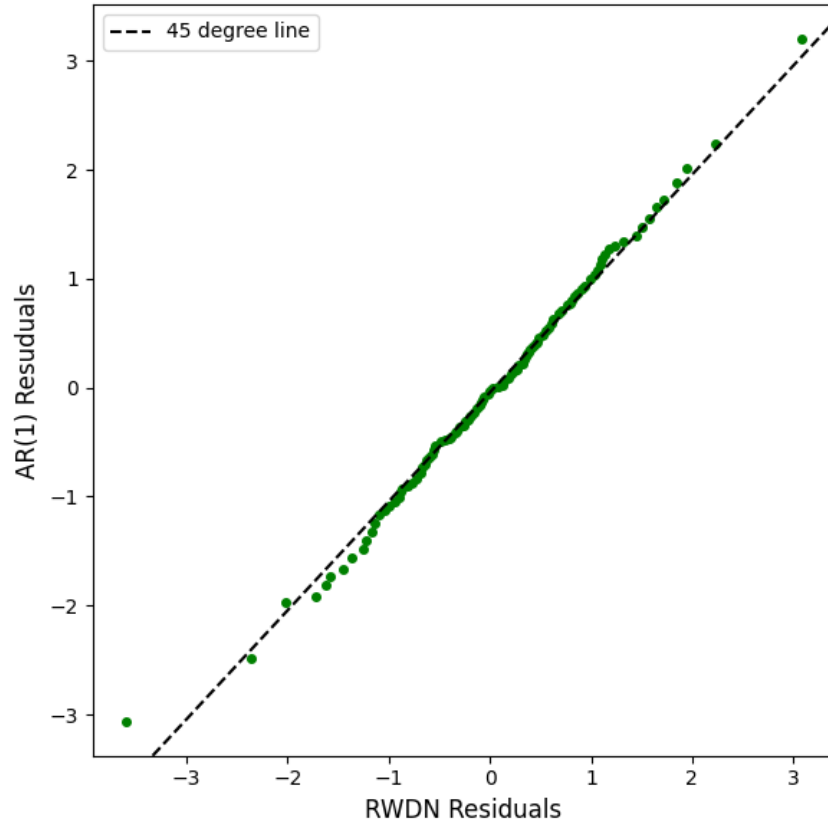


Figure B4: QQ-Plots of the AR(1) residuals. The quantiles of our residuals match with the quantiles of the $\mathcal{N}(0, 1)$.

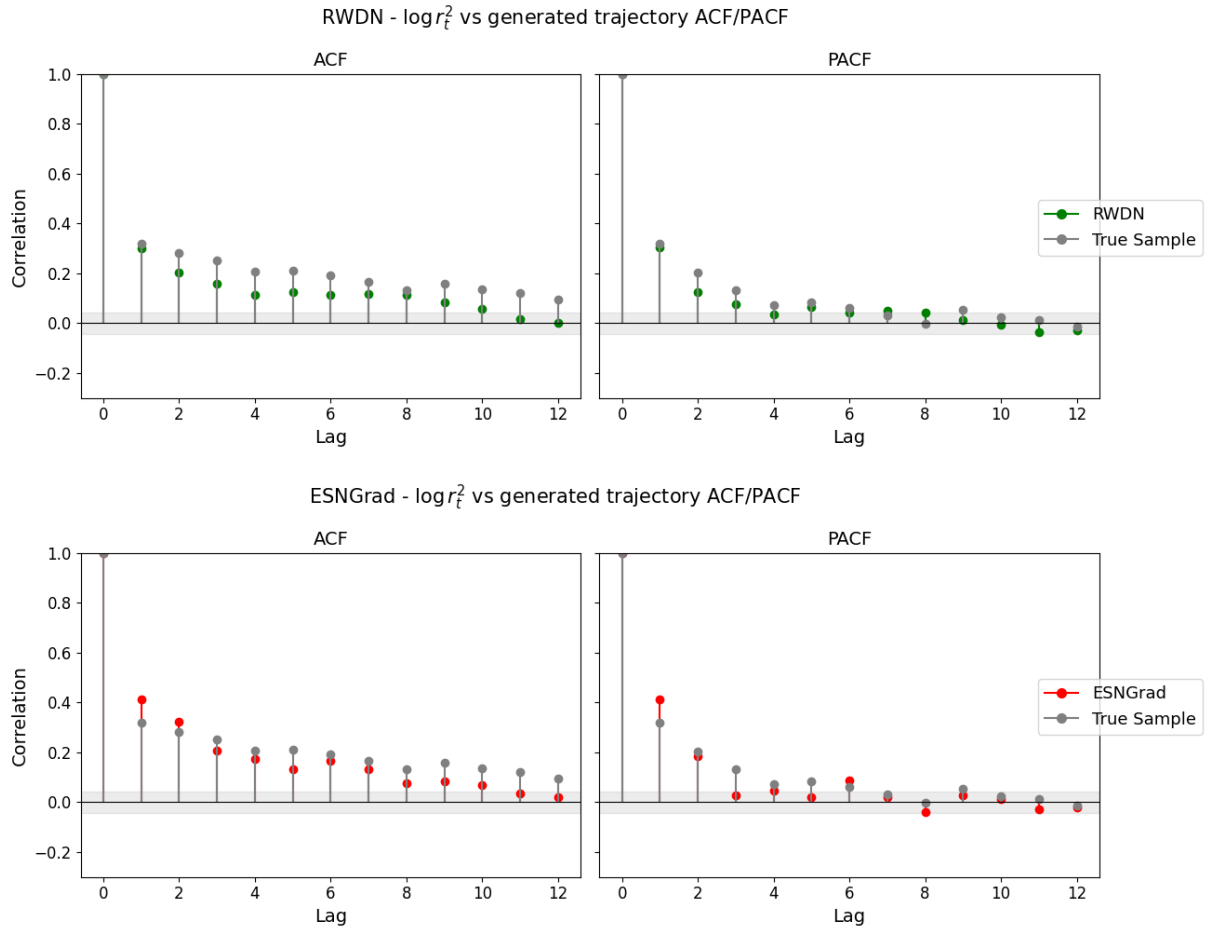


Figure B5: ACF and PACF plots for the trajectories generated by our time series diffusion models against the in-sample autocorrelation structure of $\log(r_t^2)$ with r being a GARCH(1,1) process with $\alpha + \beta = 0.95$. The gray area shows the confidence bounds.

Parameter	ESNGrad	ESN-RWDN	TimeGrad
<i>Training Parameters</i>			
Diffusion steps K	100	100	100
Variance schedule $\{\beta_k\}$	linear	linear	linear
Batch size B	64	–	64
Patience P	25	–	25
Learning rate η	10^{-3}	–	10^{-3}
Lookback window t_0	100	100	100
<i>Architecture Parameters</i>			
Denoising network	MLP	RWNN	MLP
RNN / Reservoir type	ESN	ESN	LSTM
Hidden size H	25	25	16
Denoising hidden size R	[32,64,32]	50	[32,64,32]
<i>Hyperparameters</i>			
Spectral radius ρ	0.9	0.9	–
Input scaling γ	2.5	0.1	–
Shift scaling ω	0	0	–
Bias scaling ϱ	–	–	–
Matrix scaling ν	–	–	–
Lookback L	–	–	32
Ridge penalty λ	–	10^{-4}	–
<i>Random Parameters</i>			
A_k	–	$\mathcal{N}(0, 1)$	–
$\mathbf{b}_k$	–	$\mathcal{U}(-1, 1)$	–
C	$\mathcal{N}(0, 1)$	$\mathcal{N}(0, 1)$	–
A	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$	–
ζ	–	–	–
<i>Sparsity</i>			
Sparsity of A_k	–	0.1	–
Sparsity of $\mathbf{b}_k$	–	0	–
Sparsity of C	0	0	–
Sparsity of A	0.05	0.1	–
Sparsity of ζ	–	–	–

Table B3: Models configurations and hyperparameter settings for the probabilistic forecasting example with US CPI data

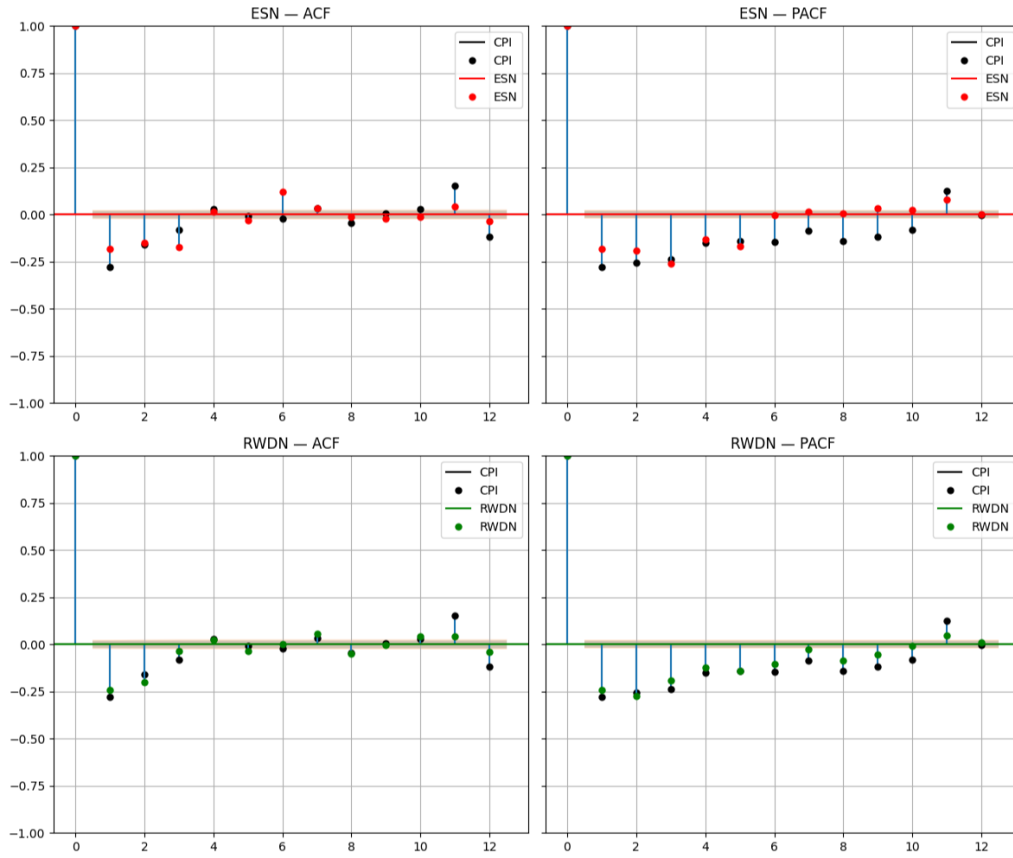


Figure B6: In-Sample ACF and PACF of the generative models against the true autocorrelation structure the first difference of annualized inflation $\Delta_1\pi$

Appendix C: Additional Figures

C.1 Model Architectures

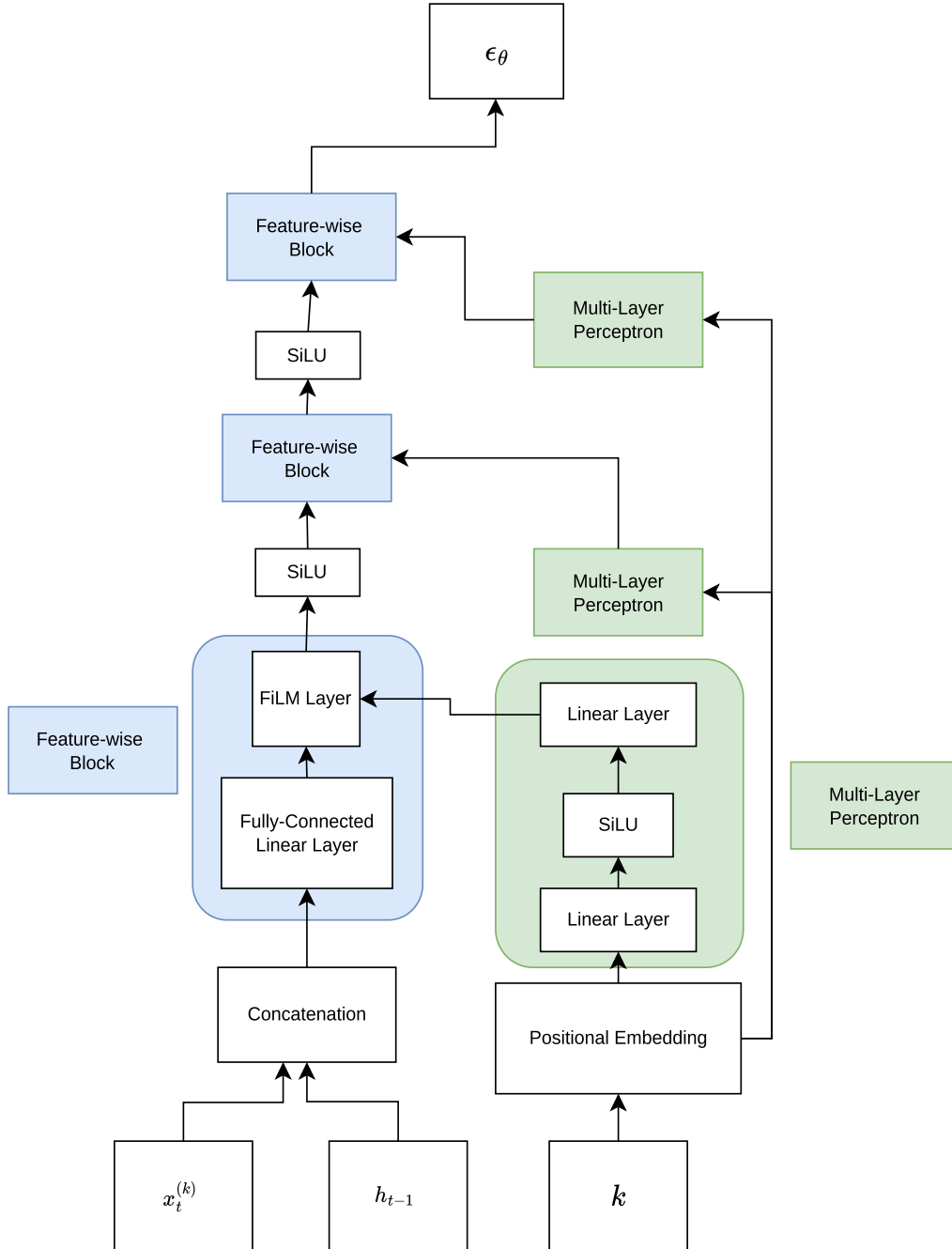


Figure C1: The denoising network $\hat{\epsilon}_\theta(\mathbf{x}_t^{(k)}, \mathbf{h}_{t-1}, k)$ of our TimeGrad and ESNGrad models using MLP to learn the positional time embedding of Vaswani et al. (2017). Own illustration.

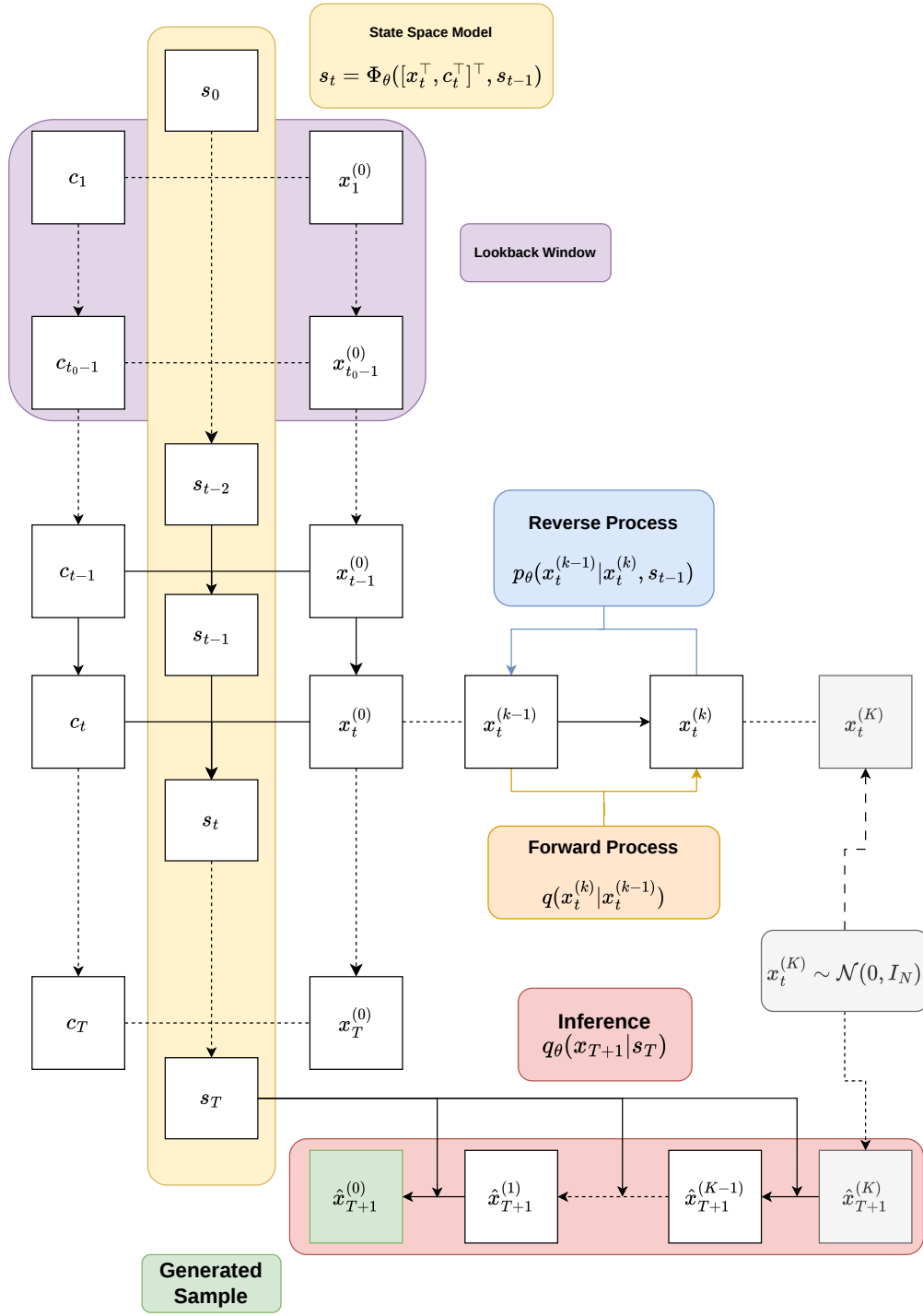


Figure C2: Schema of the time series diffusion models. Own illustration.

Appendix D: Declaration of Auxiliary tools

Aid / Tool	Purpose
ChatGPT	Help with spelling and grammar, help with LaTeX tables, figures, equation formatting and alignment and BibTeX
ClaudeAI	Debugging Python code
Python	DDPM code
OverLeaf AI assistant	Help with grammar mistakes
VSCode	IDE for Python
Draw.io	Flowchart for TimeGrad model and MLP
Inkscape	Help to format RWNN/ANN figure of Chapter 6
Overleaf	LaTeX editor

Appendix E: Disclosures

Figure 7 was originally provided by Dr. Giovanni Ballarin and has been adapted for the purposes of this work.

Appendix F: Declaration of Authorship

“I hereby declare,

- that I have written this thesis independently,
- that I have written the thesis using only the aids specified in the index;
- that all parts of the thesis produced with the help of aids have been declared;
- that I have handled both input and output responsibly when using AI. I confirm that I have therefore only read in public data or data released with consent and that I have checked, declared and comprehensibly referenced all results and/or other forms of AI assistance in the required form and that I am aware that I am responsible if incorrect content, violations of data protection law, copyright law or scientific misconduct (e.g. plagiarism) have also occurred unintentionally;
- that I have mentioned all sources used and cited them correctly according to established academic citation rules;
- that I have acquired all immaterial rights to any materials I may have used, such as images or graphics, or that these materials were created by me;
- that the topic, the thesis or parts of it have not already been the object of any work or examination of another course, unless this has been expressly agreed with the faculty member in advance and is stated as such in the thesis;
- that I am aware of the legal provisions regarding the publication and dissemination of parts or the entire thesis and that I comply with them accordingly;
- that I am aware that my thesis can be electronically checked for plagiarism and for third-party authorship of human or technical origin and that I hereby grant the University of St.Gallen the copyright according to the Examination Regulations as far as it is necessary for the administrative actions;
- that I am aware that the University will prosecute a violation of this Declaration of Authorship and that disciplinary as well as criminal consequences may result, which may lead to expulsion from the University or to the withdrawal of my title.” By submitting this thesis, I confirm through my conclusive action that I am submitting the Declaration of Authorship, that I have read and understood it, and that it is true.

JR, 18.05.2026